

3. Software-Struktur

Overwatch

21 August, 2026

Table of Contents

3.1 Systemüberblick	4
3.2 Datenmodell	5
3.3 White-Box-Sicht	6
3.4 Nutzerrollen-Konzept	7
3.5 Externe Schnittstellen des Systems	8
3.6 Interne Schnittstellen des Systems	9
3.1 Systemüberblick	10
VPN-Route	10
RTSP - Videostream der Drohne an den Gefechtsstand übertragen	10
MQTT - Die Telemetry-Daten der Drohne an den Gefechtsstand übertragen	11
HTTPS - Übertragen der Web-Anwendung "Overwatch-UI" an die Clients	12
HTTPS - Übertragung der Telemetry-Daten an die Clients	13
WebRTC - Übertragen des Videostreams an die Clients	15
HLS - Übertragung des Videostreams an die Clients	16
3.2 Datenmodell	18
A. Overwatch - Telemetry Data Adapter - Datenmodell	18
3.2.1 White-Box View Level 1	18
3.2.2 Prozesse	20
3.2.3 Konzepte	32
3.3 White-Box-Sicht	36
A. Overwatch - Telemetry Data Adapter - White-Box View	36
Main Module	36
Buffer Manager	40
Process-Worker	42
Persistenter Nachrichtenspeicher	50
ErrorHandling Package	64
Identifizier-Package	69
Validator-Package	72
DeliveryService-Package	78

Database-Package.....	89
B. Overwatch-Service - White-Box View	90
C. Overwatch-UI - White-Box View	90
D. Overwach Topic Routing Plugin - White-Box View	90
3.4 Nutzer-/Rollenkonzept	91
Rollen.....	91
Rollen-/Rechtematrix.....	91

3.1 Systemüberblick

3.2 Datenmodell

3.3 White-Box-Sicht

3.4 Nutzerrollen-Konzept

3.5 Externe Schnittstellen des Systems

3.6 Interne Schnittstellen des Systems

3.1 Systemüberblick

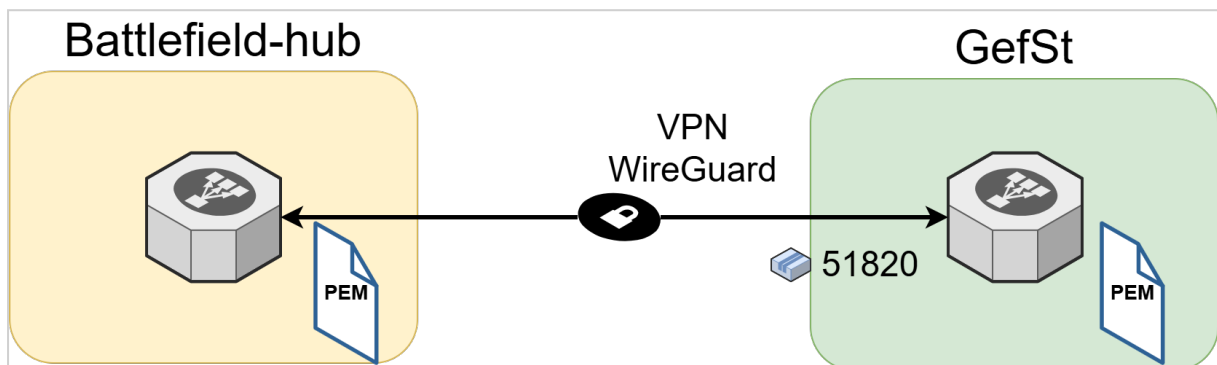
Der "Overwatch Telemetry Data Adapter" ist eine herstellerunabhängige Komponente, welche die MQTT-basierten Telemetriedaten (Message Queuing Telemetry Transport) verschiedener Drohnentypen/-Hersteller identifiziert, validiert und in ein eigens dafür definiertes Telemetry-Datenformat konvertiert. Sinn und Zweck dieser Komponente ist es, die verschiedenen herstellerspezifischen Datenstrukturen zu vereinheitlichen und somit die Daten für die Verarbeitung in der Anwendung "Overwatch" nutzbar zu machen. Auf diese Weise soll der Anschluss neuer Drohnentypen an die Anwendung Overwatch möglichst modular und einfach gestaltet werden. Er arbeitet in einer Publisher/Subscriber-Architektur zusammen mit dem "Eclipse Mosquitto MQTT Message Broker" und erzeugt für jede akzeptierte MQTT-Nachricht zwei miteinander verknüpfte Pfade:

- den **Zustellpfad für Originaltelemetrie**, der die unveränderte Telemetrie einer Drohne an den Overwatch-Service übermittelt;
- den **Verarbeitungspfad für transformierte Telemetrie**, der Art und Modell einer Drohne identifiziert, deren Telemetrierohdaten validiert und transformiert und nach erneuter Validierung die transformierte Telemetrie an den Overwatch-Service übermittelt.

Die Architektur ist als einfache und schlanke Node.js-Anwendung ausgelegt.

VPN-Route

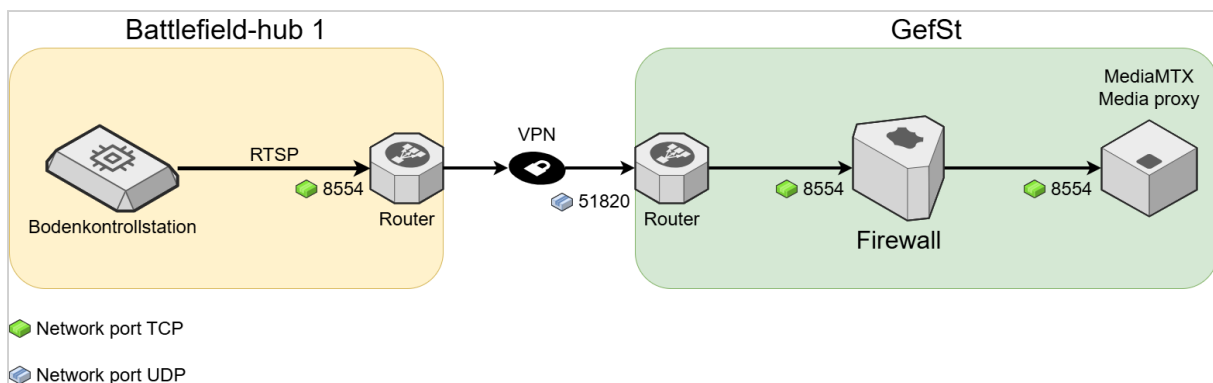
Die VPN-Verbindung zwischen einem Battlefield-hub und dem Gefechtsstand wird durch einen VPN-Tunnel abgesichert. Als Produkt zum hier "WireGuard" zum Einsatz. Zum Aufbau der Verbindung benötigt jeder Router ein x509 Zertifikat.



RTSP - Videostream der Drohne an den Gefechtsstand übertragen

Mit Hilfe des RTSP (Real-Time Streaming Protocol) wird der Videostream, welchen die Drohne aufzeichnet über deren BKS (Bodenkontrollstation) an den MediaMTX Container übertragen. Hier wird RTSPS (RTSP-Secure oder RTSP over TLS) genutzt, um die Verbindung verschlüsseln zu können.

Von	Nach	Protokoll	Port	Protokoll
Bodenkontrollstation	Router Bfh 1	RTSP over TLS	8554	TCP
Router Bfh1	Router GefSt	VPN	51820	UDP
Router GefSt	Firewall	RTSP over TLS	8554	TCP
Firewall	MediaMTX Media Proxy	RTSP over TLS	8554	TCP



MQTT - Die Telemetry-Daten der Drohne an den Gefechtsstand übertragen

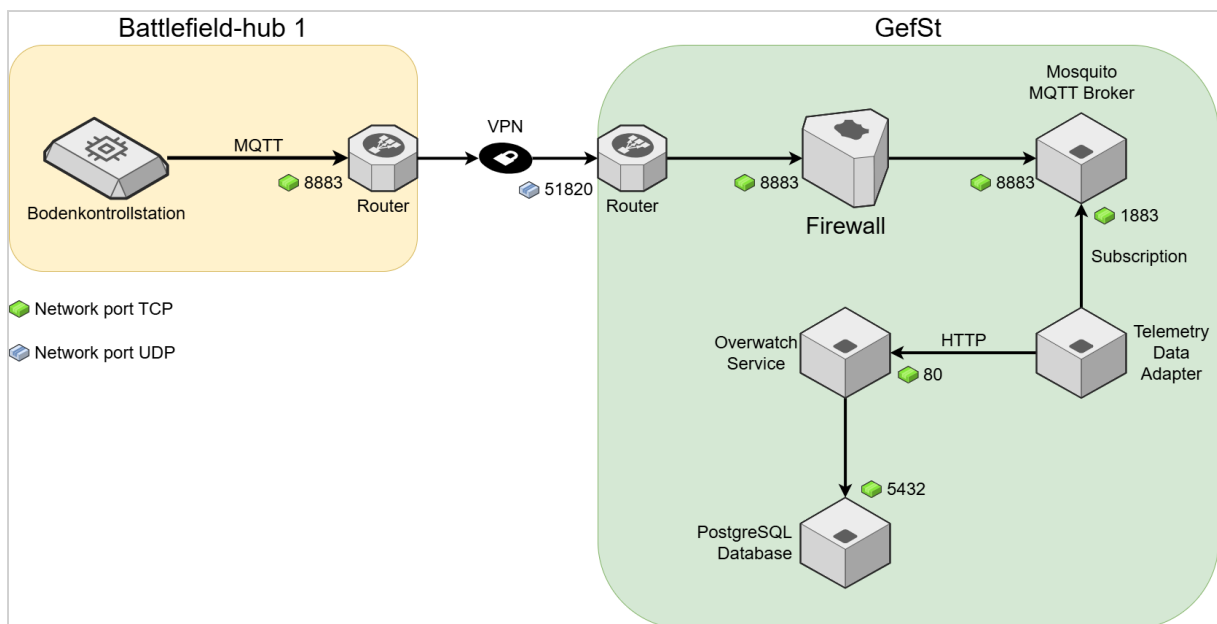
Das Protokoll MQTT (Message Queuing Telemetry Transport) dient zur Übertragung der Telemetry-Daten der Drohne an den Gefechtsstand. Um die Verbindung zu sichern wird hier MQTT over TLS (verschlüsseltes MQTT) genutzt.

Die BKS empfängt die Daten von der Drohne und sendet diese über den VPN-Tunnel an den Gefechtsstandserver. Dort werden die Daten durch den "Mosquitto MQTT Broker" empfangen. Der Mosquitto puffert die Daten.

Der Telemetry Data Adapter meldet sich am Mosquitto Broker an (Subscription) und empfängt somit die MQTT-Pakete. Er entpackt die Telemetry-Daten, welche im JSON-Format vorliegen. Da jede Drohne unterschiedliche Telemetry-Formate nutzt, ist der Adapter dafür zuständig, die unterschiedlichen Formate in ein einheitliches Format zu transformieren und die Daten an den Overwatch-Service weiterzugeben. Der Overwatch-Service dient als Schnittstelle zur Client-Software.

Von	Nach	Protokoll	Port	Protokoll
Bodenkontrollstation	Router Bfh 1	MQTT over TLS	8883	TCP
Router Bfh1	Router GefSt	VPN	51820	UDP

Von	Nach	Protokoll	Port	Protokoll
Router GefSt	Firewall	MQTT over TLS	8883	TCP
Firewall	Mosquitto Broker	MQTT over TLS	8883	TCP
Mosquitto Broker	TelemetryDataAdapter	MQTT	1883	TCP
TelemetryDataAdapter	Overwatch-Service	HTTP	80	TCP



HTTPS - Übertragen der Web-Anwendung "Overwatch-UI" an die Clients

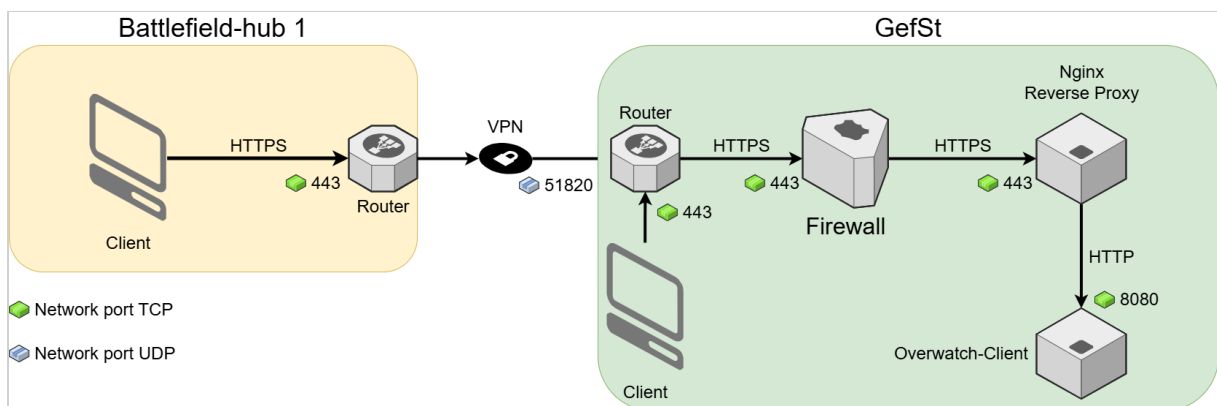
Bei "Overwatch-Client" handelt es sich um eine webbasierte Anwendung, welche mittels HTTPS an den Client im Battlefield-hub und im Gefechtsstand übertragen wird.

Mittels HTTPS wird die Anforderung des Clients im Battlefield-Hub, durch den VPN-Tunnel zum Gefechtsstand geleitet.

Dort nimmt der Reverse Proxy diese entgegen. Er terminiert die HTTPS-Verbindung und öffnet eine HTTP-Verbindung zum Webserver, dem "Overwatch-Client".

Der Webserver liefert die Web-Anwendung an den Battlefield-hub-Client aus. Auf ähnlichem Wege wird die Web-Anwendung auch an die Clients im GefSt übertragen.

Von	Nach	Protokoll	Port	Protokoll
Client Bfh 1	Router Bfh 1	HTTPS	443	TCP
Router Bfh1	Router GefSt	VPN	51820	UDP
Router GefSt	Firewall	HTTPS	443	TCP
Firewall	Nginx Reverse Proxy	HTTPS	443	TCP
Nginx Reverse Proxy	Overwatch-Client	HTTP	8080	TCP



HTTPS - Übertragung der Telemetry-Daten an die Clients

Die Telemetry-Daten, welche von den Drohnen erzeugt werden, müssen an die Web-Anwendung übertragen werden, um eine Kartendarstellung erzeugen zu können.

Es werden JSON-basierte Datensätze an die Clients übertragen.

Mittels HTTPS wird die Anforderung des Clients im Battlefield-Hub, durch den VPN-Tunnel zum Gefechtsstand geleitet.

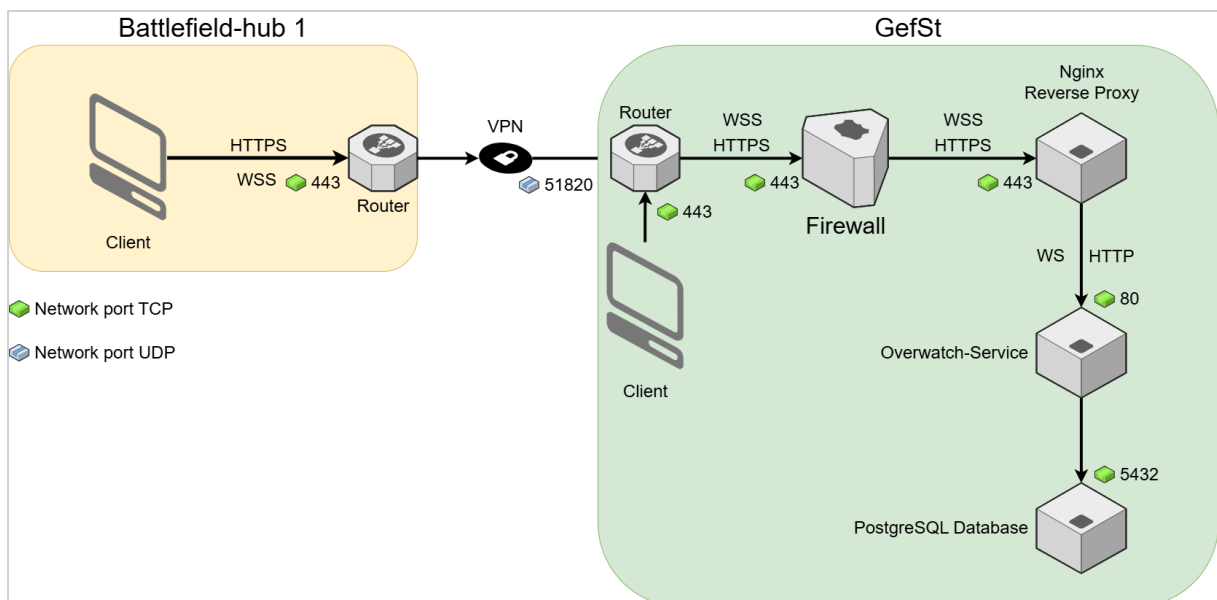
Dort nimmt der Reverse Proxy diese entgegen. Er terminiert die HTTPS-Verbindung und öffnet eine HTTP-Verbindung zur REST-Schnittstelle des Overwatch-Service.

Der Overwatch-Service nimmt die Anfrage entgegen und holt den entsprechenden Datensatz aus der PostgreSQL Datenbank.

Für ein Liveupdate der Telemetry-Daten (dauerhafte Versorgung des Clients mit aktuellen Telemetry-Daten) muss der Client eine Websocket-Verbindung zum Overwatch-Service öffnen. Bei dieser Art der Verbindung kann der Overwatch-Service Daten mittels Push an den Client senden. Dadurch wird vermieden, dass der Client ständig Anfragen zu den Telemetry-Daten senden muss.

Aus Sicherheitsgründen wird bis zum Reverse Proxy mit WebSocket Secure (WSS) gearbeitet.

Von	Nach	Protokoll	Port	Protokoll
Client Bfh 1	Router Bfh 1	HTTPS	443	TCP
Client Bfh 1	Router Bfh 1	Websocket (WSS)	443	TCP
Router Bfh1	Router GefSt	VPN	51820	UDP
Router GefSt	Firewall	HTTPS	443	TCP
Router GefSt	Firewall	Websocket (WSS)	443	TCP
Firewall	Nginx Reverse Proxy	HTTPS	443	TCP
Firewall	Nginx Reverse Proxy	Websocket (WSS)	443	TCP
Nginx Reverse Proxy	Overwatch-Service	HTTP	80	TCP
Nginx Reverse Proxy	Overwatch-Service	Websocket (WS)	80	TCP
Overwatch-Service	PostgreSQL Database	PostgreSQL Wire Protocol	5432	TCP



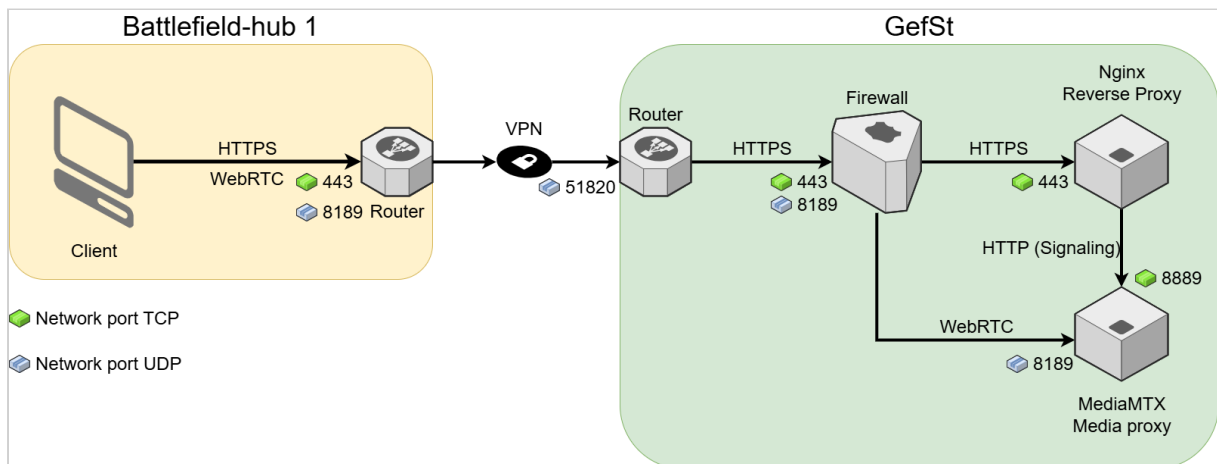
WebRTC - Übertragen des Videostreams an die Clients

Das Protokoll WebRTC ist eines von zwei Protokollen, mit dessen Hilfe der Videostream einer Drohne an die Clients übertragen werden kann. WebRTC bietet eine sehr geringe Latenz bei der Datenübertragung, weil es eine Peer-to-Peer Architektur nutzt. Der Client fordert den Videostream mittels HTTP(S) an und muss dann eine direkte Verbindung zum Medienserver (hier ist dies der MediaMTX Media Proxy) aufbauen. Der Nachteil dieser Architektur ist, dass in der Firewall ein zusätzlicher Port (8189 UDP) geöffnet werden muss.

Mittels HTTPS wird die Anforderung des Clients im Battlefield-Hub zum Gefechtsstand übertragen. Dort nimmt der Reverse Proxy diese entgegen. Er terminiert die HTTPS-Verbindung und öffnet eine HTTP-Verbindung zum MediaMTX Media Proxy um ihm die Daten weiterzuleiten.

Der MediaMTX kommuniziert mit dem Client und vereinbart mit diesem die Übertragung des Streams über Port 8189. Der Client öffnet eine neue Verbindung zum MediaMTX über Port 8189. Diese wird durch den VPN-Tunnel zum Gefechtsstand und dort direkt zum MediaMTX Media Proxy weitergeleitet.

Von	Nach	Protokoll	Port	Protokoll
Client Bfh 1	Router Bfh 1	HTTPS	443	TCP
Router Bfh1	Router GefSt	VPN	51820	UDP
Router GefSt	Firewall	HTTPS	443	TCP
Firewall	Nginx Reverse Proxy	HTTPS	443	TCP
Nginx Reverse Proxy	MediaMTX Media Proxy	HTTP	8889	TCP
Client Bfh 1	Router Bfh 1	WebRTC / ICE	8189	UDP
Router Bfh1	Router GefSt	VPN	51820	UDP
Router GefSt	Firewall	WebRTC / ICE	8189	UDP
Firewall	MediaMTX Media Proxy	WebRTC / ICE	8189	UDP

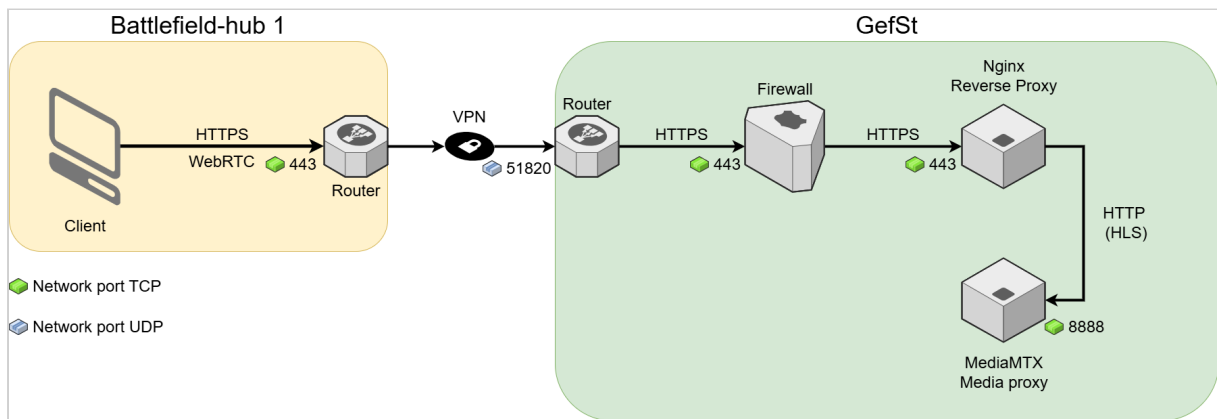


HLS - Übertragung des Videostreams an die Clients

Genau wie WebRTC ermöglicht HLS die Übertragung des Videostreams der Drohne an die Clients. Im Unterschied zu WebRTC kann HLS vollständig über den Nginx Reverse Proxy geleitet werden, so dass weniger Ports in der Firewall geöffnet werden müssen. Nachteilig wirkt sich aus, dass die Übertragungslatenz von HLS wesentlich größer ist als die von WebRTC.

Mittels HTTPS wird die Anforderung des Clients an den Battlefield-Hub übertragen. Dort nimmt der Reverse Proxy diese entgegen. Der Reverse Proxy öffnet eine HTTP-Verbindung zum HLS-Endpunkt des MediaMTX Media Proxy auf Port 8888. MediaMTX liefert die HLS-Daten über dieselbe TCP-Verbindung zurück. Der Reverse Proxy überträgt sie anschließend über die bestehende HTTPS-Verbindung an den Client.

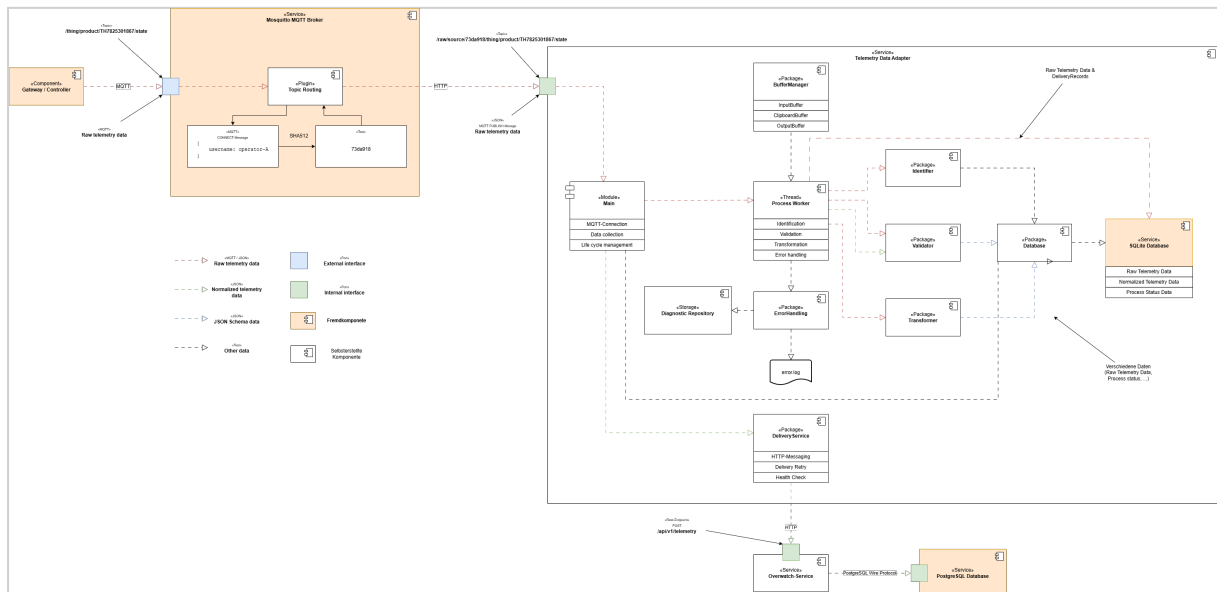
Von	Nach	Protokoll	Port	Protokoll
Client Bfh 1	Router Bfh 1	HTTPS	443	TCP
Router Bfh1	Router GefSt	VPN	51820	UDP
Router GefSt	Firewall	HTTPS	443	TCP
Firewall	Nginx Reverse Proxy	HTTPS	443	TCP
Nginx Reverse Proxy	MediaMTX Media Proxy	HLS over HTTP	8888	TCP



3.2 Datenmodell

A. Overwatch - Telemetry Data Adapter - Datenmodell

3.2.1 White-Box View Level 1



Das Gateway (Steuereinheit) einer Drohne schickt Telemetrierohdaten im MQTT-Format an den Mosquitto MQTT Broker. Innerhalb des Mosquitto wird der eingehende Topic durch das "Topic Routing"-Plugin umgeschrieben. Es liest den Username aus den MQTT-Paketdaten und generiert mit Hilfe von HMAC-SHA-512 einen stabilen Schlüssel, welcher zur eindeutigen Identifikation eines Controllers geeignet ist (es wird hier davon ausgegangen, dass Nutzernamen/Passwörter als Authentifizierungsmethode genutzt werden wird). Dieser Schritt ist notwendig, da der Username lediglich im Verbindungsaufbau (CONNECT-Message) gesendet wird. Diese Nachricht wird aber nicht an die Subscriber (Telemetry Data Adapter) weitergeleitet. Der Topic wird mit dem generierten Schlüssel ergänzt. Aus `thing/product/TH7825301867/state` wird `raw/source/73da918/thing/product/TH7825301867/state`. Der Original-Topic bleibt erhalten.

Das Enriched Topic enthält sowohl den Namensraum der `sourceId` als auch das vollständige ursprüngliche MQTT-Topic. Beispiel:

```
# Originaltopic ohne führenden Schrägstrich
raw/source/{sourceId}/{originalTopic}

# Originaltopic mit führendem Schrägstrich
raw/source/{sourceId}/{originalTopic}
```

Das Format muss den ursprünglichen Topic verlustfrei bewahren, einschließlich leerer Topic-Ebenen und eines möglichen führenden Schrägstrichs. Der Adapter ermittelt den ursprünglichen Topic über eine deterministische inverse Abbildung.

Der Telemetry Data Adapter registriert sich als Subscriber beim Mosquitto MQTT Broker und bekommt die Telemetriedaten unter dem neuen Topic zugesandt.

Das Package "Identifizier" ist dafür zuständig, Drohnenhersteller, -modell und -version genau zu identifizieren. Die hierfür notwendigen Daten können

- im Topic,
- in der Payload (im MQTT-Paket enthaltene Nachricht)
- oder in beiden verteilt gespeichert sein.

Der Telemetry Data Adapter hält Message Definition -Datensätze mit Topic- und Payload-Regeln zur Identifikation der Telemetriedaten bereit. Bevor ein Drohnentyp erkannt werden kann, müssen das zugehörige Drohnenprofil und seine Message Definition -Datensätze registriert sein. Nicht registrierte Drohnentypen können nicht erkannt werden.

Das Main-Modul initialisiert und koordiniert den Telemetrieadapter.

Der Process-Worker führt den Verarbeitungspfad für die transformierte Telemetrie aus und bleibt während der Laufzeit des Adapters aktiv. Tritt während der Verarbeitung ein Fehler auf, führt der Process-Worker auch die Fehlerbehandlung aus. Hierzu verwendet er die Funktionen des ErrorHandler Package. Mit dessen Funktionen schreibt er einen strukturierten Eintrag in die Datei `error.log` und legt das zugehörige Diagnosepaket im Diagnostic Repository an.

Der BufferManager koordiniert die drei In-Memory-FIFO-Puffer.

Das Validator-Package überprüft Telemetriedaten auf syntaktische und strukturelle Korrektheit. Vor der Identifikation prüft es, ob die Telemetrierohdaten als JSON verarbeitet werden können. Nach erfolgreicher Identifikation validiert es die Telemetrierohdaten gegen das in der ausgewählten Message-Definition referenzierte Rohdatenschema. Nach der Transformation validiert es die transformierten Telemetriedaten gegen das ebenfalls in der Message-Definition referenzierte Ergebnisschema.

Das Transformer-Package wandelt die erfolgreich validierten Telemetrierohdaten anhand der in der ausgewählten Message-Definition referenzierten JSONata-Transformation in das interne Telemetriedatenformat um. Das Transformationsergebnis wird anschließend durch das Validator-Package validiert.

Identifizier-Package, Validator-Package und Transformer-Package besitzen keinen eigenen persistenten Verarbeitungszustand. Der Process-Worker koordiniert diese Packages und bleibt für alle persistenten Zustandsänderungen des Verarbeitungspfads verantwortlich.

Der DeliveryService ist die Komponente, welche abschließend dazu benutzt wird, die Original- und transformierten Telemetriedaten an den Overwatch-Service zu übertragen (at-least-once-Zustellung). Aus Gründen der Einfachheit wird hierzu ein REST-POST-Zugriff genutzt.

Der Zugriff auf den persistenten Speicher erfolgt über die asynchronen Funktionen des Database Package.

Das `Diagnostic Repository` ist ein konfigurierter Pfad auf einem lokalen oder eingebundenen Dateisystem. Für jeden `ErrorCase` legt der `Process-Worker` genau ein ZIP-Archiv an, z. B. unter `/ diagnosticRepositoryPath/errorId.zip`.

3.2.2 Prozesse

Der `Telemetry Data Adapter` unterstützt zwei verschiedene Shutdown-Modi sowie die Datenwiederherstellung beim Startup.

3.2.2.1 Startup / Restart

Beim Startup stellt der Adapter aus dem persistenten Speicher einen konsistenten Laufzeitstand wieder her. Die FIFO-Buffer sind nicht maßgeblich. `inputBuffer` und `outputBuffer` werden aus persistenten Datensätzen rekonstruiert. Der `clipboardBuffer` startet bewusst leer, da der frühere Quellkontext nach einem Neustart nicht als vertrauenswürdig gilt.

Der Adapter darf erst neue MQTT-Messages empfangen, nachdem:

- der persistente Speicher erfolgreich geöffnet wurde,
- unterbrochene Verarbeitungs-, Zustellungs- und Fehlerzustände normalisiert (behoben),
- `inputBuffer` und `outputBuffer` wiederhergestellt wurden,
- nicht abgeschlossene `ErrorCases` ermittelt und fortgeführt wurden,
- der `Process-Worker` bereit ist,
- der `DeliveryService` und seine Timer betriebsbereit sind.

Ablauf des Startup

1. Konfiguration laden,
2. Persistenten Speicher öffnen,
3. Unterbrochene Verarbeitungs-, Zustellungs- und Fehlerzustände normalisieren,
4. `inputBuffer` rekonstruieren,
5. `outputBuffer` rekonstruieren,
6. `ReceiverState` laden bzw. initialisieren,
7. `DeliveryService`-Timer initialisieren
8. `DeliveryService`-Starten
9. `Process-Worker` starten,
10. Nicht abgeschlossene `ErrorCase`-Datensätze aus dem persistenten Speicher laden, gemäß den Wiederherstellungsregeln an den `Process-Worker` übergeben und den `ErrorCase`-retry-Timer aus allen verbleibenden `ErrorCase`-Datensätzen mit `status = retry-wait` initialisieren,

11. MQTT-Verbindung mit den persistenten Sessioninformationen wiederherstellen,
12. Wenn die MQTT-Verbindung wiederhergestellt wurde: Adapterzustand auf **running** setzen.
Andernfalls: Adapterzustand auf **reconnecting** setzen und den konfigurierten MQTT-Reconnect fortsetzen.

Kann der persistente Speicher nicht geöffnet werden, wird der Startup abgebrochen. Der Adapter darf in diesem Fall nicht automatisch einen neuen leeren Speicher anlegen.

Wiederherstellung der Telemetriedatenverarbeitung

Die Wiederherstellung basiert auf `OriginalMessage.processingStatus`.

Persistent er Status	Aktion beim Startup
<code>pending</code>	Der Status bleibt <code>pending</code> . Die Aufnahme der <code>messageId</code> in den <code>inputBuffer</code> erfolgt später zentral bei der Rekonstruktion des <code>inputBuffer</code> .
<code>processi ng</code>	Die vorherige Verarbeitung gilt als unterbrochen. Der Status wird transaktional auf <code>pending</code> zurückgesetzt. Eine separate Einplanung erfolgt nicht. Die Aufnahme der <code>messageId</code> in den <code>inputBuffer</code> erfolgt später zentral bei der Rekonstruktion des <code>inputBuffer</code> .
<code>waitin- g-for- identi- fication</code>	Die frühere Zuordnung zwischen <code>sourceId</code> und <code>deviceId</code> wird nicht automatisch wiederverwendet. Der Telemetry Data Adapter erzeugt in einer Transaktion einen <code>ErrorCase</code> mit <code>ErrorCase.processingStage = identification</code> und <code>ErrorCase.status = pending</code> und setzt <code>OriginalMessage.processingStatus = abandoned-after-restart</code> .
<code>canonica l- created</code>	Die Validierung der transformierten Telemetrie ist abgeschlossen. Die Nachricht wird nicht erneut an den <code>inputBuffer</code> angehängt.
<code>processi ng- failed</code>	Die Verarbeitung der Telemetriedaten wird nicht erneut ausgeführt. Der zugehörige <code>ErrorCase</code> bleibt maßgeblich.
<code>abandone d-at- shutdown</code>	Die Telemetriedaten werden nicht erneut verarbeitet.

Persistent er Status	Aktion beim Startup
abandoned-after-restart	Die Message wird beim Startup nicht erneut für die Verarbeitung der Telemetriedaten eingeplant. Sie bleibt als <code>OriginalMessage</code> persistent gespeichert. Ihr <code>processingStatus</code> bleibt <code>abandoned-after-restart</code> .

Wiederherstellung des Clipboard Buffer

Der `clipboardBuffer` wird standardmäßig nicht aus früheren In-Memory-Zuständen rekonstruiert.

Der Grund ist, dass die Zuordnung von `sourceId` zu `deviceId` nach einem Neustart nicht mehr zuverlässig zum aktuellen physischen Quellkontext gehören muss.

Wiederherstellung des SourceProcessingState

Die vor dem Neustart bestehende Zuordnung zwischen `sourceId` und `deviceId` wird nicht wiederverwendet. Beim Startup werden alle vorhandenen `SourceProcessingState`-Datensätze auf folgenden neutralen Zustand gesetzt:

- `sourceId` bleibt unverändert,
- `identificationStatus` wird auf `unidentified` gesetzt,
- `deviceId` wird auf `null` gesetzt,
- `lastMessageAt` wird auf `null` gesetzt.

Der `SourceProcessingState`-Datensatz kann bestehen bleiben und beim Empfang der nächsten Nachricht derselben `sourceId` erneut verwendet werden. Frühere `deviceId`-Werte dürfen nach dem Neustart nicht zur Identifikation neuer Nachrichten verwendet werden.

Wiederherstellung der Zustellung

Die Wiederherstellung basiert auf dem Zustand der persistenten `DeliveryRecord`-Datensätze.

Persisten ter Status	Aktion beim Startup
pending	Der Status bleibt pending. Die Aufnahme in den <code>outputBuffer</code> erfolgt später zentral bei der Rekonstruktion des <code>outputBuffer</code> .
in-flight	Der vorherige Zustellversuch besitzt ein unbekanntes Ergebnis. Der Status wird auf <code>pending</code> zurückgesetzt. Eine separate Einplanung erfolgt nicht. Der Datensatz wird anschließend bei der Rekonstruktion des <code>outputBuffer</code> aufgenommen.

Persistenter Status	Aktion beim Startup
retry-wait	Ist <code>nextAttemptAt</code> bereits erreicht, wird <code>status = pending</code> und <code>nextAttemptAt = null</code> gesetzt. Andernfalls verbleibt der Datensatz unverändert im Zustand <code>retry-wait</code> . In beiden Fällen wird er anschließend bei der Rekonstruktion des <code>outputBuffers</code> gemäß seiner <code>deliverySequence</code> aufgenommen.
acknowledged	Der Datensatz wird nicht erneut zugestellt. Er wird nicht in den <code>outputBuffer</code> aufgenommen.
blocked	Der Zustand bleibt <code>blocked</code> . Der Datensatz wird in den <code>outputBuffer</code> aufgenommen und bleibt aufgrund seiner <code>deliverySequence</code> der global blockierende <code>DeliveryRecord</code> , bis er administrativ auf <code>pending</code> gesetzt wird.

Der `outputBuffer` enthält alle unbestätigten `DeliveryRecord`-Datensätze in aufsteigender Reihenfolge ihrer `deliverySequence`. Ob der führende `DeliveryRecord` aktuell zugestellt werden darf, ergibt sich aus seinem persistenten status und dem `ReceiverState`. Die Auswahl des nächsten Zustellversuchs und das Head-of-Line-Blocking erfolgen gemäß den im Abschnitt `DeliveryService-Package` definierten Regeln.

Bei der Wiederholung eines zuvor `in-flight` befindlichen Datensatzes wird dieselbe `deliveryId` verwendet. Dadurch kann der Empfänger eine bereits gespeicherte Zustellung idempotent erkennen.

Wiederherstellung des Empfängerstatus

Der Telemetry Data Adapter lädt den persistenten `ReceiverState` beim Startup. Existiert beim erstmaligen Startup noch kein `ReceiverState`, erzeugt der Telemetry Data Adapter einen persistenten Datensatz mit folgenden Werten:

- `receiverId` : Konfigurierte ID des Overwatch-Service
- `availabilityStatus = available`
- `unavailableSince = null`
- `lastHealthCheckAt = null`
- `nextHealthCheckAt = null`
- `lastHealthCheckStatus = null`

Die Bedeutung von `ReceiverState.availabilityStatus` ist:

- `ReceiverState.availabilityStatus = available` : Die reguläre Zustellung kann fortgesetzt werden
- `ReceiverState.availabilityStatus = unavailable` : Es wird keine reguläre Zustellung durchgeführt, stattdessen müssen HealthChecks eingeplant werden (`ReceiverState.nextHealthCheckAt`).

Wiederherstellung der Fehlerbehandlung

Die Wiederherstellung der Fehlerbehandlung basiert auf `ErrorCase.status`. Der Process-Worker liest nicht abgeschlossene ErrorCases aus dem persistenten Speicher.

Wert	Bedeutung
<code>pending</code>	Der Process-Worker lädt den <code>ErrorCase</code> und führt die noch nicht begonnene Fehlerbehandlung aus.
<code>processing</code>	Die vorherige Fehlerbehandlung gilt als unterbrochen. Ein unvollständiges temporäres Diagnosepaket wird entfernt. Der Status wird auf <code>pending</code> zurückgesetzt und der <code>ErrorCase</code> wird erneut verarbeitet.
<code>retry-wait</code>	Der <code>ErrorCase</code> bleibt für einen späteren Wiederholungsversuch vorgemerkt. Sobald der konfigurierte Wiederholungszeitpunkt erreicht ist, nimmt der Process-Worker die Fehlerbehandlung erneut auf.
<code>completed</code>	Logeintrag und Diagnosepaket sind vollständig erzeugt. Der <code>ErrorCase</code> wird nicht erneut verarbeitet.

Bei der Wiederaufnahme verwendet der Process-Worker die gleiche `errorId`. Das Diagnosepaket wird zunächst an einem temporären Speicherort erzeugt und erst nach erfolgreicher Prüfung an seinen endgültigen Speicherort verschoben. Dadurch wird verhindert, dass teilweise geschriebene Pakete als vollständig angesehen werden.

Zeitgesteuerte Wiederherstellung

`DeliveryRecord.nextAttemptAt` und `ErrorCase.nextAttemptAt` sind persistente Zeitpunkte. Die zugehörigen Timer sind nicht persistent und werden beim Startup aus den persistent gespeicherten Zuständen neu initiiert.

Wiederholungsplanung von DeliveryRecords

Der `DeliveryService` verwaltet einen Timer für den kleinsten zukünftigen `nextAttemptAt`-Wert aller `DeliveryRecord`-Datensätze mit `status = retry-wait`.

Beim Ablauf eines Timers führt der `DeliveryService` folgende Schritte aus:

1. alle inzwischen fälligen `DeliveryRecord`-Datensätze mit `status = retry-wait` aus dem persistenten Speicher laden,
2. für jeden fälligen `DeliveryRecord` `status = pending` und `nextAttemptAt = null` setzen und diese Änderung speichern. Der `DeliveryRecord` befindet sich bereits im `outputBuffer` und wird nicht erneut eingeplant.
3. Nachdem alle fälligen `DeliveryRecord`-Datensätze auf `status = pending` gesetzt wurden, ruft der `DeliveryService` einmal `DeliveryService.processNextDelivery` auf. Die Zustellreihenfolge ergibt sich aus dem `outputBuffer`.
4. Den kleinsten zukünftigen `nextAttemptAt`-Wert aller verbleibenden `DeliveryRecord`-Datensätze mit `status = retry-wait` ermitteln.
5. Den Timer auf diesen Zeitpunkt neu setzen. Existiert kein zukünftiger `nextAttemptAt`-Wert, bleibt der Timer deaktiviert.

Der Timer wird außerdem neu berechnet, wenn:

- ein `DeliveryRecord` in den Zustand `retry-wait` wechselt,
- sich `DeliveryRecord.nextAttemptAt` ändert,
- ein Wiederholungsversuch abgeschlossen wird oder
- der Telemetry Data Adapter nach einem Startup die persistenten Zustände wiederhergestellt hat.

Wiederholungsplanung von ErrorCases

Der Process-Worker verwaltet einen Timer für den kleinsten zukünftigen `nextAttemptAt`-Wert aller `ErrorCase`-Datensätze mit `status = retry-wait`.

Beim Ablauf des Timers:

1. lädt der Process-Worker alle inzwischen fälligen `ErrorCase`-Datensätze mit `status = retry-wait` aus dem persistenten Speicher.
2. ruft er für jeden fälligen Datensatz `processErrorCase(errorId)` auf,
3. ermittelt er anschließend den kleinsten zukünftigen `nextAttemptAt`-Wert aller verbleibenden `ErrorCase`-Datensätze mit `status = retry-wait`,
4. setzt er den Timer auf diesen Zeitpunkt neu. Existiert kein zukünftiger `nextAttemptAt`-Wert, bleibt der Timer deaktiviert.

Der Timer wird außerdem neu berechnet, wenn ein `ErrorCase` in den Zustand `retry-wait` wechselt, sich `ErrorCase.nextAttemptAt` ändert, ein Wiederholungsversuch abgeschlossen wird oder der Telemetry Data Adapter beim Startup die persistenten Fehlerzustände wiederhergestellt hat.

Zeitsteuerung des HealthChecks

Der DeliveryService verwaltet außerdem einen Timer für das Attribut

`ReceiverState.nextHealthCheckAt`, wenn `availabilityStatus = unavailable` ist.

Wenn `ReceiverState.nextHealthCheckAt` erreicht ist, führt der DeliveryService folgende Schritte aus:

- Starten eines HealthChecks
- `ReceiverState.lastHealthCheckAt` = aktueller Zeitpunkt
- `ReceiverState.lastHealthCheckStatus` = Ergebnis des letzten Health Checks

War der HealthCheck nicht erfolgreich:

- `ReceiverState.availabilityStatus = unavailable` beibehalten
- `ReceiverState.nextHealthCheckAt` = Zeitpunkt des nächsten Health Checks
- Speichern der Änderungen am `ReceiverState`

War der HealthCheck erfolgreich:

- `ReceiverState.availabilityStatus = available` setzen
- `ReceiverState.unavailableSince = null`
- `ReceiverState.nextHealthCheckAt = null`
- Speichern der Änderungen am `ReceiverState`
- Wiederaufnahme der regulären Zustellung gemäß der im Abschnitt **DeliveryService-Package** definierten Regeln.

Rekonstruktion der FIFO-Buffer

InputBuffer

Nach der Normalisierung unterbrochener Verarbeitungszustände wird der `inputBuffer` aus allen `OriginalMessage`-Datensätzen mit `processingStatus = pending` aufgebaut. Die Sortierung erfolgt nach `ingressSequence` in aufsteigender Reihenfolge. Jede `messageId` wird genau einmal aufgenommen.

OutputBuffer

Nach der Normalisierung unterbrochener Verarbeitungszustände wird der `outputBuffer` aus allen `DeliveryRecord`-Datensätzen mit `status != acknowledged` aufgebaut. Die Sortierung erfolgt nach `deliverySequence` in aufsteigender Reihenfolge. Jeder `DeliveryRecord` wird genau einmal aufgenommen. Auch `DeliveryRecord`-Datensätze mit `status = retry-wait` oder `status = blocked` sind Bestandteil des `outputBuffers`, da sie aufgrund der globalen

Zustellreihenfolge alle späteren `DeliveryRecord`-Datensätze blockieren. Der Kopf des rekonstruierten `outputBuffer` ist damit stets der führende unbestätigte `DeliveryRecord`.

ClipboardBuffer

Der `clipboardBuffer` startet leer. `OriginalMessage`-Datensätze, die vor dem Neustart `processingStatus = waiting-for-identification` besaßen, werden gemäß den Startup-Regeln behandelt und nicht erneut in den `clipboardBuffer` aufgenommen.

Die Fehlerbehandlung verwendet keinen FIFO-Buffer. Nicht abgeschlossene `ErrorCases` werden vom `Process-Worker` direkt aus dem persistenten Speicher geladen.

Wiederherstellung der MQTT-Verbindung

Die MQTT-Verbindung wird erst aufgebaut, nachdem die lokale Datenwiederherstellung abgeschlossen ist.

Der Adapter verwendet:

- dieselbe stabile `clientId`,
- `Clean Start = false`,
- ein von null verschiedenes Session Expiry Interval,
- `Receive Maximum = 1`,
- die vorhandenen Telemetrieabonnements mit QoS 1.

Eine spätere Erhöhung von `Receive Maximum` erfordert eine erneute Bewertung von Teilen der Architektur.



Wiederherstellung nicht bestätigter MQTT-Zustellungen

Hat der Adapter für eine MQTT-Message vor dem Shutdown immediate oder dem Absturz kein MQTT-PUBACK gesendet, kann Mosquitto die Nachricht nach Wiederaufnahme der persistenten Session erneut zustellen. Wurde die MQTT-Ingestion-Transaktion noch nicht committed, wird die erneut zugestellte Message regulär angenommen.

Wurde die MQTT-Ingestion-Transaktion committed, aber das MQTT-PUBACK wurde noch nicht gesendet, kann Mosquitto dieselbe Telemetrie erneut zustellen. Dadurch kann eine weitere `OriginalMessage` entstehen. Dieser Fall entspricht der At-least-once-Semantik und kann zu Duplikaten führen.

Bereits gespeicherte `OriginalMessage`- und `DeliveryRecord`-Datensätze bleiben beim Startup erhalten. Für die Rekonstruktion des Laufzeitzustands werden sie entsprechend ihrer persistenten Zustände gemäß den zuvor beschriebenen Wiederherstellungsregeln berücksichtigt.

Abschluss des Startups

Der Adapter wechselt erst dann in den Zustand **running**, wenn:

- der persistente Nachrichtenspeicher schreibbereit ist,
- unterbrochene Statuswerte normalisiert wurden,
- `inputBuffer` und `outputBuffer` rekonstruiert und der `clipboardBuffer` leer initialisiert wurden,
- nicht abgeschlossene ErrorCases ermittelt wurden,
- der Process-Worker betriebsbereit ist,
- der DeliveryService und seine Timer betriebsbereit sind,
- die MQTT-Verbindung hergestellt wurde.

Sind alle lokalen Startup-Schritte abgeschlossen, kann die MQTT-Verbindung jedoch nicht hergestellt werden, so wechselt der Adapter in den Zustand **reconnecting**. Im Zustand **reconnecting**:

- werden keine neuen MQTT-Messages empfangen oder bestätigt,
- können bereits persistierte Messages weiterverarbeitet werden,
- können ausstehende HTTP-Zustellungen fortgesetzt werden,
- versucht der Telemetry Data Adapter weiterhin, die persistente Session wiederherzustellen.

Nach erfolgreicher MQTT-Verbindung wechselt der Adapter von **reconnecting** zu **running**.

3.2.2.2 Shutdown normal (geordnetes Herunterfahren)

Ein geordnetes Herunterfahren stoppt die Datenannahme und hinterlässt alle akzeptierten Daten in einem konsistenten persistenten Zustand.

Zeitbegrenzung

Für den Shutdown normal gilt ab dem Wechsel in den Zustand **normal-shutdown** eine maximale Gesamtdauer von 30 Sekunden. Innerhalb dieser Zeit versucht der Adapter, die beschriebenen Shutdown-Schritte geordnet abzuschließen. Sind nach 30 Sekunden noch Schritte offen, wird der Shutdown normal abgebrochen und unmittelbar der Shutdown immediate ausgeführt.

Ablauf Shutdown normal

1. Adapterzustand auf **normal-shutdown** setzen,
2. Shutdown-Timer mit 30 Sekunden starten,
3. Annahme von MQTT-Messages unterbinden,
4. Bereits laufende Transaktionen der MQTT-Ingestion, des Process-Workers und des DeliveryService abschließen oder zurückrollen,
5. Verbindung zu Mosquitto trennen, ohne das Abonnement oder die persistente Session zu löschen,
6. Aktuelle Telemetriedatenverarbeitung des Process-Workers abschließen lassen,

7. InputBuffer vollständig abarbeiten,
8. Jede noch auflösbare Clipboard-Partition verarbeiten,
9. Nicht auflösbare Identifizierungsversuche abbrechen und protokollieren,
10. Einen Error-Handling-Vorgang, der nach dem Beginn des Shutdowns gestartet wurde kontrolliert abschließen,
11. Process-Worker und DeliveryService stoppen,
12. Datenbank ordnungsgemäß schließen,
13. Adapterprozess beenden.

Für jeden nicht erfolgreichen Identifizierungsversuch einer Message werden in einer Transaktion:

- ein `ErrorCase` mit `ErrorCase.processingStage = identification` und `ErrorCase.status = pending` erstellen,
- `OriginalMessage.processingStatus` auf `abandoned-at-shutdown` setzen.

Beim Herunterfahren wird nicht darauf gewartet, dass der Empfänger alle Zustellungen bestätigt. Unbestätigte ursprüngliche und kanonische `DeliveryRecord`-Einträge bleiben persistent und werden nach dem Neustart fortgesetzt.

3.2.2.3 Shutdown immediate (Emergency Shutdown)

Das sofortige Herunterfahren beendet den Adapter möglichst schnell, ohne die In-Memory-Buffer abzuarbeiten und ohne auf ausstehende Zustellungen oder Diagnosevorgänge zu warten.

Es wird beispielsweise verwendet, wenn:

- ein Administrator einen unmittelbaren Shutdown anfordert,
- ein schwerwiegender interner Fehler erkannt wurde,
- der persistente Speicher nicht mehr zuverlässig verwendet werden kann,
- die für einen Shutdown normal notwendige Zeit nicht mehr ausreicht.

Der Shutdown immediate darf keine bereits persistent akzeptierten Telemetrierohdaten löschen. Nicht persistierte Arbeit darf hingegen verworfen werden und wird gegebenenfalls durch MQTT oder die Startwiederherstellung erneut bereitgestellt.

Zeitbegrenzung

Für den Shutdown immediate gilt ab dem Wechsel in den Zustand **immediate-shutdown** eine maximale Gesamtdauer von 10 Sekunden. Nach Ablauf dieser Frist wird der Telemetry Data Adapter beendet, auch wenn einzelne Abschlussarbeiten nicht vollständig beendet wurden. Nicht abgeschlossene Arbeit wird beim nächsten Startup ausschließlich anhand des persistenten Zustands behandelt.

Ablauf Shutdown immediate

1. Adapterzustand auf **immediate-shutdown** setzen,

2. Annahme von MQTT-Messages unterbinden,
3. Keine neuen Verarbeitungs-, Zustell- oder Diagnoseaufträge beginnen,
4. Keine neuen Datenbanktransaktionen beginnen und alle noch nicht bestätigten Transaktionen gemäß den folgenden Regeln behandeln,
5. Verbindung zu Mosquitto trennen, ohne das Abonnement oder die persistente Session zu löschen,
6. Einem bereits laufenden Error-Handling-Vorgang eine maximale Frist von 5 Sekunden einräumen, um zu einem kontrollierten Abschluss zu kommen. Es wird keine neue Fehlerbehandlung begonnen.
7. Process-Worker und DeliveryService stoppen,
8. Datenbank ordnungsgemäß schließen,
9. Adapterprozess beenden.

Gemäß Schritt 4 werden nach Beginn des Shutdown keine neuen Datenbanktransaktionen für MQTT-Ingestion, reguläre Telemetrieverarbeitung und alle anderen Prozesse mehr gestartet. Ein bereits laufender ErrorHandling-Vorgang darf innerhalb der in Schritt 6 definierten Grace Period die für seinen kontrollierten Abschluss erforderlichen Aktualisierungen durchführen.

Behandlung aktiver Datenbanktransaktionen

MQTT-Ingestion

- Eine noch nicht bestätigte Ingestion-Transaktion wird zurückgerollt.
- Für die betreffende MQTT-Message wird kein PUBACK gesendet.

Verarbeitung der Telemetrierohdaten

- Eine noch nicht bestätigte Transaktion des Process-Workers wird zurückgerollt.
- Eine unvollständige CanonicalMessage oder ein unvollständiger DeliveryRecord für transformierte Daten darf nicht persistent bestätigt werden.

HTTP-Zustellung

- Eine noch nicht bestätigte Änderung des `DeliveryRecord` wird zurückgerollt.
- Bei unbekanntem Ergebnis des HTTP-Requests darf der `DeliveryRecord` nicht auf `acknowledged` gesetzt werden.

Error handling

- Eine laufende Fehlerbehandlung darf innerhalb der festgelegten Grace Period von 5 Sekunden ihre aktuelle Operation abschließen.
- Wird sie nicht rechtzeitig abgeschlossen, werden noch nicht bestätigte Änderungen des `ErrorCase` zurückgerollt.

FIFO-Buffer

Die FIFO-Buffer werden beim Shutdown immediate ausdrücklich nicht fertig abgearbeitet. Für eine aktuell laufende MQTT-Verarbeitung gilt während des Shutdowns:

- **Die Transaktion wurde noch nicht committed:** Die Transaktion wird zurückgerollt und der Adapter sendet kein MQTT-PUBACK.
- **Die Transaktion wurde committed, aber das MQTT-PUBACK wurde noch nicht gesendet:** Die Message ist bereits akzeptiert und persistent gespeichert. Sie wird nicht weiter verändert.
- **Die Transaktion wurde committed und das PUBACK wurde gesendet:** Für diese Message ist während des Shutdowns keine weitere Aktion notwendig.

Verarbeitung der Telemetrierohdaten

Eine laufende Verarbeitung wird abgebrochen.

- Noch nicht bestätigte Datenbankänderungen werden zurückgerollt,
- Es wird keine neue CanonicalMessage erzeugt und gespeichert,
- Es wird kein neuer DeliveryRecord erzeugt und gespeichert,
- Eine laufende Bearbeitung eines ErrorCase darf ausschließlich gemäß der im Abschnitt Error Handling definierten Grace Period fortgesetzt werden und wird danach gegebenenfalls abgebrochen,
- Es werden keine neuen Einträge in die FIFO-Buffer geschrieben.

Bereits committete Datensätze bleiben unverändert im persistenten Speicher erhalten.

Aktive Zustellungen

Eine laufende HTTP-Anfrage wird nach Möglichkeit abgebrochen. Der Adapter wartet nicht auf die Antwort des Empfängers. Der zugehörige `DeliveryRecord` wird während des Shutdown immediate nicht weiter verändert, wenn nicht eindeutig festgestellt werden kann, ob der Empfänger die Nachricht gespeichert hat.

Error Handling (Process-Worker)

Nach Beginn des Shutdown immediate beginnt der Process-Worker keine neue Fehlerbehandlung. Eine bereits laufende Fehlerbehandlung darf innerhalb einer festen **Grace Period** von höchstens 5 Sekunden abgeschlossen werden.

Ein gerade erzeugtes Diagnosepaket darf nur dann als vollständig markiert werden, wenn:

- alle Dateien geschrieben wurden,
- die Vollständigkeitsprüfung erfolgreich war,
- das Paket an seinen endgültigen Speicherort verschoben wurde und
- die abschließende Änderung von `ErrorCase.status` persistent bestätigt wurde.

Ist das Error-Handling nach 5 Sekunden nicht vollständig abgeschlossen, wird es abgebrochen. Der `ErrorCase` darf in diesem Fall nicht auf `completed` gesetzt werden. Er verbleibt in einem Zustand, aus dem das Error handling beim nächsten Startup wiederaufgenommen werden kann.

Die 5-sekündige Grace Period ist Bestandteil der 10-Sekundendauer des Shutdown immediate.

Ist die frühere MQTT-Session nicht mehr vorhanden, stellt der Adapter die benötigten Abonnements neu her. Bereits im lokalen Nachrichtenspeicher vorhandene Daten werden dadurch nicht verändert.

3.2.3 Konzepte

3.2.3.1 No-Data-Loss Guarantee

Die No-Data-Loss Guarantee besagt, dass jede akzeptierte ursprüngliche MQTT-Telemetrienachricht persistent gespeichert bleibt und zur Zustellung vorgemerkt wird, bis der Empfänger den persistenten Empfang bestätigt. Die Garantie beginnt, nachdem der Telemetry Data Adapter eine MQTT-Nachricht akzeptiert hat.

Eine Nachricht gilt erst dann als akzeptiert, wenn:

1. Die Originalnachricht (Telemetrirohdaten) in den persistenten Speicher geschrieben wurde.
2. Ein `DeliveryRecord` für die Telemetrirohdaten erzeugt wurde.
3. Beide Datensätze gemeinsam erfolgreich in einer Transaktion bestätigt wurden.

Erst nach einem erfolgreichen Commit darf der Telemetry Data Adapter die MQTT-Zustellung mit PUBACK bestätigen.

Die Garantie umfasst:

- Abstürze des Adapterprozesses,
- Neustarts des Adapters und des Betriebssystems,
- vorübergehenden Stromausfall bei korrekter Speicherkonfiguration,
- vorübergehende MQTT-Verbindungsabbrüche,
- vorübergehende Netzwerkfehler,
- vorübergehende Ausfälle des Empfängers,
- verlorene HTTP-Antworten,
- wiederholte Zustellversuche.

Die Garantie umfasst nicht die dauerhafte Zerstörung der einzigen persistenten Speicherkopie. Der Schutz vor dem Verlust eines Datenträgers, Hosts oder Standorts erfordert replizierten Speicher, einen hochverfügbaren externen Datenspeicher oder Sicherungen mit einem ausdrücklich akzeptierten Recovery Point Objective.

Die Zustellung verwendet **At-least-once-Semantik**: Keine akzeptierte Originalnachricht wird absichtlich verworfen. Dieselbe Nachricht kann mehrfach zugestellt werden.

Der Empfänger muss Zustellungen deshalb idempotent verarbeiten. Die Idempotenz über `deliveryId` verhindert ausschließlich die mehrfache Verarbeitung wiederholter HTTP-

Zustellversuche desselben `DeliveryRecord`. Sie verhindert keine logischen Duplikate, die durch eine erneute MQTT-Zustellung nach bereits erfolgreichem Commit der MQTT-Ingestion-Transaktion, aber vor erfolgreicher Übermittlung des MQTT-PUBACK entstehen. In diesem Fall kann eine neue `OriginalMessage` mit einem neuen `DeliveryRecord` und einer neuen `deliveryId` entstehen. Solche Duplikate sind Bestandteil der beschriebenen At-least-once-Semantik.

Voraussetzungen

Alle folgenden Voraussetzungen sind für die angegebene Garantie erforderlich.

QoS des Publishers

Publisher müssen MQTT QoS 1 oder QoS 2 verwenden. QoS 0 kann keine Datenverlustfreiheit garantieren. Der Adapter abonniert mit QoS 1. Ein mit QoS 2 veröffentlichtes Ereignis kann deshalb mit QoS 1 an den Adapter zugestellt werden. Dadurch entsteht mit PUBACK eine eindeutige persistente Verantwortungsgränze.

Persistenter Mosquitto-Speicher

Mosquitto muss Folgendes persistieren:

- Sessionzustand,
- gepufferte QoS-1- und QoS-2-Nachrichten,
- den in-flight-Protokollzustand bereits übertragener, aber noch nicht bestätigter QoS-1- und QoS-2-PUBLISH-Messages der persistenten Session.

Der Brokerspeicher muss funktionsfähig und ausreichend dimensioniert sein.

Persistente Adapter-Session

Der Adapter muss:

- eine stabile `MQTT-clientId`,
- MQTT 5 mit `Clean Start = false`,
- ein von null verschiedenes Session Expiry Interval, das den maximal unterstützten Ausfall abdeckt

verwenden.

Nachrichtenablauf

Das MQTT Message Expiry Interval muss fehlen oder länger als jeder unterstützte Adapterausfall sein. Eine beim Broker abgelaufene Nachricht darf verworfen werden und liegt deshalb außerhalb der Garantie.

Persistente MQTT-Bestätigungsgrenze

PUBACK darf erst freigegeben werden, nachdem die `OriginalMessage` und ihr `DeliveryRecord` erfolgreich bestätigt wurden.

Transaktionaler persistenter Speicher

Der persistente Nachrichtenspeicher muss atomare und persistente Transaktionen bereitstellen. SQLite und das zugrunde liegende Dateisystem müssen so konfiguriert sein, dass die erforderlichen Synchronisierungsoperationen ausgeführt werden.

Ausreichender Speicher und Backpressure

Der Adapter muss den persistenten Speicher überwachen. Kann eine Nachricht nicht persistiert werden, muss der Adapter MQTT-Bestätigungen zurückhalten oder die Verbindung trennen. Er darf Nachrichten nicht ausschließlich in flüchtigem Speicher akzeptieren.

Die erforderliche Kapazität basiert mindestens auf:

$$\text{maximale persistente Datenrate [Byte/s]} \times \text{maximal unterstützte Ausfalldauer [s]} + \text{betriebliche Sicherheitsreserve}$$

Bei der Ermittlung der persistenten Datenrate sind insbesondere `OriginalMessage`, `CanonicalMessage`, `DeliveryRecord`, `ErrorCase` sowie der Speicherbedarf der Diagnosepakete zu berücksichtigen.

Persistente Bestätigung durch den Empfänger

Der Empfänger muss die Zustellung persistent und idempotent bestätigen. Der vollständige Empfänger-Contract ist im Abschnitt "Bestätigung / Acknowledgement" beschrieben.

Aufbewahrung unbestätigter Zustellungen

Eine ausstehende Originalzustellung darf nicht wegen ausgeschöpfter Wiederholungen, Nichtverfügbarkeit des Empfängers, Herunterfahren oder fehlgeschlagener Verarbeitung von Telemetrierohdaten verworfen werden.

Eine manuelle administrative Löschung hebt die Garantie für die gelöschte Nachricht auf.

Persistente Identifikatoren und Sequenzen

`messageId`, `deliveryId`, `ingressSequence` und `deliverySequence` müssen kollisionsfrei sein und über Neustarts hinweg gültig bleiben. Die Korrektheit darf nicht davon abhängen, dass Zeitstempel eindeutig sind.

- `ingressSequence` wird global monoton bei der persistenten Annahme einer `OriginalMessage` vergeben

- `deliverySequence` wird global monoton bei der persistenten Erzeugung eines `DeliveryRecord` vergeben. Die Vergabe erfolgt innerhalb derselben Transaktion, in der der `DeliveryRecord` persistiert wird.

Die Erzeugung von `DeliveryRecord`-Datensätzen muss so serialisiert werden, dass ein später erfolgreich committeter `DeliveryRecord` keine kleinere `deliverySequence` als ein zuvor erfolgreich committeter `DeliveryRecord` besitzen kann. Dadurch können neu erzeugte `DeliveryRecord`-Datensätze während des normalen Betriebs am Ende des `outputBuffer` angefügt werden, ohne dessen FIFO-Reihenfolge zu verletzen.

Die persistente Erzeugung eines `DeliveryRecord` und die anschließende Aufnahme seiner `deliveryId` in den `outputBuffer` werden über einen gemeinsamen serialisierten Scheduling-Pfad koordiniert. Eine `deliveryId` mit einer größeren `deliverySequence` darf nicht vor einer bereits committeten `deliveryId` mit kleinerer `deliverySequence` in den `outputBuffer` aufgenommen werden. Dadurch entspricht die Reihenfolge der Enqueue-Operationen jederzeit der Reihenfolge der `deliverySequence`.

Wiederherstellungsverhalten

Der Start muss abgebrochen werden, wenn die vorhandene Datenbank nicht geöffnet werden kann. Nach Verlust des Zugriffs auf den maßgeblichen Speicher darf der Adapter nicht stillschweigend einen neuen leeren Speicher erzeugen.

Betriebsüberwachung

Folgendes muss überwacht werden:

- Persistierungsfehler von Mosquitto,
- Datenbankfehler und Integritätsprüfungen,
- Speicherkapazität,
- Wachstum ausstehender Zustellungen,
- Dauer des Empfängerausfalls,
- wiederholte HTTP-Fehler,
- Wiederherstellung der MQTT-Session,
- Nachrichten, deren Ablaufzeitpunkt näher rückt.

3.3 White-Box-Sicht

A. Overwatch - Telemetry Data Adapter - White-Box View

Main Module

Das Hauptmodul initialisiert und koordiniert den Telemetry Data Adapter. Seine Aufgaben im Detail sind:

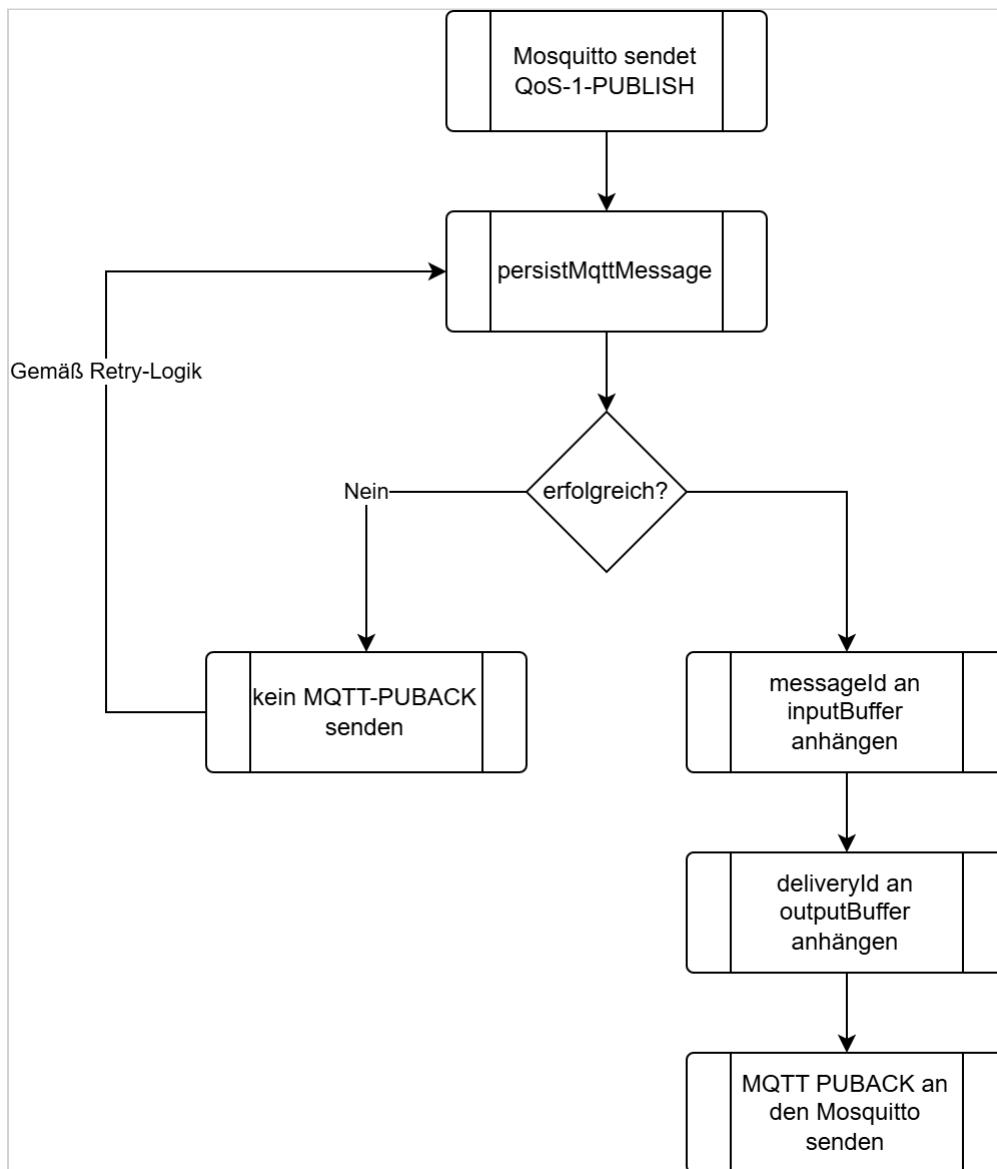
- stellt die MQTT-5-Verbindung zu Mosquitto her und verwendet eine persistente MQTT-Session,
- abonniert die konfigurierten Telemetrie-Topics mit QoS 1, um MQTT-Nutzlasten zu empfangen,
- startet und überwacht den `Process-Worker` ,
- startet und überwacht den `DeliveryService` ,
- koordiniert das geordnete Herunterfahren,
- koordiniert die Wiederherstellung beim Startup

MQTT-Ingestion

Für den ersten Persistierungsversuch und alle Wiederholungsversuche wird dieselbe logische Operation `persistMqttMessage` verwendet. `persistMqttMessage` führt genau eine Datenbanktransaktion aus. Innerhalb dieser Transaktion:

- wird die `OriginalMessage` mit `processingStatus = pending` und der nächsten `ingressSequence` erzeugt.
- wird der zugehörige `DeliveryRecord` für die Originaltelemetrie mit `status = pending` und der nächsten globalen `deliverySequence` erzeugt.

Die Operation ist nur erfolgreich, wenn beide Datensätze gemeinsam committed wurden. Bei einem Fehler wird die gesamte Transaktion zurückgerollt. Nach einem erfolgreichen Commit werden `messageId` und `deliveryId` gemäß den Regeln des Buffer Managers in `inputBuffer` beziehungsweise `outputBuffer` aufgenommen. Erst danach darf MQTT-PUBACK gesendet werden.



Backpressure bei Persistierungsfehlern

Kann die MQTT-Message nicht persistent gespeichert werden, sendet der Telemetry Data Adapter kein MQTT-PUBACK. Da `Receive Maximum = 1` verwendet wird, darf Mosquitto währenddessen keine weitere noch nicht bestätigte QoS-1- oder QoS-2-Message an den Telemetry Data Adapter senden. Das Zurückhalten von PUBACK zusammen mit `Receive Maximum = 1` bildet die Backpressure-Strategie.

Eskalation bei anhaltenden Persistierungsfehlern

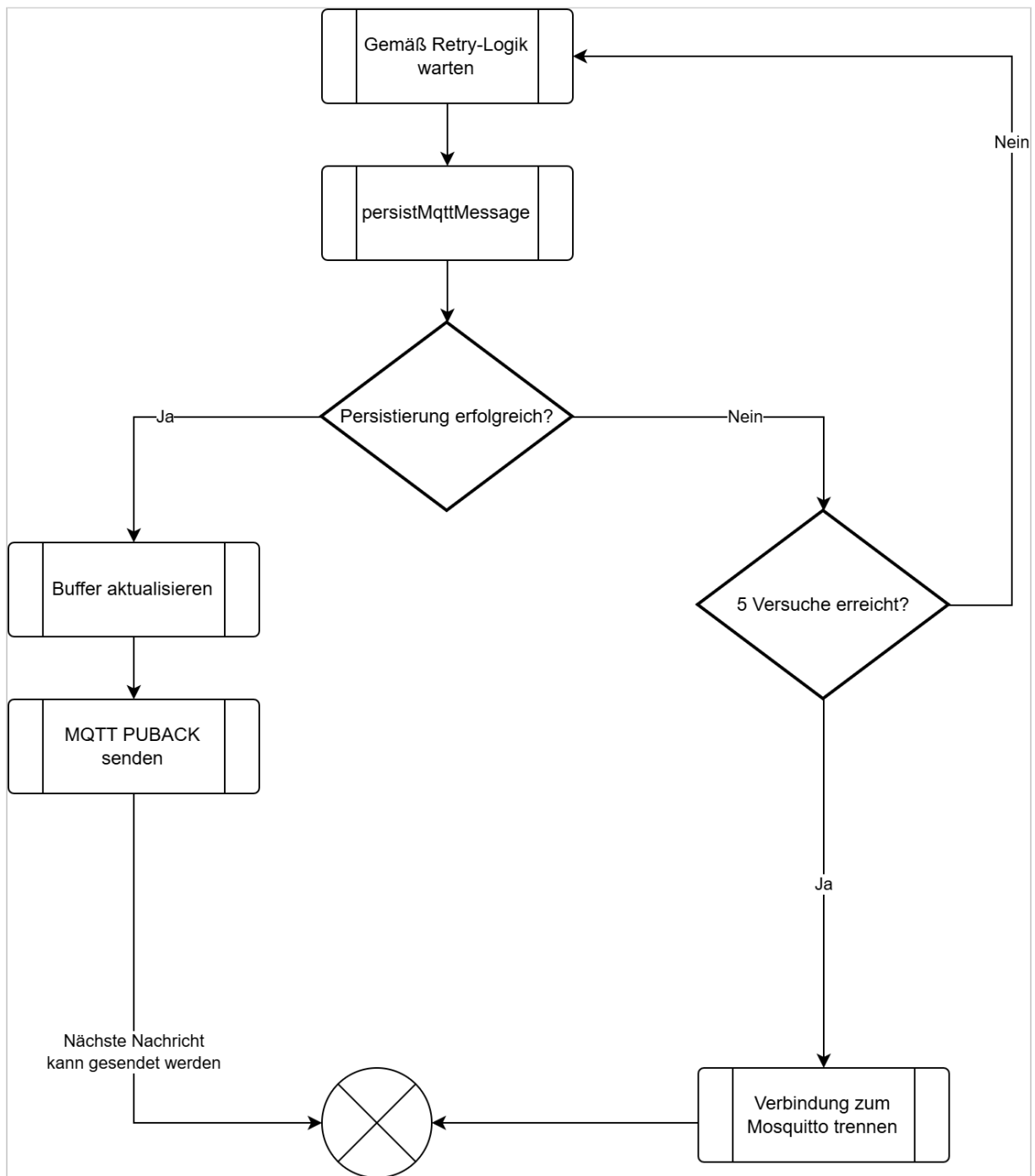
Kann die Message trotz der konfigurierten Anzahl von max. 5 Persistierungsversuchen nicht gespeichert werden, trennt der Adapter die MQTT-Verbindung, ohne die persistente Session oder das

Abonnement zu löschen. Die Trennung der Verbindung ist keine Backpressure-Maßnahme, sondern die Eskalation eines anhaltenden Problems.

Der erste Persistierungsversuch wird unmittelbar ausgeführt. Schlägt er fehl, führt der Adapter höchstens 4 weitere Versuche aus:

- Wiederherstellungsversuch 1 nach 100 Millisekunden,
- Wiederherstellungsversuch 2 nach 500 Millisekunden,
- Wiederherstellungsversuch 3 nach 1.000 Millisekunden,
- Wiederherstellungsversuch 4 nach 2.000 Millisekunden

Damit werden höchstens 5 Versuche ausgeführt.



Wiederherstellung nach Persistierungsfehlern

Nachdem das Problem administrativ behoben wurde, kann sich der Telemetry Data Adapter mit derselben `clientId`, `Clean Start = false` und dem bestehenden `Session Expiry Interval` erneut mit Mosquitto verbinden.

Buffer Manager

Der Buffer Manager koordiniert drei In-Memory-FIFO-Buffer. Dies sind:

- `inputBuffer` : globale FIFO-Warteschlange für noch zu verarbeitende `OriginalMessage`-Datensätze
- `clipboardBuffer` : nach `sourceId` partitionierte FIFO-Warteschlange für `OriginalMessage`-Datensätze, die auf eine eindeutige Identifikation ihrer Quelle warten
- `outputBuffer` : globale FIFO-Warteschlange aller noch nicht bestätigten `DeliveryRecord`-Datensätze

Die Buffer enthalten ausschließlich Identifier. Der persistente Nachrichtenspeicher bleibt die maßgebliche Zustandsquelle des Adapters.

Für jeden Buffer gelten außerhalb eines gerade ausgeführten Zustandsübergangs folgende Invarianten:

- Eine `messageId` befindet sich genau dann im `inputBuffer`, wenn die zugehörige `OriginalMessage.processingStatus = pending` besitzt und aktuell nicht vom Process-Worker verarbeitet wird.
- Eine `messageId` befindet sich genau dann in einer Partition des `clipboardBuffer`, wenn die zugehörige `OriginalMessage.processingStatus = waiting-for-identification` besitzt.
- Eine `deliveryId` befindet sich genau dann im `outputBuffer`, wenn der zugehörige `DeliveryRecord.status != acknowledged` ist.

Die Aufgaben des Buffer Manager umfassen:

- stellt FIFO-Operationen zum Einfügen und Lesen des Kopfes (peek) und Entfernen bereitzustellen,
- signalisiert Konsumenten, wenn Arbeit verfügbar ist,
- partitioniert den Clipboard-Buffer nach `sourceId`,
- verhindert, dass derselbe persistente Datensatz mehrfach eingeplant wird,
- baut `inputBuffer` und `outputBuffer` nach dem Start aus persistenten Datensätzen wieder auf und initialisiert den `clipboardBuffer` leer,
- verhindert widersprüchliche Buffer-Changes während Übergängen beim Herunterfahren.

Buffer	Partitionierung	Identifizier	Bedeutung
inputBuffer		message Id	Enthält alle zur Verarbeitung anstehenden OriginalMessage-Datensätze mit <code>processingStatus = pending</code> . Die Reihenfolge richtet sich nach <code>OriginalMessage.ingressSequence</code> aufsteigend. Der Process-Worker verarbeitet immer den Datensatz am Kopf des Buffers.
clipboardBuffer	sourceId	message Id	Enthält OriginalMessage-Datensätze mit <code>processingStatus = waiting-for-identification</code> . Innerhalb jeder <code>sourceId</code> -Partition erfolgt die Reihenfolge nach <code>ingressSequence</code> aufsteigend. Der Buffer ist kein persistenter Zustand und wird beim Startup nicht rekonstruiert.
outputBuffer		deliveryId	Enthält jeden <code>DeliveryRecord</code> mit <code>status != acknowledged</code> genau einmal. Die Reihenfolge richtet sich global nach <code>DeliveryRecord.deliverySequence</code> aufsteigend. Der Datensatz am Kopf des Buffers ist damit immer der führende unbestätigte <code>DeliveryRecord</code> . Ein Eintrag verbleibt im <code>outputBuffer</code> , solange sein Status <code>pending</code> , <code>in-flight</code> , <code>retry-wait</code> oder <code>blocked</code> ist. Er wird erst entfernt, nachdem der zugehörige <code>DeliveryRecord</code> persistent den Status <code>acknowledged</code> erhalten hat.

Ist für eine `sourceId` keine vertrauenswürdige `deviceId` bekannt (`SourceProcessingState.identificationStatus = unidentified`), versucht der Process-Worker zunächst, die aktuell verarbeitete Nachricht zur Identifikation der Drohne zu verwenden. Kann die Drohne anhand dieser Nachricht nicht eindeutig identifiziert werden, setzt der Process-Worker `OriginalMessage.processingStatus = waiting-for-identification` und fügt die `messageId` am Ende der zugehörigen `sourceId`-Partition des `clipboardBuffers` ein.

Kann eine spätere Nachricht derselben `sourceId` die Drohne eindeutig identifizieren, aktualisiert der Process-Worker zunächst den `SourceProcessingState`. Anschließend verarbeitet er die bereits wartenden Nachrichten dieser `clipboardBuffer`-Partition in Reihenfolge ihrer `ingressSequence`. Erst nachdem die Partition vollständig abgearbeitet wurde, setzt er die Verarbeitung der Nachricht fort, durch die die Identifikation möglich wurde.

Bevor der Process-Worker eine `OriginalMessage` aus einer `clipboardBuffer`-Partition weiterverarbeitet, setzt er `OriginalMessage.processingStatus` von `waiting-for-`

identification auf processing. Nach erfolgreicher Persistierung entfernt er die messageId aus der betreffenden clipboardBuffer-Partition. Anschließend wird für diese Nachricht unter Verwendung des nun bekannten Drohnenprofils gemäß den Regeln des Identifier-Packages die passende Message Definition durch bestimmt. Danach durchläuft die Nachricht die Rohdatenvalidierung, Transformation und Validierung der transformierten Daten. Nach Abschluss der Verarbeitung setzt der Process-Worker den entsprechenden terminalen processingStatus und fährt mit dem nächsten Eintrag der Partition fort.

Beispiel:

```
Nachricht 0 → Drohne kann nicht identifiziert werden
Nachricht 1 → Drohne kann nicht identifiziert werden
Nachricht 2 → Drohne kann nicht identifiziert werden
Nachricht 3 → identifiziert die Drohne
```

Nachdem Nachricht 3 die Drohne identifiziert hat, aktualisiert der Process-Worker den SourceProcessingState. Anschließend verarbeitet er die Nachrichten 0, 1 und 2 aus der betreffenden clipboardBuffer-Partition in aufsteigender Reihenfolge ihrer ingressSequence. Erst danach setzt er die Verarbeitung von Nachricht 3 fort. Die Nachrichten 0, 1 und 2 werden dabei nicht erneut in den inputBuffer eingestellt.

Die DeliveryRecords der Telemetrierohdaten sind davon unabhängig und können vom Empfänger bereits bestätigt worden sein.

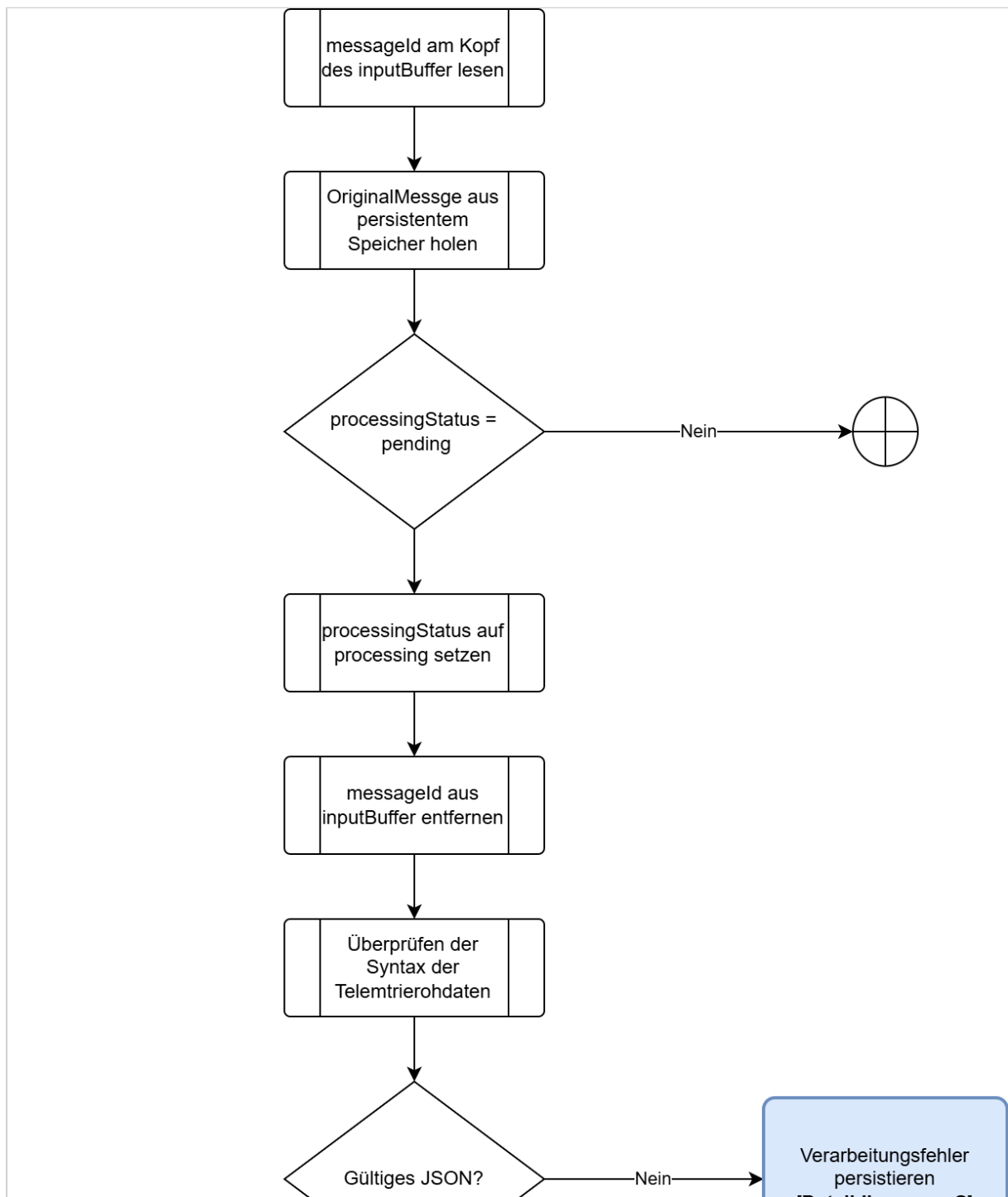
Process-Worker

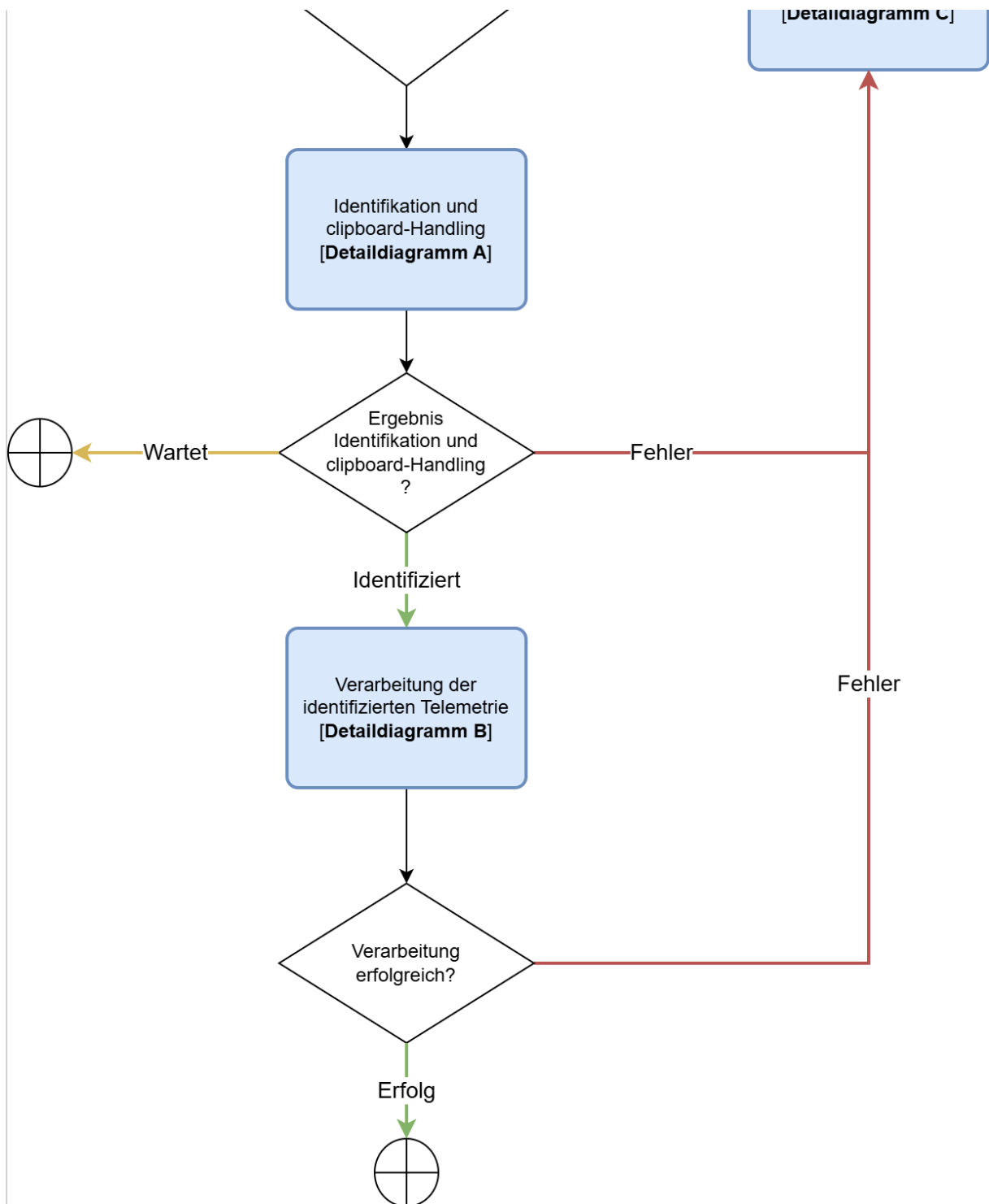
Der Process-Worker führt den Verarbeitungspfad für transformierte Telemetrie aus und bleibt während der Laufzeit des Adapters aktiv. Seine Tätigkeiten umfassen die Identifikation des Senders (Drohne), die Validierung der Telemetrierohdaten, die Transformation der Rohdaten in das interne Datenformat und die Validierung der transformierten Telemetrie. Hierfür greift er auf die Packages Identifier, Validator und Transformer zu.

Der Prozess führt außerdem die Fehlerbehandlung für fehlgeschlagene Verarbeitungsvorgänge durch. Bevor der Process-Worker eine Nachricht an das Identifier-Package übergibt, lässt er die OriginalMessage.originalPayload durch das Validator-Package als JSON einlesen. Kann die Payload nicht als gültiges JSON verarbeitet werden, führt der Process-Worker den gemeinsamen Fehlerpfad mit processingStage = raw-data-validation aus. Da zu diesem Zeitpunkt noch keine Message Definition eindeutig ausgewählt wurde, wird kein ErrorCaseProcessingContext erzeugt. Der [ErrorCase \(see page 59\)](#), OriginalMessage.processingStatus = processing-failed und die zugehörigen Fehlerdaten werden in einer Transaktion persistent gespeichert. Nach erfolgreichem Commit ruft der Process-Worker processErrorCase(errorId) auf. Die Nachricht wird nicht in den clipboardBuffer aufgenommen.

Bei erfolgreicher Identifikation erhält der Process-Worker neben der `deviceId` auch die für die aktuelle Nachricht ausgewählte `Message Definition`. Deren Referenzen `rawSchema`, `transformationId` und `normalizedSchema` bestimmen die anschließend zu verwendenden Validierungs- und Transformationsartefakte.

Arbeitsablauf



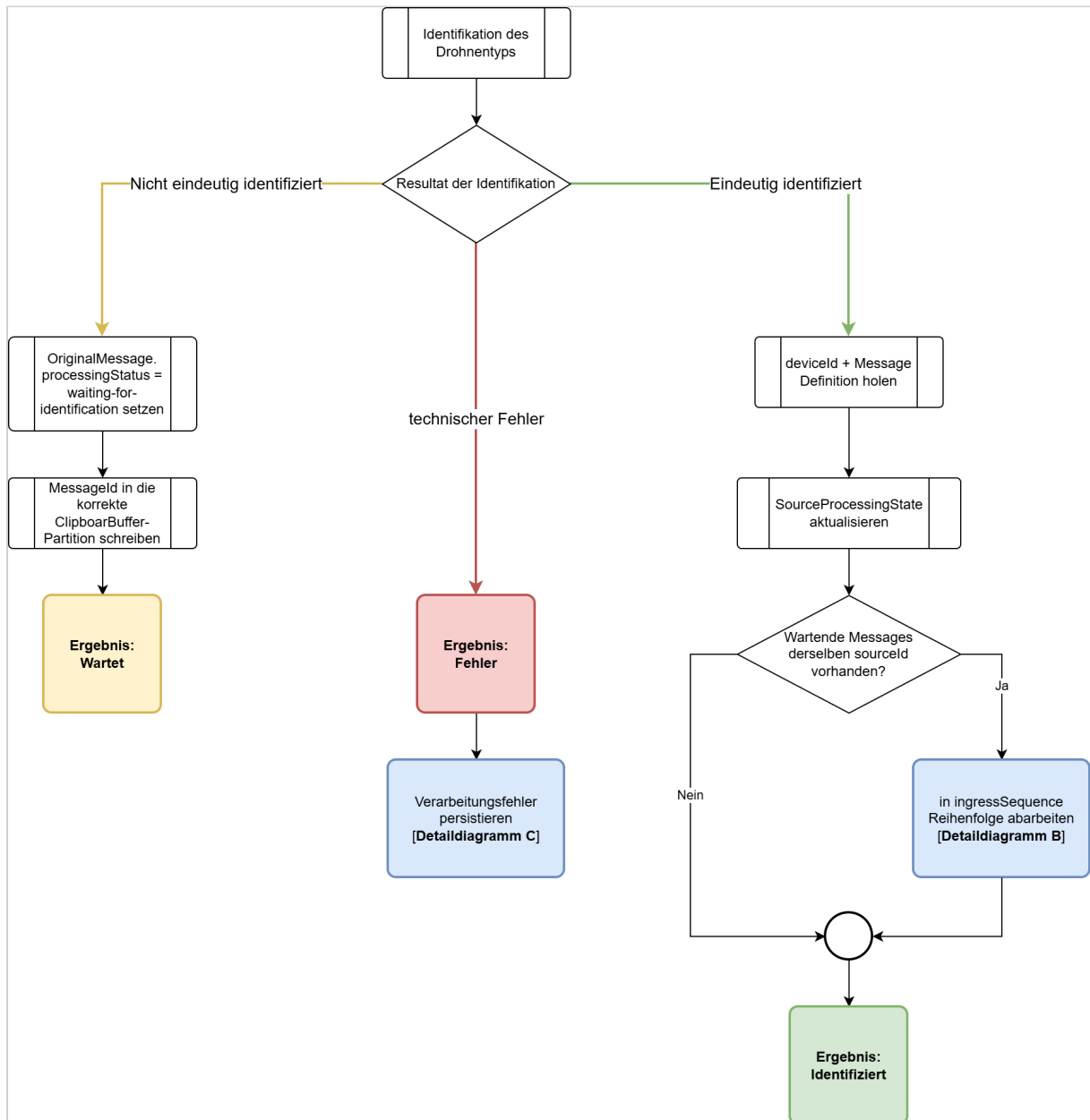


Die persistente Erzeugung des `DeliveryRecord` für die Telemetrierohdaten ist von der Transformation unabhängig. Deshalb werden die Rohdaten schon vor dem Verarbeitungsprozess dauerhaft zur Zustellung vorgemerkt. Die Originalnachricht wird deshalb auch dann zugestellt, wenn:

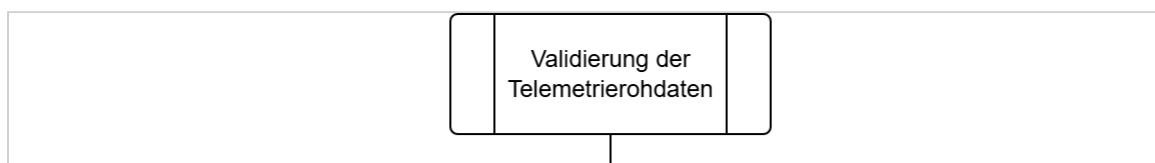
- die Drohne nicht identifiziert werden kann,
- die Rohdatenvalidierung fehlschlägt,

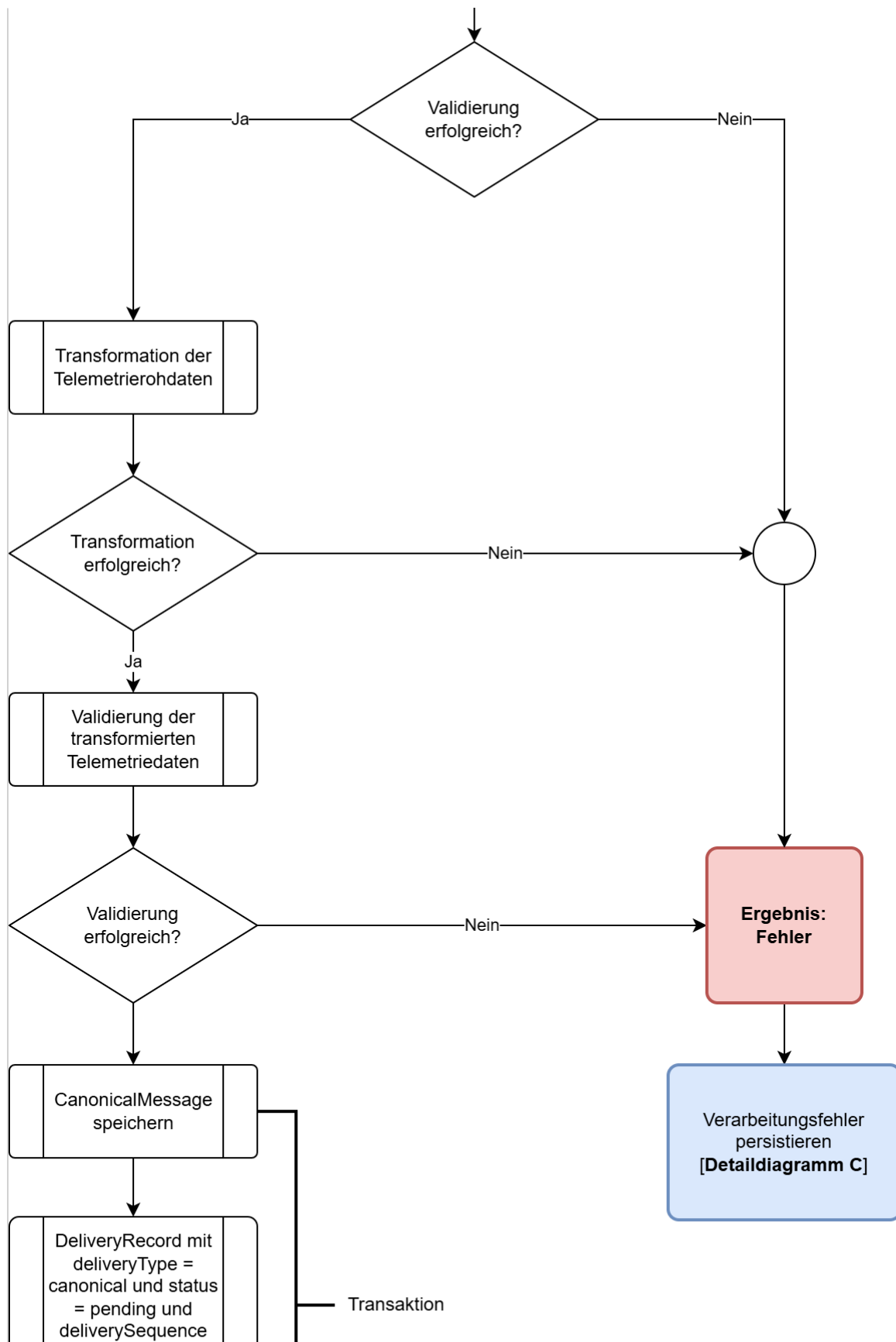
- die Transformation fehlschlägt,
- die Validierung der transformierten Daten fehlschlägt.

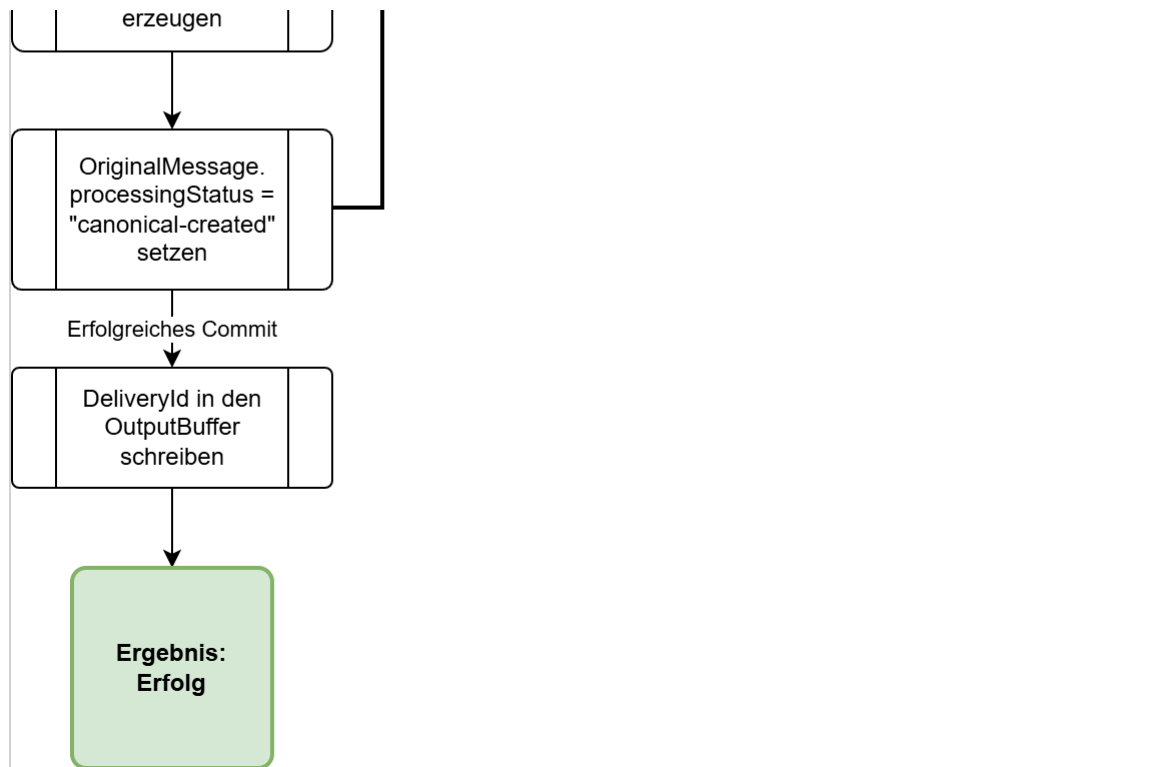
Identifikation und Clipboard-Handling - Detaildiagramm A



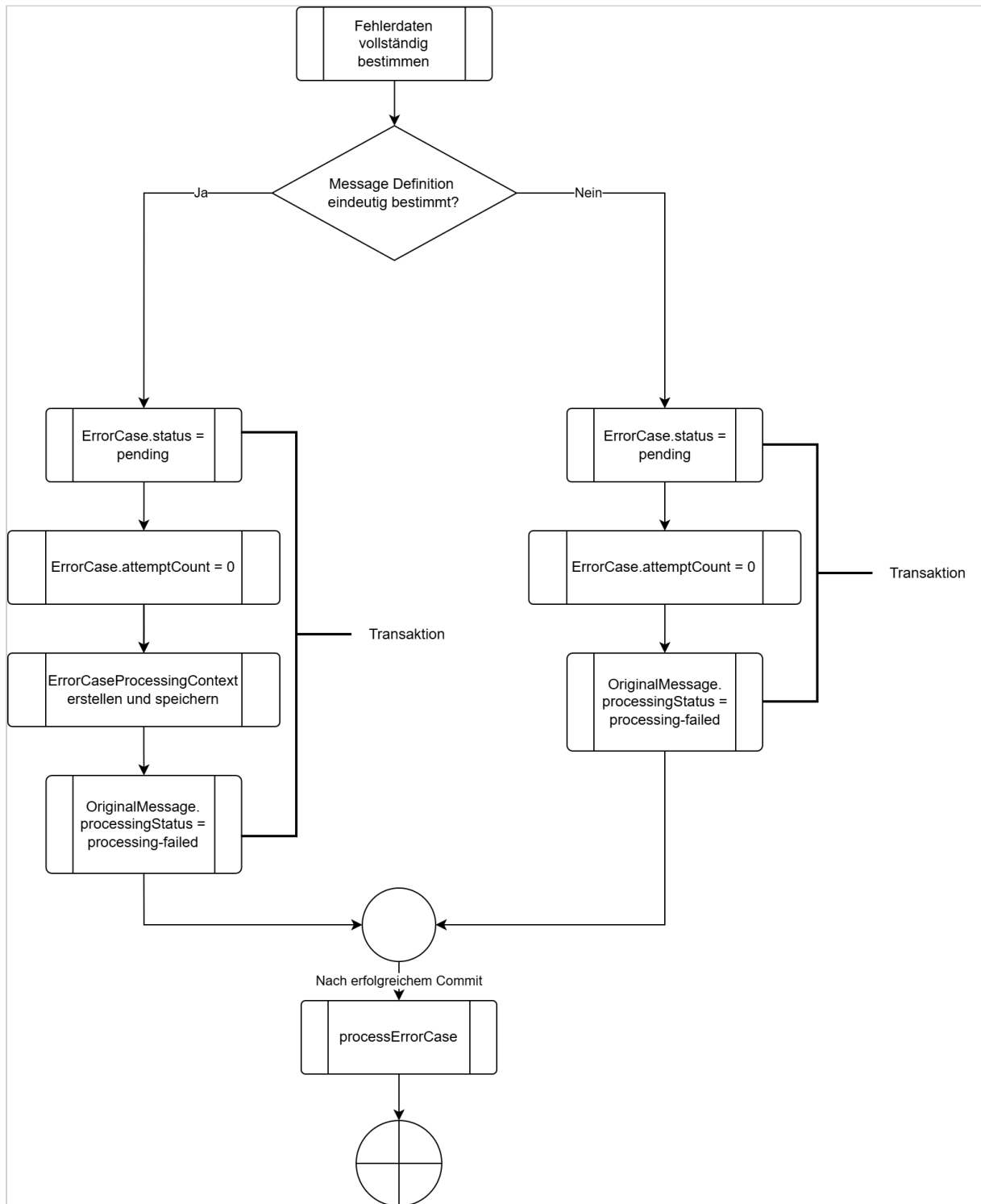
Verarbeitung der identifizierten Telemetrie - Detaildiagramm B







Persistierung eines Verarbeitungsfehlers - Detaildiagramm C



Fehlerklassifikationen

Fehlerfall	processingStage
OriginalPayload ist syntaktisch kein gültiges JSON	raw-data-validation
Rohdaten entsprechen nicht dem rawSchema	raw-data-validation
Validierungsschema kann technisch nicht geladen/verwendet werden	internal-processing
Technischer Fehler während der regulären Identifikation	internal-processing
Transformation kann nicht geladen werden oder ist technisch ungültig	internal-processing
Gültige Transformation kann nicht ausgeführt werden	transformation
Transformation liefert kein verwendbares JSON-Ergebnis	transformation
Transformierte Payload entspricht nicht dem normalizedSchema	transformed-data-validation
Schema für transformierte Telemetrie kann technisch nicht verwendet werden	internal-processing

Error handling

processErrorCase

Der `Process-Worker` behandelt einen `ErrorCase` (see page 59) über die Operation `processErrorCase(errorId)`. Dieselbe Operation wird sowohl für den ersten Diagnoseversuch als auch für alle späteren Wiederholungsversuche verwendet.

Zu Beginn eines Diagnoseversuchs aktualisiert der `Process-Worker` den `ErrorCase` (see page 59) persistent:

- `ErrorCase` (see page 59).status = processing,

- `ErrorCase (see page 59).attemptCount` um 1 erhöhen,
- `ErrorCase (see page 59).lastAttemptAt` auf den aktuellen Zeitpunkt setzen,
- `ErrorCase (see page 59).nextAttemptAt = null` setzen.

Anschließend:

- erzeugt er das Diagnosepaket unter Verwendung derselben `errorId` zunächst an einem temporären Speicherort,
- prüft er das Diagnosepaket auf Vollständigkeit und Prüfsummen,
- verschiebt bzw. ersetzt er das Diagnosepaket nach erfolgreicher Prüfung an seinen endgültigen Speicherort,
- speichert er `ErrorCase (see page 59).diagnosticPackageLocation`,
- schreibt er einen strukturierten Eintrag in die Datei `error.log`,
- setzt er `ErrorCase (see page 59).status` auf `completed` und `ErrorCase (see page 59).completedAt` = aktueller Zeitpunkt.

Schlägt eine erforderliche Diagnoseoperation fehl, setzt der `Process-Worker ErrorCase (see page 59).status` = `retry-wait` und `ErrorCase (see page 59).nextAttemptAt` = aktueller Zeitpunkt + 5 Sekunden. Der aktuelle Diagnoseversuch ist damit beendet. Der `ErrorCase-Retry-Timer` führt zum vorgesehenen Zeitpunkt einen neuen Aufruf von `processErrorCase(errorId)` aus.

Sollte der `Telemetry Data Adapter` zwischenzeitlich beendet worden sein (abgestürzt sein), so bleibt `ErrorCase.nextAttemptAt` persistent erhalten. Ist der Zeitpunkt beim Startup bereits erreicht, kann der Diagnoseversuch unmittelbar wieder aufgenommen werden. Liegt `ErrorCase (see page 59).nextAttemptAt` noch in der Zukunft, verbleibt der `ErrorCase (see page 59)` bis zu diesem Zeitpunkt im Zustand `retry-wait`. Diagnoseversuche besitzen keine fest maximale Versuchszahl. Sie werden wiederholt, bis `processErrorCase` erfolgreich abgeschlossen wurde oder administrativ eingegriffen wird.

Fehlgeschlagene Diagnoseoperationen werden, soweit `error.log` schreibbar ist, dort protokolliert. Ein Fehler beim Schreiben von `error.log` selbst wird ausschließlich über den persistenten `ErrorCase (see page 59)` und dessen Wiederholungszustand erfasst. Hierfür wird kein rekursiver Logeintrag erzwungen.

Persistenter Nachrichtenspeicher

Der persistente Nachrichtenspeicher ist die maßgebliche Zustandsquelle des Adapters (Single Source of Truth). Zu seinen Aufgaben zählen:

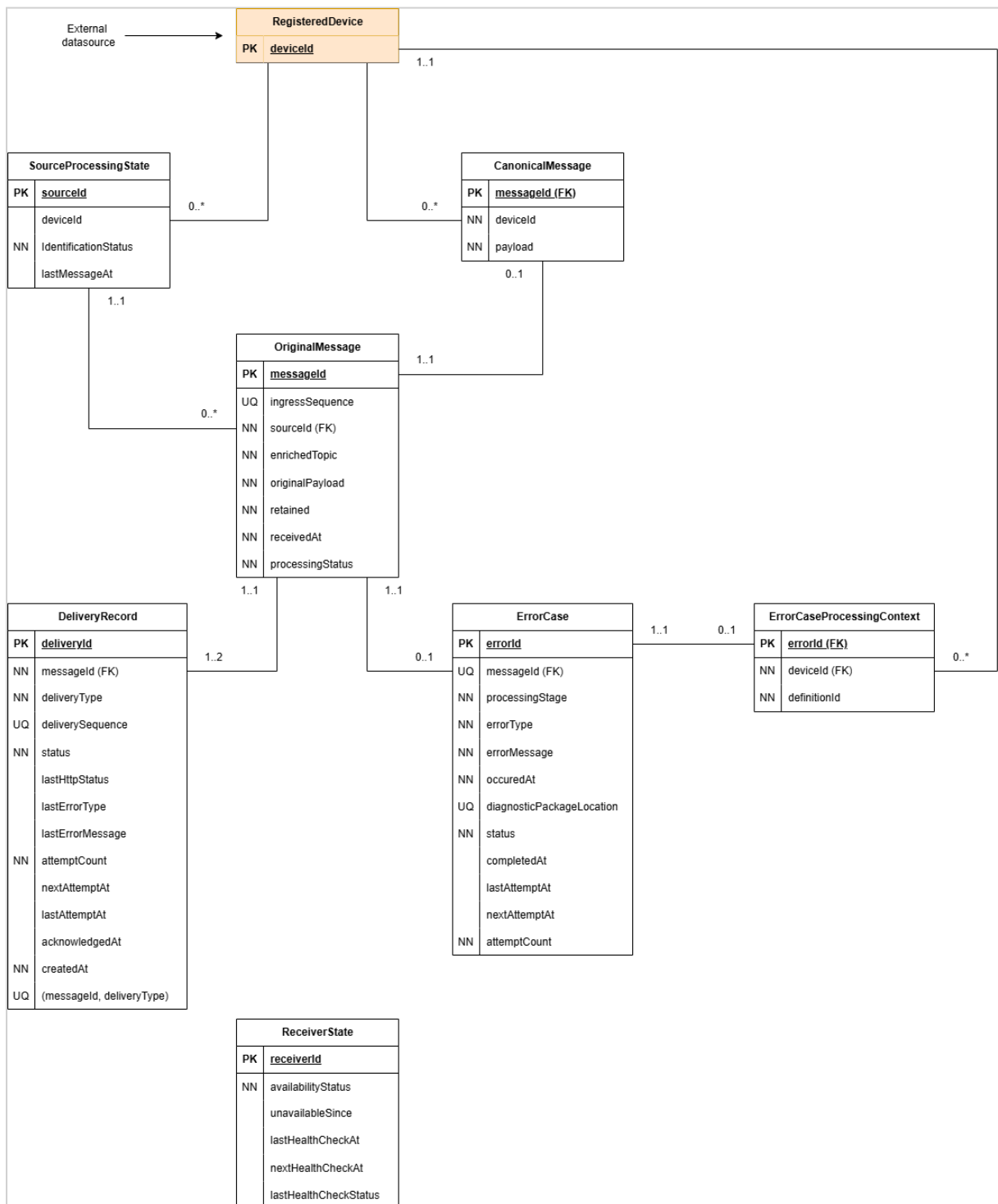
- speichern akzeptierter Telemetrirohdaten,
- erzeugen eines `DeliveryRecords` für Telemetrirohdaten in derselben Transaktion,
- speichern erfolgreich transformierter und validierter Telemetrie,

- speichern des Status der Verarbeitung von Telemetriedaten (`OriginalMessage.processingStatus`),
- speichern der HTTP-Zustellversuche, Wiederholungsplanung und Bestätigungen,
- speichern von Verarbeitungsfehlern,
- unterstützen der Wiederherstellung nach einem Prozess- oder Hostneustart,
- verhindern der Löschung akzeptierter Daten vor der Bestätigung durch den Empfänger.

Datenbank und zugrunde liegendes Dateisystem müssen Synchronisierungs- und Flush-Operationen korrekt ausführen. Eine erfolgreiche Datenbankbestätigung ist die lokale Persistenzgrenze.

Datenmodell

Alle maßgeblichen Datensätze werden im persistenten Nachrichtenspeicher abgelegt. Die FIFO-Buffer enthalten ausschließlich Identifier, die auf diese Datensätze verweisen.



Zusätzliche Semantik, die nicht im Diagramm abbildbar ist:

- Ein `DeliveryRecord` mit `DeliveryRecord.deliveryType = canonical` kann nur existieren, wenn ein Datensatz in `CanonicalMessage` mit der gleichen `messageId` existiert.

- Die Datensätze in `CanonicalMessage` und `DeliveryRecord` müssen in der gleichen Transaktion generiert werden, in der auch `OriginalMessage.processingStatus = canonical-created` gesetzt wird.
- Der `DeliveryRecord` mit `deliveryType = original` muss in der gleichen Transaktion generiert werden wie der Datensatz in `OriginalMessage`.
- `ErrorCase (see page 59).diagnosticPackageLocation` kann nur dann `null` sein, wenn `ErrorCase (see page 59).status` einen der Werte `pending`, `processing` oder `retry-wait` hat.
- Hat `ErrorCase (see page 59).status` den Wert `completed`, muss `ErrorCase (see page 59).diagnosticPackageLocation` gesetzt sein.
- `DeliveryRecord.acknowledgedAt` muss gesetzt sein, wenn `DeliveryRecord.status = acknowledged`.
- `DeliveryRecord.nextAttemptAt` muss gesetzt sein, wenn `DeliveryRecord.status = retry-wait`.
- `ReceiverState.nextHealthCheckAt` muss gesetzt sein, wenn `ReceiverState.availabilityStatus = unavailable`.
- `ErrorCase (see page 59).nextAttemptAt` muss gesetzt sein, wenn `ErrorCase (see page 59).status = retry-wait`.
- `ErrorCase (see page 59).completedAt` muss gesetzt sein, wenn `ErrorCase (see page 59).status = completed`.
- Ein `ErrorCaseProcessingContext` darf nur existieren, wenn ein zugehöriger `ErrorCase (see page 59)` mit derselben `errorId` existiert.

Wurde vor dem Auftreten eines Verarbeitungsfehlers bereits eine `Message Definition` eindeutig ausgewählt, erzeugt der `Process-Worker` in derselben Datenbanktransaktion wie den `ErrorCase (see page 59)` genau einen `ErrorCaseProcessingContext`. Er speichert darin die verwendete `deviceId` und `definitionId`. In derselben Transaktion setzt er `OriginalMessage.processingStatus = processing-failed`. Ist zum Zeitpunkt des Fehlers noch keine `Message Definition` eindeutig bestimmt, wird kein `ErrorCaseProcessingContext` erzeugt. `ErrorCase` und `OriginalMessage.processingStatus = processing-failed` werden dennoch in der selben Transaktion gespeichert.

Entitäten

OriginalMessage

Sie speichert die MQTT-Nachricht, die der Adapter akzeptiert hat.

Attribut	Erklärung
<code>messageId</code>	Primärschlüssel der akzeptierten Nachricht. Er verknüpft Originaltelemetrie, transformierte Telemetrie, Verarbeitungsfehler und Zustelldatensätze.
<code>ingressSequence</code>	Fortlaufende Nummer, die bei der Annahme der Telemetrierohdaten vergeben wird. Sie bewahrt die Reihenfolge der Originalnachrichten, ohne sich auf Zeitstempel zu verlassen.
<code>sourceId</code>	Identifiziert den Controller beziehungsweise die veröffentlichende Quelle. Der Wert wird aus dem Namensraum extrahiert, den das Mosquitto Topic Routing Plugin ergänzt.
<code>enrichedTopic</code>	Speichert den vollständig angereicherten MQTT-Topic. Der ursprüngliche Topic muss daraus verlustfrei rekonstruiert werden können.
<code>originalPayload</code>	Speichert die unveränderte MQTT-Payload (Originaltelemetriedaten / Telemetrierohdaten).
<code>retained</code>	Bewahrt das Retained-Flag der eingehenden MQTT-PUBLISH-Message auf.
<code>receivedAt</code>	Zeitpunkt, zu dem der Adapter die MQTT-Message empfangen hat.
<code>processingStatus</code>	Verfolgt Zustand und Endergebnis des Verarbeitungspaths für transformierte Telemetrie. Der Wert beeinflusst die Zustellung der Originalnachricht nicht.

Zulässige Werte für `processingStatus` :

Wert	Bedeutung
<code>pending</code>	Die <code>OriginalMessage</code> wurde persistent gespeichert, aber die Transformation wurde noch nicht begonnen. Solange die Nachricht nicht gerade vom <code>Process-Worker</code> übernommen wird, befindet sich ihre <code>messageId</code> genau einmal im <code>inputBuffer</code> .
<code>waiting-for-identification</code>	Die Nachricht kann noch nicht verarbeitet werden, weil die Quelle beziehungsweise die Drohne nicht eindeutig identifiziert werden konnte. Die <code>messageId</code> befindet sich genau einmal in der der <code>sourceId</code> entsprechenden Partition des <code>clipboardBuffer</code> und wartet auf eine spätere eindeutige Identifizierung.

Wert	Bedeutung
<code>processing</code>	Die Nachricht wird aktuell identifiziert, validiert oder transformiert. Dieser Zustand ist meist nur vorübergehend. Die Nachricht befindet sich weder im <code>inputBuffer</code> noch im <code>clipboardBuffer</code> . Sie wird aktuell vom <code>Process-Worker</code> verarbeitet.
<code>canonical-created</code>	Identifikation, Rohdatenvalidierung, Transformation und Validierung der transformierten Telemetrie waren erfolgreich. Eine <code>CanonicalMessage</code> und der zugehörige <code>DeliveryRecord</code> wurden persistent erzeugt. Die <code>messageId</code> befindet sich in keinem Buffer.
<code>processing-failed</code>	Die Verarbeitung der Telemetriedaten ist aufgrund eines Validierungs-, Transformations- oder sonstigen Verarbeitungsfehlers endgültig fehlgeschlagen. Der <code>Process-Worker</code> hat den zugehörigen <code>ErrorCase</code> persistent gespeichert. Der Bearbeitungszustand von <code>error.log</code> und Diagnosepaket wird separat über <code>ErrorCase.status</code> verfolgt. Die Originaltelemetrie bleibt davon unberührt. Die <code>messageId</code> befindet sich in keinem Buffer.
<code>abandoned-at-shutdown</code>	Die Nachricht konnte beim Herunterfahren nicht mehr eindeutig identifiziert und deshalb nicht verarbeitet werden. Der Verarbeitungspfad für transformierte Telemetrie wurde kontrolliert beendet und als Fehler dokumentiert. Die <code>messageId</code> befindet sich in keinem Buffer.
<code>abandoned-after-restart</code>	Die Nachricht stammt aus einem früheren, nicht mehr vertrauenswürdigen Quellkontext. Nach dem Neustart konnte die frühere Zuordnung zwischen <code>sourceId</code> und <code>deviceId</code> nicht sicher wiederverwendet werden, weshalb die Verarbeitung abgebrochen wurde. Die <code>messageId</code> befindet sich in keinem Buffer.

Ein zugehöriger `ErrorCase` dokumentiert den Abbruch bei:

- `OriginalMessage.processingStatus = processing-failed`,
- `OriginalMessage.processingStatus = abandoned-at-shutdown`,
- `OriginalMessage.processingStatus = abandoned-after-restart`.

CanonicalMessage

Sie speichert das validierte Ergebnis einer erfolgreichen Verarbeitung der Telemetrierohdaten.

Attribut	Erklärung
<code>messageId</code>	Primärschlüssel der <code>CanonicalMessage</code> . Er verweist auf die <code>OriginalMessage</code> , aus der die <code>CanonicalMessage</code> abgeleitet wurde.

Attribut	Erklärung
deviceId	Identifiziert die registrierte Drohne, zu der die Telemetrie gehört.
payload	Speichert die transformierten und validierten Telemetriedaten.

Ein solcher Datensatz existiert nur, wenn alle Verarbeitungsschritte erfolgreich waren. Nachrichten einer `sourceId` werden in der Reihenfolge ihres Eingangs verarbeitet. Clipboard-Nachrichten werden vor der späteren Nachricht verarbeitet, die die erforderlichen Identifikationsinformationen geliefert hat.

DeliveryRecord

Er speichert den persistenten Zustand einer Zustellung an den Empfänger.

Attribut	Erklärung
deliveryId	Unique Id einer HTTP-Zustellung. Die Verwendung als Idempotenzschlüssel ist im Abschnitt "Idempotenz" geregelt.
messageId	Verweist auf die <code>OriginalMessage</code> und korreliert deren Zustellung mit der Zustellung der zugehörigen <code>CanonicalMessage</code> .
deliveryType	Unterscheidet die Arten der Zustellungen.
deliverySequence	Fortlaufende Nummer, mit der der <code>DeliveryService</code> eine strikte Zustellreihenfolge erzwingt. Ein späterer Datensatz darf einen früheren unbestätigten Datensatz nicht überholen.
status	Aktueller Lebenszykluszustand der Zustellung.
lastHttpStatus	Der zuletzt empfangene HTTP-Status.
lastErrorType	Maschinenlesbare Version des zuletzt aufgetretenen HTTP-Zustellungsfehlers.
lastErrorMessage	Kurzbeschreibung des zuletzt aufgetretenen HTTP-Zustellungsfehlers.

Attribut	Erklärung
<code>attemptCount</code>	Anzahl der im aktuellen Zustellzyklus gestarteten HTTP-Zustellversuche einschließlich des ersten Zustellversuchs. Der Wert ist vor dem ersten Versuch 0 und wird unmittelbar vor jedem neuen HTTP-Zustellversuch um 1 erhöht. Er bestimmt das Wiederholungsverhalten und wird nach einem erfolgreichen HealthCheck oder einer administrativen Freigabe auf 0 zurückgesetzt.
<code>nextAttemptAt</code>	Frühester Zeitpunkt, zu dem eine Zustellung im Wiederholungszustand erneut versucht werden darf.
<code>lastAttemptAt</code>	Zeitpunkt, zu dem der aktuelle beziehungsweise letzte HTTP-Zustellversuch gestartet wurde. Derselbe Wert wird für den aktuellen Versuch als <code>sentAt</code> in die HTTP-Message übernommen.
<code>acknowledgedAt</code>	Zeitpunkt, zu dem der Empfänger den persistenten Empfang bestätigt hat. Der Wert bleibt bis zur Bestätigung leer.
<code>createdAt</code>	Zeitpunkt, zu dem der Zustelldatensatz erzeugt wurde.

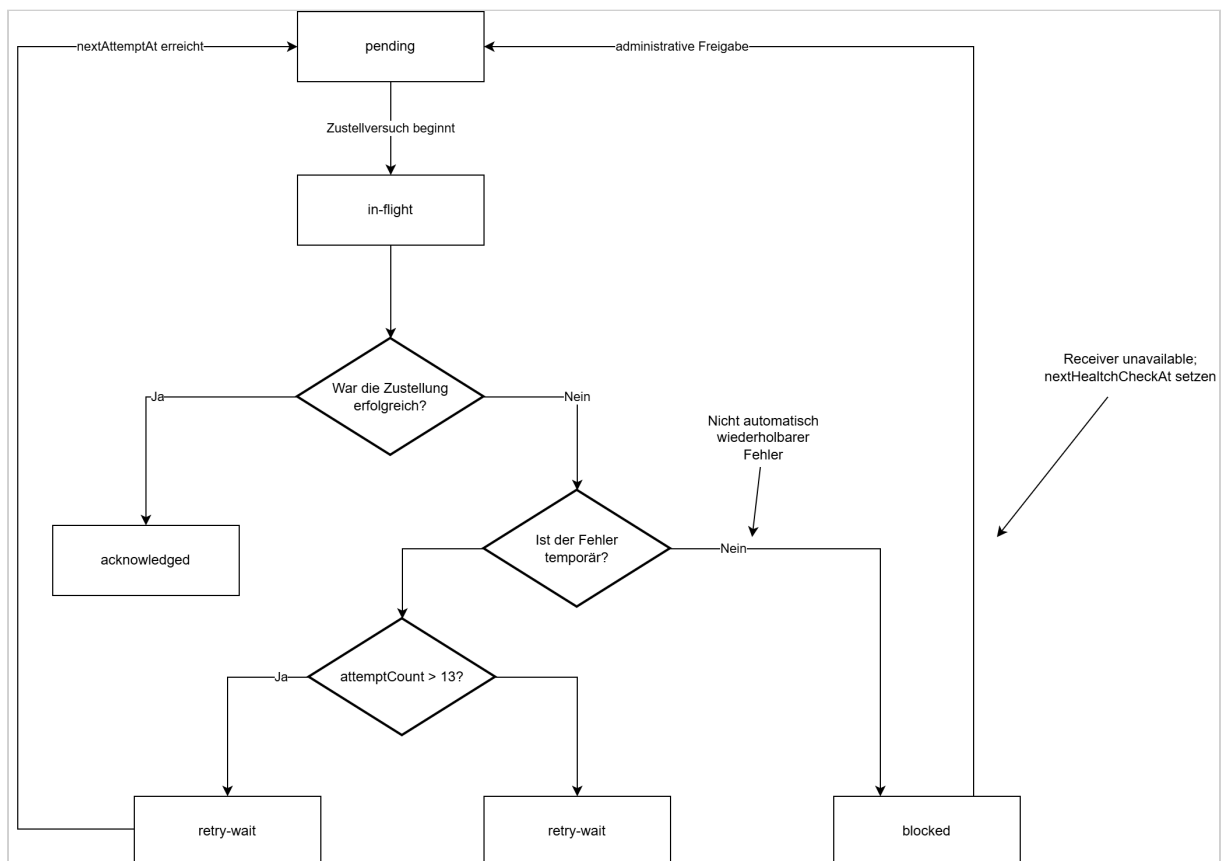
Zulässige Werte für `deliveryType` :

Wert	Bedeutung
<code>original</code>	Es handelt sich um die Zustellung einer OriginalMessage.
<code>canonical</code>	Es handelt sich um die Zustellung einer CanonicalMessage.

Zulässige Werte für `status` :

Wert	Bedeutung
<code>pending</code>	Der DeliveryRecord wurde persistent erzeugt und wartet auf seinen ersten oder nächsten Zustellversuch.
<code>in-flight</code>	Der DeliveryService verarbeitet den Datensatz aktuell. Die HTTP-Anfrage wurde gestartet oder wartet noch auf die Antwort des Overwatch-Service.

Wert	Bedeutung
retry-wait	Der letzte Zustellversuch ist fehlgeschlagen. Der Datensatz bleibt persistent gespeichert und wartet bis zum Zeitpunkt <code>nextAttemptAt</code> auf den nächsten Versuch.
acknowledged	Der Empfänger hat die Zustellung nach persistenter Speicherung erfolgreich bestätigt. <code>acknowledgedAt</code> ist gesetzt; weitere Zustellversuche sind nicht erforderlich.
blocked	Der Empfänger hat einen nicht automatisch wiederholbaren Fehler zurückgegeben. Der <code>DeliveryRecord</code> bleibt persistent gespeichert, wird nicht automatisch zugestellt und blockiert alle späteren <code>deliverySequence</code> -Werte, bis er administrativ auf pending zurückgesetzt wird.



ReceiverState

Attribut	Erklärung
receiverId	Identifiziert den Empfänger.

Attribut	Erklärung
availabilityStatus	Status der Verfügbarkeit.
unavailableSince	Zeitpunkt, ab dem der Empfänger als nicht verfügbar gekennzeichnet wurde.
lastHealthCheckAt	Zeitpunkt, zu dem der letzte HealthCheck ausgeführt wurde.
nextHealthCheckAt	Zeitpunkt, zu dem der nächste HealthCheck ausgeführt werden soll.
lastHealthCheckStatus	Letzter Status eines HealthChecks

Zulässige Werte für availabilityStatus :

Wert	Bedeutung
available	Die HTTP-Zustellung an den Empfänger funktioniert regulär.
unavailable	Die HTTP-Zustellung an den Empfänger funktioniert nicht. Zustellversuche werden eingestellt und es werden nur noch HealthChecks durchgeführt.

ErrorCase

[ErrorCase \(see page 59\)](#) speichert die Metadaten eines Fehlers im Verarbeitungspfad für transformierte Telemetrie. Der [ErrorCase \(see page 59\)](#) wird vom `Process-Worker` erzeugt und persistent gespeichert.

Attribut	Erklärung
errorId	Eindeutiger Identifikator des Fehlerfalls und des Diagnosepakets.
messageId	Verweist auf die vollständig gespeicherte fehlerhafte <code>OriginalMessage</code> .
processingStage	Verarbeitungsschritt, in dem der Fehler auftrat.
errorType	Maschinenlesbare Klassifikation des Fehlers.

Attribut	Erklärung
errorMessage	Menschenlesbare Fehlerbeschreibung.
occurredAt	Zeitpunkt des Verarbeitungsfehlers.
diagnosticPackageLocation	Speicherort oder Abrufschlüssel des vollständigen Diagnosepakets
status	Bearbeitungszustand der Fehlerbehandlung.
completedAt	Zeitpunkt, zu dem Logeintrag und Diagnosepaket vollständig erstellt wurden.
lastAttemptAt	Zeitpunkt, zu dem zuletzt versucht wurde, die Fehlerbehandlung durchzuführen.
nextAttemptAt	Zeitpunkt, zu dem die Behandlung dieses <code>ErrorCase</code> erneut durchgeführt werden soll.
attemptCount	Anzahl der gestarteten <code>processErrorCase</code> -Versuche. Der Wert ist vor dem ersten Versuch 0 und wird zu Beginn jedes Diagnoseversuchs um 1 erhöht.

Zulässige Werte für `processingStage` :

Wert	Bedeutung
identification	Der Verarbeitungspfad wurde beendet, weil die Zuordnung der Nachricht zu einer Drohne nicht abschließend durchgeführt werden konnte, beispielsweise beim Shutdown oder nach einem unsicheren Restart. Technische Fehler der Identifikationsverarbeitung werden als <code>internal-processing</code> klassifiziert.
raw-data-validation	Die Telemetrierohdaten konnten nicht validiert werden.
transformation	Die Ausführung der JSON-Transformation der Telemetrierohdaten ist fehlgeschlagen.
transformed-data-validation	Die transformierte Telemetrie konnte nicht validiert werden (entspricht nicht dem erwarteten Ergebnis).

Wert	Bedeutung
internal-processing	Technischer- oder Konfigurationsfehler, der nicht als fachlich ungültige Telemetrie klassifiziert wird, beispielsweise ein nicht verfügbares oder technisch ungültiges Validierungsschema, eine nicht verfügbare beziehungsweise technisch ungültige Transformation oder ein unerwarteter Programmfehler.

Zulässige Werte für `status` :

status	Bedeutung
pending	Der <code>ErrorCase</code> (see page 59) ist persistent gespeichert. Der <code>Process-Worker</code> hat die Erzeugung von Logeintrag und Diagnosepaket noch nicht begonnen.
processing	Der <code>Process-Worker</code> erzeugt mit Hilfe des <code>ErrorHandling Packages</code> aktuell den <code>error.log</code> -Eintrag und das Diagnosepaket.
retry-wait	Eine erforderliche Diagnoseoperation ist fehlgeschlagen, z. B. das Schreiben von <code>error.log</code> , das Erzeugen oder Prüfen des ZIP-Archivs oder dessen Ablage im Diagnostic Repository. Der <code>Process-Worker</code> führt später einen erneuten Versuch aus.
completed	Der <code>Process-Worker</code> hat Logeintrag und Diagnosepaket vollständig erstellt und geprüft.

ErrorCaseProcessingContext

`ErrorCaseProcessingContext` speichert den bereits eindeutig bestimmten Verarbeitungskontext eines `ErrorCase` (see page 59). Ein Datensatz wird nur erzeugt, wenn vor dem Verarbeitungsfehler bereits eine `Message Definition` und eine `deviceId` bestimmt wurden. Der Kontext wird in derselben Transaktion wie der zugehörige `ErrorCase` (see page 59) gespeichert. Dadurch kann die Fehlerbehandlung auch nach einem Neustart eindeutig feststellen, welche `Message Definition` und welche Drohne für den fehlgeschlagenen Verarbeitungsvorgang verwendet wurden.

Attribut	Erklärung
<code>errorId</code>	Primärschlüssel des <code>ErrorCaseProcessingContext</code> und zugleich Fremdschlüssel auf <code>ErrorCase (see page 59).errorId</code> .
<code>deviceId</code>	Identifiziert die Drohne, die für den fehlgeschlagenen Verarbeitungsvorgang bereits bestimmt worden war.
<code>definitionId</code>	Identifiziert die für die konkrete Nachricht ausgewählte <code>Message Definition</code> . Über diese Definition können das verwendete Rohdatenschema, die Transformation und das Schema für die transformierten Daten erneut eindeutig geladen werden.

SourceProcessingState

`SourceProcessingState` verfolgt die aktuelle Zuordnung zwischen Quelle und Drohne.

`SourceProcessingState` ist über einen Neustart hinweg persistent. Die Zuordnung von `sourceId` zu `deviceId` wird beim Startup jedoch zurückgesetzt, da sie nicht als vertrauenswürdiger neuer Kontext verwendet werden darf.

Attribut	Erklärung
<code>sourceId</code>	Identifiziert den Controller beziehungsweise die veröffentlichende Quelle, deren aktuelle Zuordnung verfolgt wird.
<code>identificationStatus</code>	Gibt an, ob die Quelle <code>unidentified</code> oder <code>identified</code> ist.
<code>deviceId</code>	Identifiziert die aktuell zugeordnete registrierte Drohne. Der Wert bleibt leer, solange die Quelle nicht identifiziert ist.
<code>lastMessageAt</code>	Zeitpunkt der letzten Nachricht der Quelle. Der Wert unterstützt die konfigurierte Inaktivitätsbasierte Regel zur erneuten Identifikation.

Zulässige Werte für `identificationStatus`:

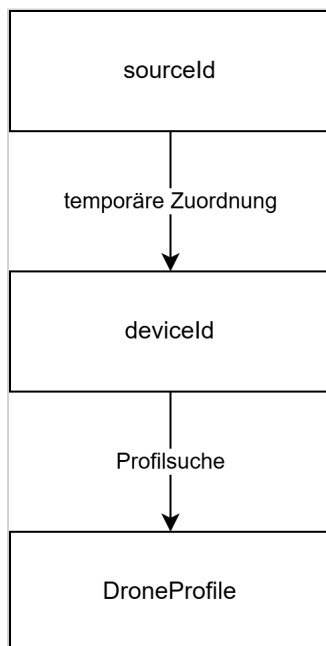
Wert	Bedeutung
unidentified	Der Quelle ist aktuell keine Drohne zugeordnet.
identified	Der Adapter hat für den aktuellen Quellkontext eine deviceId ausgewählt. Spätere Nachrichten derselben Quelle dürfen das aufgelöste Profil verwenden, auch wenn sie die Drohne nicht selbstständig identifizieren.

Bei jeder syntaktisch gültigen `OriginalMessage`, die der `Process-Worker` an das Identifier-Package übergibt, prüft er vor der Aktualisierung von `lastMessageAt`, ob eine bestehende Zuordnung weiterhin vertrauenswürdig ist. Ist `identificationStatus = identified` und überschreitet die Differenz zwischen `OriginalMessage.receivedAt` und dem bisherigen `SourceProcessingState.lastMessageAt` das konfigurierte Inaktivitätsintervall, verwirft der `Process-Worker` die bisherige Zuordnung. Er setzt `identificationStatus = unidentified` und `deviceId = null`.

Widerspricht die aktuelle Nachricht dem bisher aktiven Drohnenprofil, wird die Zuordnung ebenfalls auf `unidentified / deviceId = null` zurückgesetzt und die aktuelle Message erneut zur Identifikation verwendet. Anschließend setzt der `Process-Worker` `SourceProcessingState.lastMessageAt = OriginalMessage.receivedAt` und speichert den Zustand.

Nachrichten, die nachträglich aus dem `clipboardBuffer` abgearbeitet werden, aktualisieren `lastMessageAt` nicht. Dadurch kann ein älterer `receivedAt`-Wert den bereits gespeicherten Zeitpunkt einer später eingegangenen Nachricht nicht überschreiten.

Die Zuordnung `sourceId` zu `deviceId` ist temporär:



Eine `sourceId` darf niemals als dauerhafte Drohnenidentität behandelt werden.

ErrorHandling Package

Das `ErrorHandling` Package enthält alle Funktionen zum Management des `error.log` und der Diagnosepackages im Diagnostic Repository. Seine Funktionen werden durch den Process-Worker genutzt. Es stellt Funktionen bereit für:

- die Erstellung eines strukturierten Eintrags im `error.log`
- die Erzeugung eines Diagnosepakets im Diagnostic Repository
- die Ablage der verwendeten `Message Definition`, Validierungsschemata und Transformation als Snapshots im Diagnosepaket
- die Überprüfung des Diagnosepakets (Prüfsumme und Vollständigkeit)

Besitzt ein `ErrorCase` (see page 59) einen `ErrorCaseProcessingContext`, verwendet das `ErrorHandling-Package` dessen `definitionId`, um über das `Database-Package` die beim ursprünglichen Verarbeitungsvorgang verwendete `Message Definition` zu laden. Über diese `Message Definition` werden anschließend das Rohdatenschema, die JSONata-Transformation und das Schema für die transformierte Telemetrie geladen. Die `deviceId` des `ErrorCaseProcessingContext` bestimmt die beim Verarbeitungsvorgang bereits identifizierte Drohne. Da die referenzierten Repository-Einträge unveränderlich beziehungsweise versioniert sind, können diese Artefakte auch bei einem späteren Retry oder nach einem Neustart eindeutig wiederhergestellt werden.

Inhalt des error.log

`error.log` wird als Text-Datei geführt. Jede Zeile enthält einen eigenständigen strukturierten Logdatensatz. Das Schreiben von Einträgen in `error.log` erfolgt mit At-least-once-Semantik.

Das Anhängen eines Logeintrages und die persistente Aktualisierung des zugehörigen `ErrorCase` (see page 59) können nicht gemeinsam in einer atomaren Transaktion erfolgen. Wird der Adapter zwischen diesen beiden Operationen unterbrochen, kann bei der Wiederholung ein weiterer Logeintrag mit derselben `errorId` entstehen. Mehrere Logeinträge mit derselben `errorId` beschreiben denselben logischen Fehlerfall. Anwendungen, die das `error.log` auswerten, müssen diese Einträge anhand von `errorId` korrelieren bzw. deduplizieren.

Attribut	Bedeutung
<code>timestamp</code>	Zeitpunkt, zu dem der Logeintrag geschrieben wurde.
<code>level</code>	Log-Level
<code>component</code>	Komponente, in der der Verarbeitungsfehler auftrat.
<code>errorId</code>	Identifikator des Fehlerfalls und Diagnosepakets.
<code>messageId</code>	Identifikator der fehlerhaften Originalnachricht.
<code>sourceId</code>	Controller beziehungsweise Quelle der Nachricht.
<code>processingStage</code>	Verarbeitungsschritt, in dem der Fehler auftrat.
<code>errorType</code>	Maschinenlesbare Fehlerklassifikation.
<code>errorMessage</code>	Menschenlesbare Fehlerbeschreibung.
<code>occurredAt</code>	Zeitpunkt des ursprünglichen Verarbeitungsfehlers.
<code>diagnosticPackageLocation</code>	Vollständiger Dateipfad des ZIP-Archivs.

Die vollständige Nutzlast und die vollständigen Schemata werden nicht in `error.log` geschrieben. Sie befinden sich im Diagnosepaket.

Diagnosepaket

Das Diagnosepaket legt alle für Reproduktion und Analyse benötigten Daten gemeinsam unter derselben `errorId` ab. Die folgende Struktur ist die interne Struktur eines ZIP-Archivs (jedes Diagnosepaket ist ein ZIP-Archiv).

```
errors/
├─ {errorId}/
│  ├─ manifest.json
│  ├─ error.json
│  ├─ original-message.json
│  ├─ mqtt-metadata.json
│  ├─ identification/
│  │  ├─ profiles/
│  │  ├─ Message-Definitions/
│  │  └─ schemas/
│  ├─ validation/
│  │  ├─ raw-schema.json
│  │  ├─ raw-validation-result.json
│  │  ├─ canonical-schema.json
│  │  └─ canonical-validation-result.json
│  └─ transformation/
│     ├─ transformation.jsonata
│     ├─ transformation-metadata.json
│     └─ transformed-payload.json
```

Nicht vorhandene Zwischenergebnisse werden ausgelassen. Besitzt der `ErrorCase` (see page 59) einen `ErrorCaseProcessingContext`, lädt das `ErrorHandling`-Package die durch `definitionId` referenzierte unveränderliche `Message Definition` sowie die von ihr referenzierten Validierungsschemata und die Transformation. Diese Artefakte werden als Dateien in das Diagnosepaket kopiert, damit das Diagnosepaket vollständig und unabhängig vom Repository ausgewertet werden kann. Da bereits verwendete Repository-Versionen nicht überschrieben werden, entsprechen die geladenen Artefakte den beim ursprünglichen Verarbeitungsvorgang verwendeten Versionen.

Rekonstruktion der Zwischenergebnisse

- Die für das Diagnosepaket benötigten Zwischenergebnisse werden bei `processErrorCase` anhand der persistenten Originalnachricht und des `ErrorCaseProcessingContext` reproduziert. Die hierfür verwendeten Funktionen des `Validator`- und `Transformer`-Packages sind zustandslos und verändern während der Diagnose weder `OriginalMessage.processingStatus` noch andere persistente Verarbeitungszustände. Besitzt der `ErrorCase` (see page 59) keinen `ErrorCaseProcessingContext`, können

nur die Artefakte erzeugt werden, für die keine eindeutig ausgewählte Message Definition erforderlich ist. Nicht erzeugbare Dateien werden gemäß der beschriebenen Paketstruktur ausgelassen.

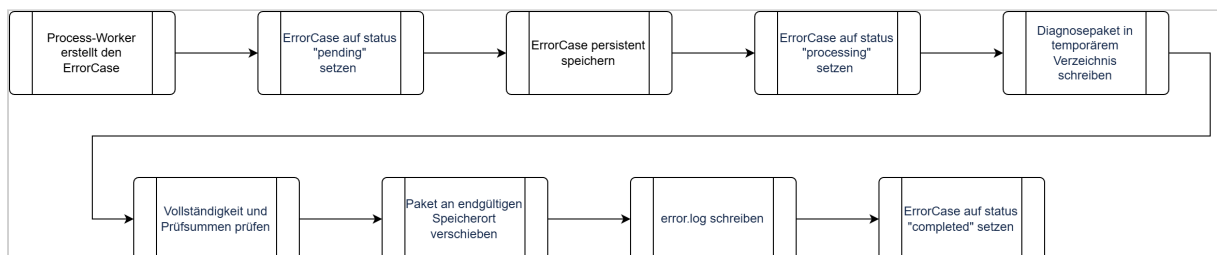
Besitzt der ErrorCase einen ErrorCaseProcessingContext, wird zunächst die durch definitionId referenzierte Message Definition geladen. Anschließend gelten folgende Regeln:

- raw-schema.json enthält das durch MessageDefinition.rawSchema referenzierte Rohdatenschema.
- raw-validation-result.json wird erzeugt, indem OriginalMessage.originalPayload erneut mit parseRawPayload eingelesen und anschließend gegen MessageDefinition.rawSchema validiert wird.
- transformation.jsonata enthält die durch MessageDefinition.transformationId referenzierte Transformation.
- transformation-metadata.json enthält mindestens definitionId und transformationId sowie vorhandene Metadaten der Transformation.
- transformed-payload.json wird erzeugt, wenn die Rohdatenvalidierung erfolgreich ist und die Transformation reproduzierbar ausgeführt werden kann.
- canonical-schema.json enthält das durch MessageDefinition.normalizedSchema referenzierte Schema.
- canonical-validation-result.json wird erzeugt, wenn eine transformierte Payload vorliegt, und enthält das Ergebnis der erneuten Validierung gegen MessageDefinition.normalizedSchema.

Schlägt ein für die Diagnose notwendiger Rekonstruktionsschritt technisch fehl, gilt dies als fehlgeschlagene Diagnoseoperation. `processErrorCase` verwendet dafür den bereits definierten Zustand `retry-wait`. Es wird kein weiterer rekursiver `ErrorCase` erzeugt.

manifest.json enthält mindestens errorId, messageId, Erstellungszeitpunkt sowie für jede Datei Pfad, Inhaltstyp, Größe und Prüfsumme.

Schreibablauf



Kann nicht sichergestellt werden, ob der Logeintrag bereits geschrieben wurde, darf er erneut geschrieben werden. Dabei gilt die im Abschnitt **Inhalt des `error.log`** definierte At-least-once-Semantik.

Fehlerklassifikation

Persistierung der MQTT-Message schlägt fehl:

- MQTT-PUBACK nicht freigeben und dadurch Backpressure anwenden,
- `persistMqttMessage` gemäß dem definierten Retry-Schema erneut ausführen,
- nach insgesamt 5 erfolglosen Versuchen die MQTT-Verbindung trennen, ohne die persistente Session oder das Abonnement zu löschen.

Drohne kann noch nicht identifiziert werden:

- Originalzustellung läuft weiter,
- Verarbeitungspfad für transformierte Telemetrie schreibt in den `clipboardBuffer`.

Drohne bleibt beim Herunterfahren oder nach einem unsicheren Neustart nicht identifiziert:

- Originalzustellung bleibt garantiert,
- Verarbeitungspfad für transformierte Telemetrie wird abgebrochen und protokolliert.

Validierung oder Transformation schlägt fehl:

- Originalzustellung bleibt garantiert,
- der `Process-Worker` persistiert den `ErrorMessage` (see page 59) und, sofern bereits eine `Message Definition` bestimmt wurde, den zugehörigen `ErrorMessageProcessingContext`,
- der `Process-Worker` setzt den terminalen `OriginalMessage.processingStatus` in derselben Transaktion,
- nach erfolgreichem Commit ruft der `Process-Worker` `processErrorMessage(errorId)` auf,
- `processErrorMessage` erzeugt Diagnosepaket und `error.log`-Eintrag gemäß der definierten Retry-Semantik.

Empfänger ist nicht verfügbar:

- `DeliveryRecords` bleiben ausstehend,
- Healthchecks werden fortgesetzt,
- Zustellung wird nach der Wiederherstellung fortgesetzt.

Adapter stürzt ab:

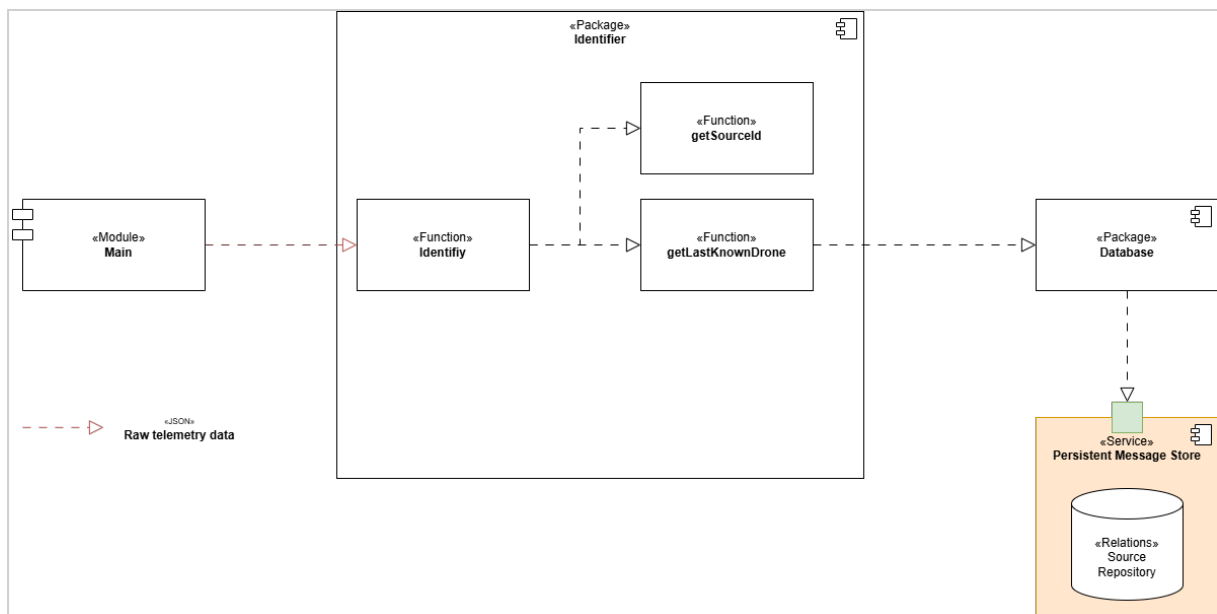
- akzeptierte Telemetrie und ausstehende Zustellungen werden aus der Datenbank wiederhergestellt

Persistenter Speicher ist voll oder nicht verfügbar:

- Akzeptieren und Bestätigen von MQTT-Nachrichten stoppen

Empfänger bestätigt Speicherung, aber HTTP-Antwort geht verloren:

- Adapter wiederholt dieselbe deliveryId
- Empfänger dedupliziert

Identifizier-Package

Das Identifizier-Package erhält die Telemetrierohdaten als Eingabe. In der Funktion `getSourceId` extrahiert es die `sourceId` aus dem Topic, d. h. aus dem Topic `raw/source/73da918/thing/product/TH7825301867/osd` wird der Wert `73da918` entnommen. Dadurch kann der Controller / das Gateway identifiziert werden.

Message Definition

Zu jeder Message, welche ein Drohnentyp senden kann, muss eine Message Definition existieren. Nun wird versucht, die vorliegende Message zu erkennen. Dazu werden alle für die aktuelle Identifikation relevanten Message Definition -Datensätze gegen die Nachricht geprüft. Eine Nachricht gilt nur dann als eindeutig zugeordnet, wenn anschließend genau eine passende Message

Definition verbleibt. Eine Message Definition enthält folgende Attribute (hier als JSON dargestellt):

Message Definition	
1	{
2	"definitionId": 16,
3	"topic": {
4	"normalizedTopic": "thing/product/state",
5	"topicPattern": "^thing/product/[A-Z0-9]{12}/state\$",
6	},
7	"payloadSelectors": [
8	{
9	"pointer": "/method",
10	"operator": "equals",
11	"value": "target_detect_result_report"
12	}
13	],
14	"rawSchema": 42,
15	"normalizedSchema": 34,
16	"transformationId": 47
17	}

Attribut	Bedeutung
definitionId	Eindeutige Kennung dieser Message Definition
normalizedTopic	Ein Topic, aus dem alle veränderlichen Bestandteile (Seriennummer, Modellnummer, ...) entfernt wurden. Er dient zur Identifikation von Message Definition -Datensätzen, die zu diesem Topic passen können.
topicPattern	Ein regulärer Ausdruck, mit dessen Hilfe aus einem Topic (z. B. thing/product/TH7825301867/state) der normalisierte Topic thing/product/state kreiert werden kann.

Attribut	Bedeutung
payloadSelectors	Payload Selectors sind Regeln, die zur einfachen Identifizierung einer Nachricht anhand der Payload dienen. Im obigen Beispiel bedeutet <code>/ method equals target_detect_result_report</code> , dass in der Payload der Message das Attribut <code>method</code> auf erster Ebene mit dem Wert <code>target_detect_result_report</code> vorkommen muss. Wird diese Kombination vorgefunden, ist der betreffende <code>payloadSelector</code> erfüllt. Eine Nachricht gilt erst dann als eindeutig identifiziert, wenn <code>topicPattern</code> und <code>payloadSelectors</code> zu genau einer Kombination aus Drohnenprofil und <code>Message Definition</code> passen.
rawSchema	ID des JSON-Schemas zur Validierung der Telemetrierohdaten dieser <code>Message Definition</code> . Das Schema wird nicht zur Identifikation der Nachricht verwendet.
normalizedSchema	Numerische Id des JSON-Schemas zur Validierung der transformierten Telemetriedaten der Message.
transformationId	ID der JSONata-Transformation, mit der die Telemetrierohdaten dieser <code>Message Definition</code> transformiert werden.

Für jede zu verarbeitende Nachricht muss das `Identifizier-Package` eine eindeutig passende `Message Definition` ermitteln. Ist für die `sourceId` bereits eine vertrauenswürdige `deviceId` bekannt, werden zunächst die `Message Definition`-Datensätze dieses Drohnenprofils betrachtet. Für jede Definition wird geprüft, ob der ursprüngliche MQTT-Topic dem `topicPattern` entspricht und die definierten `payloadSelectors` erfüllt sind. Ist noch keine vertrauenswürdige `deviceId` bekannt oder passt keine `Message Definition` des aktuell zugeordneten Drohnenprofils, wird die Identifikation gemäß der bestehenden Identifikationsstrategie auf die übrigen registrierten Drohnenprofile ausgeweitet.

Eine Nachricht gilt nur dann als eindeutig identifiziert, wenn genau eine passende Kombination aus Drohnenprofil und `Message Definition` ermittelt werden kann. Mehrdeutige Ergebnisse gelten nicht als erfolgreiche Identifikation. Das `rawSchema` wird nicht zur Identifikation verwendet. Die fachliche Prüfung der Telemetrierohdaten gegen dieses Schema erfolgt anschließend ausschließlich durch das `Validator-Package`. Bei erfolgreicher Identifikation liefert das `Identifizier-Package` dem `Process-Worker` mindestens `sourceId`, `deviceId` und die ausgewählte `Message Definition` zurück.

- `Message Definition`-Datensätze und die von ihnen referenzierten Validierungsschemata und Transformationen werden versioniert beziehungsweise unveränderlich behandelt. Der Inhalt eines bereits vergebenen Identifikators darf nicht nachträglich durch eine fachlich andere Definition ersetzt werden. Wird eine `Message Definition`, ein Validierungsschema oder eine Transformation geändert, erhält die neue Version einen neuen Identifikator. Bereits referenzierte Versionen bleiben mindestens so lange lesbar, wie sie von persistenten `ErrorCase` (see page 59) - beziehungsweise `ErrorCaseProcessingContext`-Datensätzen benötigt werden. Dadurch kann die Fehlerbehandlung über `definitionId` auch bei einem späteren Retry oder nach einem Neustart wieder genau die Artefakte laden, die beim ursprünglichen Verarbeitungsvorgang verwendet wurden.

Validator-Package

Das `Validator-Package` ist für die Validierung der Telemetriedaten im Verarbeitungspfad des `Process-Workers` verantwortlich. Es prüft sowohl die Telemetrierohdaten als auch die vom Transformer erzeugten transformierten Telemetriedaten. Das Package ist zustandslos. Es verändert keine persistenten Datensätze und besitzt keinen eigenen Buffer. Insbesondere setzt es weder `OriginalMessage.processingStatus` noch erzeugt es einen `ErrorCase` (see page 59) . Diese Entscheidungen trifft ausschließlich der `Process-Worker` .

Das `Validator-Package` führt keine Identifikation und keine Topic-Normalisierung durch. Das für eine fachliche Validierung zu verwendende Schema wird bereits durch die zuvor ausgewählte `Message Definition` bestimmt. Für die Rohdaten wird `MessageDefinition.rawSchema` verwendet. Für die transformierten Daten wird `MessageDefinition.normalizedSchema` verwendet.

Wesentliche Funktionen

Funktion	Aufgabe
<code>parseRawPayload</code>	Liest <code>OriginalMessage.originalPayload</code> als JSON ein. Bei syntaktisch ungültigem JSON liefert die Funktion einen Fehler zurück. Die persistente Originalpayload wird dabei nicht verändert.
<code>validate</code>	Validiert eine JSON-Payload gegen genau das durch <code>schemaId</code> referenzierte JSON-Schema und liefert ein <code>ValidationResult</code> zurück. Dieselbe Funktion wird für Rohdaten und transformierte Daten verwendet.

Funktion	Aufgabe
<code>getValidationSchema</code>	Lädt das durch <code>schemaId</code> eindeutig bestimmte JSON-Schema über das Database-Package.

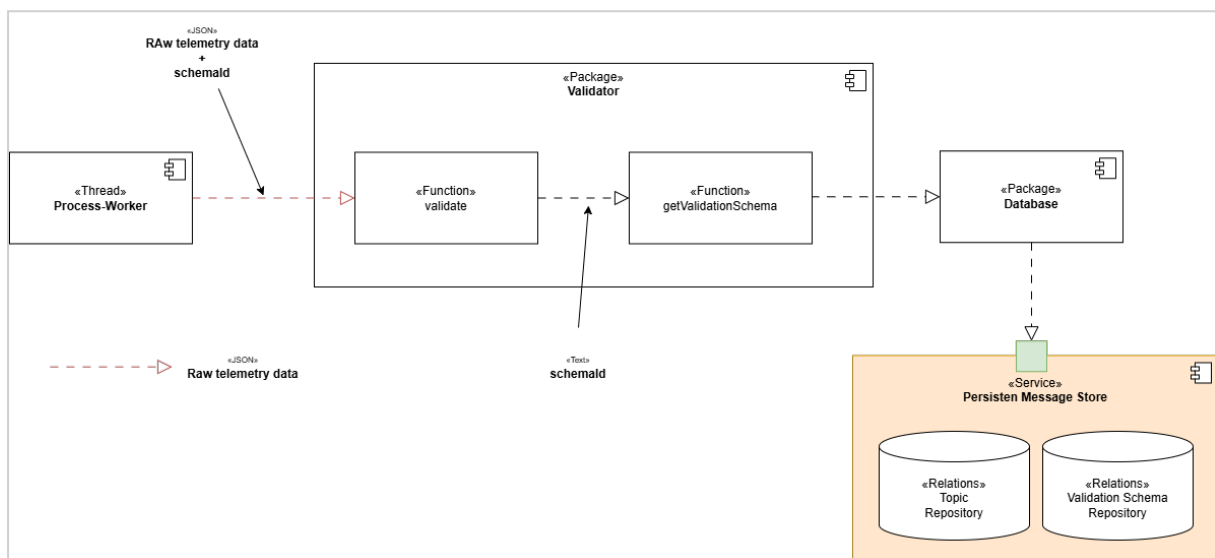
ValidationResult

`validate` liefert ein strukturiertes `ValidationResult` zurück.

Attribut	Bedeutung
<code>valid</code>	Gibt an, ob die Payload dem angegebenen Schema entspricht.
<code>schemaId</code>	ID des tatsächlich verwendeten Schemas.
<code>errors</code>	Maschinenlesbare Liste der festgestellten Schema-Verletzungen. Bei erfolgreicher Validierung ist die Liste leer.

Eine nicht als JSON interpretierbare Originalpayload gilt als fehlgeschlagene Rohdatenvalidierung. Kann dagegen ein referenziertes Validierungsschema nicht geladen werden oder ist das Schema selbst technisch nicht verwendbar, handelt es sich nicht um ungültige Telemetrie, sondern um einen technischen Verarbeitungsfehler. Der `Process-Worker` verwendet hierfür `ErrorCase` (see page 59) `.processingStage = internal-processing`.

Validierung der Telemetrierohdaten



Nachdem `parseRawPayload` erfolgreich war und das `Identifizier-Package` eine `Message Definition` ermittelt hat, ruft der `Process-Worker` `validate(parsedPayload, MessageDefinition.rawSchema)` auf. `validate` lädt mit `getValidationSchema` das referenzierte JSON-Schema und prüft die Payload gegen dieses Schema. Ist `ValidationResult.valid = true`, setzt der `Process-Worker` die Verarbeitung mit dem Transformer fort. Ist `ValidationResult.valid = false`, setzt der `Process-Worker` für den gemeinsamen Fehlerpfad `processingStage = raw-data-validation` und einen passenden `errorType`. Da `deviceId` und `Message Definition` zu diesem Zeitpunkt bereits eindeutig bestimmt sind, werden `ErrorCase` (see page 59), `ErrorcaseProcessingContext` und `OriginalMessage.processingStatus = processing-failed` in derselben Transaktion persistent gespeichert. Nach erfolgreichem Commit ruft der `Process-Worker` `processErrorCase(errorId)` auf. Der bereits bei der MQTT-Annahme erzeugte `DeliveryRecord` für die Originaltelemetrie bleibt davon unberührt.

Validierung der transformierten Telemetriedaten

Nach erfolgreicher Transformation erhält der `Process-Worker` ein `TransformationResult` mit `successful = true`. Die in `TransformationResult.payload` enthaltene transformierte JSON-Payload wird anschließend mit `validate(TransformationResult.payload, MessageDefinition.normalizedSchema)` validiert.

Bei `ValidationResult.valid = true` erzeugt der `Process-Worker` in der bereits definierten Transaktion:

- die `CanonicalMessage`,
- den zugehörigen `DeliveryRecord` mit `deliveryType = canonical`,
- die nächste globale `deliverySequence`,
- `OriginalMessage.processingStatus = canonical-created`.

Erst nach erfolgreichem Commit wird die `deliveryId` in den `outputBuffer` aufgenommen. Bei `ValidationResult.valid = false` werden weder `CanonicalMessage` noch der `DeliveryRecord` der transformierten Telemetrie erzeugt. Der `Process-Worker` setzt für den gemeinsamen Fehlerpfad `processingStage = transformed-data-validation` und einen passenden `errorType`. Da `deviceId` und `Message Definition` bereits bestimmt sind, werden `ErrorCase` (see page 59), `ErrorcaseProcessingContext` und `OriginalMessage.processingStatus = processing-failed` in derselben Transaktion persistent gespeichert. Nach erfolgreichem Commit ruft der `Process-Worker` `processErrorCase(errorId)` auf.

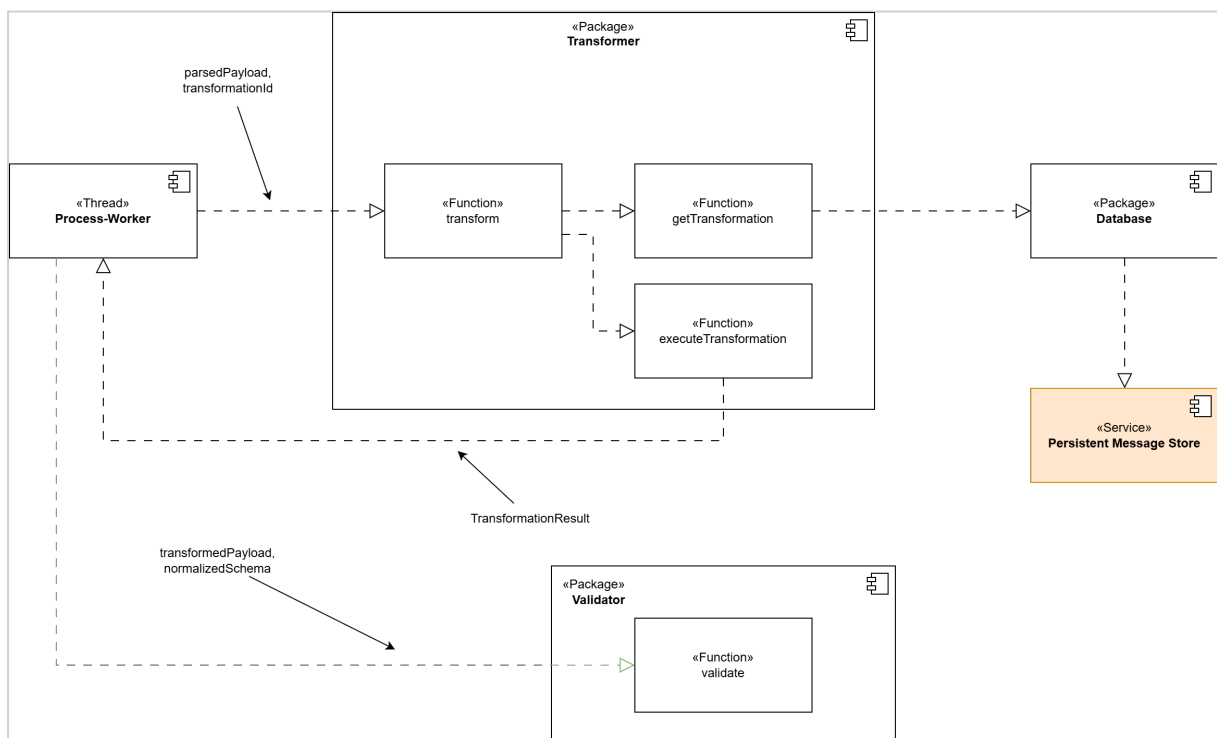
Transformer-Package

Das `Transformer-Package` transformiert bereits erfolgreich validierte Telemetrierohdaten in das interne Telemetriedatenformat. Der `Process-Worker` ruft das `Transformer-Package` erst auf, nachdem:

- die Originalpayload erfolgreich als JSON eingelesen wurde,
- die Nachricht eindeutig identifiziert wurde,
- eine `Message Definition` ausgewählt wurde und
- die Telemetrierohdaten erfolgreich gegen `MessageDefinition.rawSchema` validiert wurden.

Als Eingabe erhält das `Transformer-Package` die validierte JSON-Payload und `MessageDefinition.transformationId`.

Das Package ist zustandslos. Es persistiert keine Daten und verändert weder `OriginalMessage.processingStatus` noch andere persistente Zustände. Fehlerbehandlung und Persistierung bleiben Aufgabe des `Process-Workers`.



Wesentliche Funktionen

Funktion	Aufgabe
<code>transform</code>	Koordiniert genau einen Transformationsversuch. Lädt die durch <code>transformationId</code> referenzierte Transformation, führt sie aus und liefert ein <code>TransformationResult</code> zurück.
<code>getTransformation</code>	Lädt die durch <code>transformationId</code> eindeutig bestimmte JSONata-Transformation über das <code>Database-Package</code> .
<code>executeTransformation</code>	Führt die geladene JSONata-Transformation gegen die übergebene validierte Rohdaten-Payload aus.

TransformationResult

`transform` liefert ein strukturiertes `TransformationResult` an den `Process-Worker` zurück.

Attribut	Bedeutung
<code>successful</code>	Gibt an, ob die Transformation erfolgreich ausgeführt wurde.
<code>payload</code>	Enthält bei erfolgreicher Transformation die erzeugte transformierte JSON-Payload. Bei einem Fehler ist der Wert leer.
<code>errorType</code>	Maschinenlesbare Fehlerklassifikation bei fehlgeschlagener Transformation. Bei erfolgreicher Transformation leer.
<code>errorMessage</code>	Menschenlesbare Beschreibung eines Transformationsfehlers. Bei erfolgreicher Transformation leer.

Transformation der Telemetrierohdaten

Nachdem die Telemetrierohdaten erfolgreich validiert wurden, ruft der `Process-Worker` `transform(parsedPayload, MessageDefinition.transformationId)` auf. `transform` ruft zunächst `getTransformation(transformationId)` auf. Die Funktion lädt die referenzierte JSONata-Transformation über das `Database-Package`. Anschließend führt `executeTransformation` die geladene Transformation gegen die validierte Rohdaten-Payload aus.

Bei erfolgreicher Ausführung wird die transformierte JSON-Payload in `TransformationResult.payload` zurückgegeben. Der Transformer persistiert das Ergebnis nicht. Der `Process-Worker` übergibt `TransformationResult.payload` zunächst an das `Validator-Package` und validiert es gegen `MessageDefinition.normalizedSchema`.

Erst wenn auch diese Validierung erfolgreich war, erzeugt der `Process-Worker` die `CanonicalMessage` und den zugehörigen `DeliveryRecord`.

Fehlerbehandlung

Transformation kann nicht geladen werden

Kann die durch `transformationId` referenzierte Transformation nicht geladen werden oder existiert der referenzierte Repository-Eintrag nicht, handelt es sich nicht um fehlerhafte Telemetrie, sondern um einen technischen bzw. Konfigurationsfehler. Der `Process-Worker` verwendet `ErrorCase (see page 59)`. `processingStage = internal-processing`. Ein Beispiel für einen solchen Fehler könnte sein: `transformation-definition-unavailable`.

Transformation ist technisch ungültig

Kann die gespeicherte JSONata-Transformation nicht kompiliert beziehungsweise technisch nicht verwendet werden, gilt ebenfalls: `ErrorCase (see page 59)`. `processingStage = internal-processing`. Ein Beispiel für einen solchen Fehler könnte eine ungültige Transformation sein (`errorType = transformation-definition-invalid`).

Transformation kann mit den Daten nicht ausgeführt werden

Ist die Transformation selbst gültig, ihre Ausführung gegen die konkrete Rohdaten-Payload schlägt jedoch fehl, gilt: `ErrorCase (see page 59)`. `processingStage = transformation` (`errorType = transformation-execution-failed`).

Ergebnis ist nicht als JSON-Payload verwendbar

Liefert die Transformation kein Ergebnis, das als transformierte JSON-Payload weiterverarbeitet werden kann, gilt ebenfalls: `ErrorCase (see page 59)`. `processingStage = transformation`. Die fachliche Struktur des Ergebnisses prüft dagegen nicht der Transformer. Diese Prüfung erfolgt anschließend ausschließlich durch das `Validator-Package` (`errorType = transformation-result-invalid`).

Für alle hier beschriebenen Transformationsfehler führt der `Process-Worker` anschließend den gemeinsamen Fehlerpfad aus. Da vor dem Aufruf des `Transformer-Packages` bereits eine eindeutige `Message Definition` und `deviceId` vorliegen, werden dabei `ErrorCase (see page 59)`,

`ErrorMessageProcessingContext` und `OriginalMessage.processingStatus = processing-failed` in derselben Datenbanktransaktion gespeichert. Nach erfolgreichem Commit wird `processErrorMessage(errorId)` aufgerufen.

Determinismus

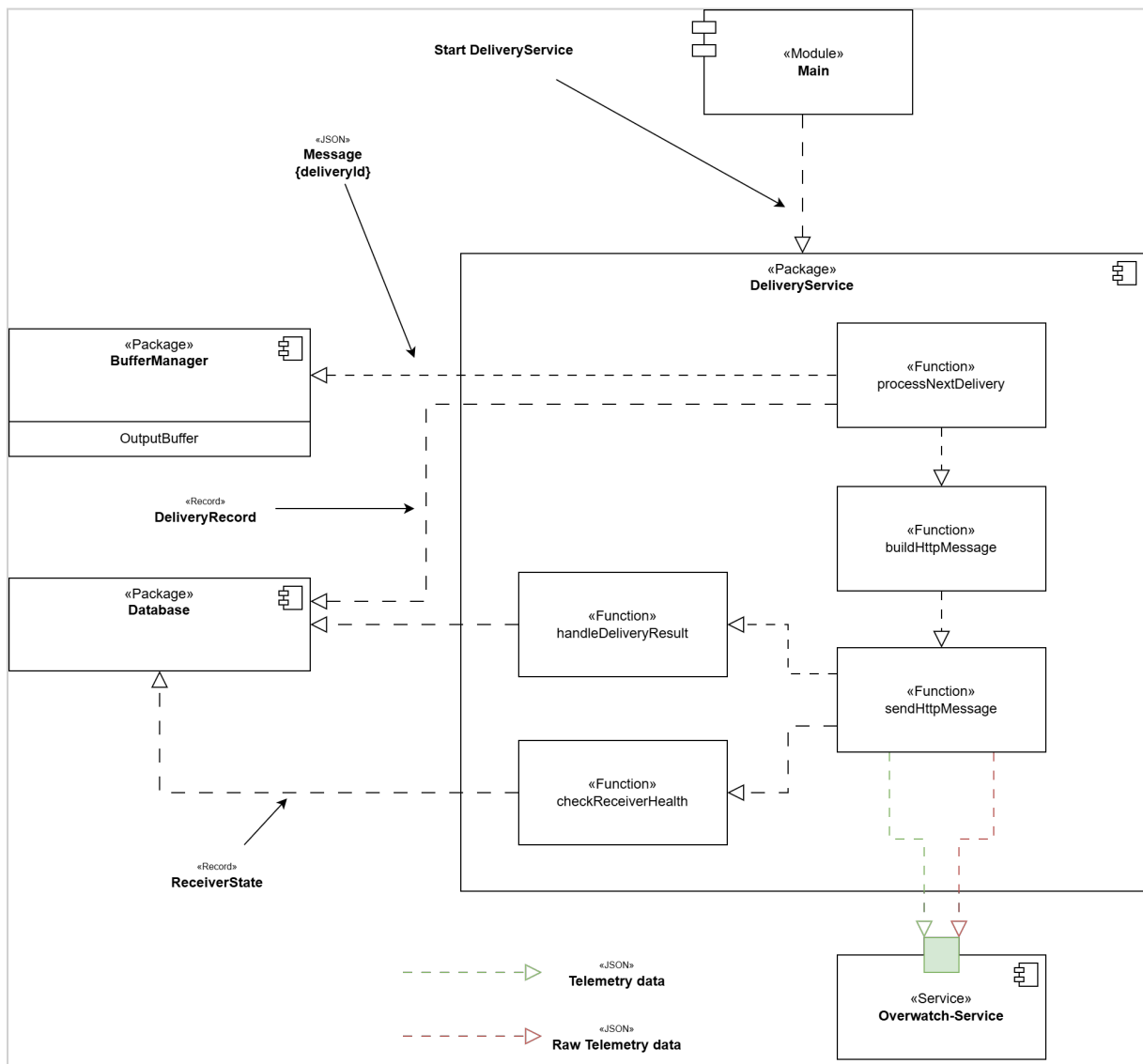
Für dieselbe validierte Eingabe und dieselbe `transformationId` muss der Transformer dasselbe Transformationsergebnis erzeugen. Eine Transformation darf deshalb nicht von veränderlichen externen Zuständen wie Netzwerkzugriffen, aktuellen Repository-Werten, Zufallswerten oder dem aktuellen Zeitpunkt abhängen. Der einzige fachliche Eingabewert der JSONata-Transformation ist die validierte Telemetrierohdaten-Payload. Repository-Zugriffe des Transformer-Packages dienen ausschließlich dazu, die durch die `transformationId` bestimmte Transformation zu laden.

DeliveryService-Package

Das `DeliveryService`-Package ist für die persistente At-least-once-Zustellung von Telemetrierohdaten und transformierter Telemetrie an den Overwatch-Service verantwortlich. Die zuzustellenden Daten selbst werden nicht im `DeliveryService` gehalten. Der `outputBuffer` enthält ausschließlich `deliveryId`-Werte und bildet die globale Zustellreihenfolge ab. Jeder `DeliveryRecord` mit `status != acknowledged` befindet sich genau einmal im `outputBuffer`. Die Reihenfolge entspricht `deliverySequence` aufsteigend. Daher ist der Datensatz am Kopf des `outputBuffer` immer der führende unbestätigte `DeliveryRecord`. Der persistente `DeliveryRecord` bleibt die maßgebliche Quelle für den Zustellzustand. Der `outputBuffer` bestimmt ausschließlich, welcher `DeliveryRecord` als nächster betrachtet werden darf.

Ein `DeliveryRecord` wird erst aus dem `outputBuffer` entfernt, nachdem sein Status `acknowledged` gesetzt wurde.

`DeliveryRecord`-Datensätze für Telemetrierohdaten erhalten ihre `deliverySequence` bei der Annahme der MQTT-Message. `DeliveryRecords` für transformierte Telemetrie erhalten ihre `deliverySequence`, sobald die Verarbeitung für transformierte Telemetrie abgeschlossen wurde. Alle `DeliveryRecords` verwenden eine gemeinsame globale `deliverySequence`. Dadurch wird eine globale Zustellreihenfolge für Original- und transformierte Telemetrie gewährleistet.



Der BufferManager stellt dem DeliveryService die deliveryId am Kopf des outputBuffers bereit. Der zugehörige persistente DeliveryRecord bestimmt, ob aktuell eine Zustellung ausgeführt werden darf. Ein führender DeliveryRecord mit status in-flight, retry-wait oder blocked verbleibt am Kopf des outputBuffer und blockiert dadurch sämtliche spätere DeliveryRecord-Datensätze unabhängig von deren deliveryType.

Das Database Package stellt dem DeliveryService den persistenten DeliveryRecord, die referenzierten Telemetriedaten und den ReceiverState zur Verfügung.

Die Erzeugung des DeliveryRecord-Datensatzes für die Rohdaten ist unabhängig von der Transformation. Die spätere HTTP-Zustellung von Telemetrierohdaten und transformierter Telemetrie unterliegt jedoch der gemeinsamen globalen deliverySequence.

Daher gilt:

- Originalnachricht N kann vor der transformierten Nachricht N eintreffen.
- Originalnachricht N+1 kann vor der transformierten Nachricht N eintreffen.

Der Overwatch-Service verbindet beide Darstellungen über das Attribut `messageId`.

Normative Zustandsübergänge

Die folgende Tabelle definiert die maßgeblichen Zustandsübergänge. Ablaufdiagramme und Funktionsbeschreibungen konkretisieren diese Regeln, dürfen ihnen jedoch nicht widersprechen.

Ereignis	Bedingung	Persistente Änderung	Folge
Zustellversuch starten	führender <code>DeliveryRecord</code> hat <code>status = pending</code> , <code>ReceiverState.availabilityStatus = available</code>	<code>DeliveryRecord.status = in-flight</code> , <code>DeliveryRecord.attemptCount++</code> , <code>DeliveryRecord.lastAttemptAt = now</code> , <code>DeliveryRecord.nextAttemptAt = null</code>	genau einen HTTP-Zustellversuch starten
HTTP 2xx-Response	<code>in-flight</code>	<code>DeliveryRecord.status = acknowledged</code> , <code>DeliveryRecord.acknowledgedAt = now</code>	nach Commit Kopf aus <code>outputBuffer</code> entfernen ggf. nächsten <code>DeliveryRecord</code> verarbeiten.
temporärer Fehler	<code>attemptCount < 13</code>	<code>DeliveryRecord.status = retry-wait</code> , <code>DeliveryRecord.nextAttemptAt</code> gemäß Retry-Schema	<code>DeliveryRecord</code> bleibt am Kopf und der Retry-Timer muss aktualisiert werden

Ereignis	Bedingung	Persistente Änderung	Folge
temporärer Fehler	attemptCount = 13	<code>DeliveryRecord.status = pending,</code> <code>DeliveryRecord.nextAttemptAt = null</code> <code>ReceiverState.availabilityStatus = unavailable,</code> <code>ReceiverState.unavailableSince = now,</code> <code>ReceiverState.nextHealthCheckAt =</code> Zeitpunkt des nächsten HealthChecks	<code>DeliveryRecord</code> bleibt am Kopf des <code>outputBuffer</code> . die reguläre Zustellung wird ausgesetzt.
nicht automatisch wiederholbare Fehler	HTTP 3xx bzw. nicht temporärer HTTP 4xx Fehler	<code>DeliveryRecord.status = blocked</code>	<code>DeliveryRecord</code> bleibt am Kopf des <code>outputBuffer</code> und blockiert spätere <code>DeliveryRecord</code> -Datensätze
Retry-Zeitpunkt erreicht	führender <code>DeliveryRecord.status = retry-wait</code>	<code>DeliveryRecord.status = pending,</code> <code>DeliveryRecord.nextAttemptAt = null</code>	<code>DeliveryService.processNextDelivery</code> aufrufen
administrative Freigabe	<code>DeliveryRecord.status = blocked</code>	<code>DeliveryRecord.status = pending,</code> <code>DeliveryRecord.nextAttemptAt = null</code> <code>DeliveryRecord.attemptCount = 0</code>	<code>DeliveryService.processNextDelivery</code> aufrufen

Ereignis	Bedingung	Persistente Änderung	Folge
HealthCheck erfolgreich	<code>ReceiverState = unavailable</code>	<code>ReceiverState.availabilityStatus = available,</code> <code>ReceiverState.unavailableSince = null,</code> <code>ReceiverState.nextHealthCheckAt = null,</code> führender <code>DeliveryRecord.attemptCount = 0</code>	<code>DeliveryService.processNextDelivery</code> aufrufen
HealthCheck fehlgeschlagen	<code>ReceiverState = unavailable</code>	<code>ReceiverState.availabilityStatus = unavailable,</code> <code>ReceiverState.nextHealthCheckAt =</code> Zeitpunkt des nächsten HealthChecks	keine reguläre Zustellung

Wesentliche Funktionen

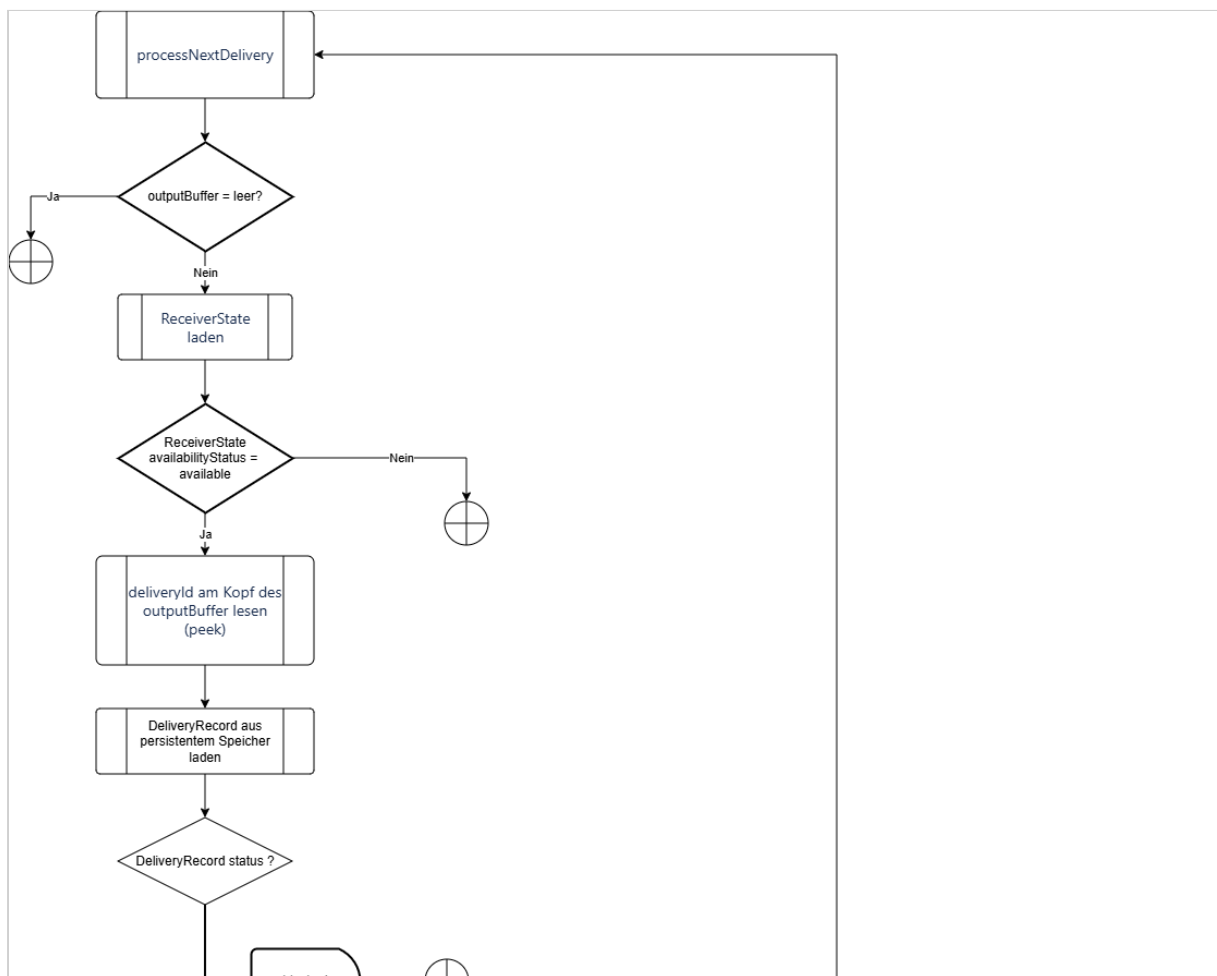
Funktion	Aufgabe
<code>processNextDelivery</code>	Koordiniert die Verarbeitung des führenden <code>DeliveryRecord</code> am Kopf des <code>outputBuffer</code> . Die Funktion prüft den <code>ReceiverState</code> und den persistenten Zustand dieses <code>DeliveryRecord</code> , lädt bei einer zulässigen Zustellung die benötigten persistenten Daten und startet genau einen Zustellversuch.
<code>buildHttpMessage</code>	Erzeugt aus dem <code>DeliveryRecord</code> und den referenzierten Telemetriedaten die an den Overwatch-Service zu sendende HTTP-Message. Der Aufbau hängt vom <code>deliveryType</code> ab.
<code>sendHttpMessage</code>	Führt genau einen HTTP-POST-Zustellversuch an den Overwatch-Service aus und liefert das Ergebnis des Versuchs zurück. Die Funktion führt selbst keine Wiederholungsversuche aus.

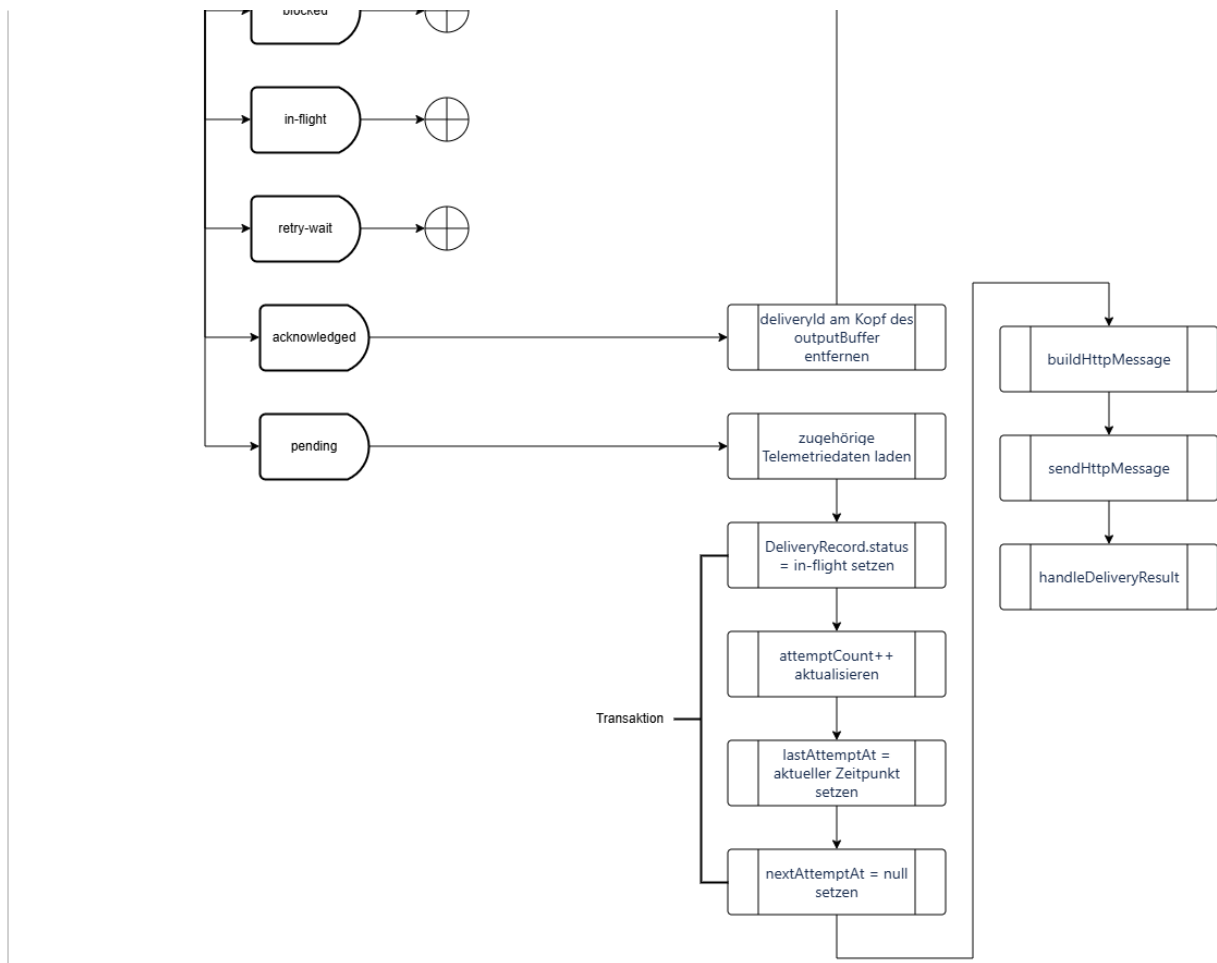
Funktion	Aufgabe
handleDeliveryResult	Bewertet das Ergebnis eines Zustellversuchs und aktualisiert den persistenten DeliveryRecord. Abhängig vom Ergebnis wird die Zustellung bestätigt, für eine Wiederholung vorgemerkt oder blockiert. Bei länger anhaltender Nichterreichbarkeit wird zusätzlich der ReceiverState aktualisiert.
checkReceiverHealth	Prüft die Erreichbarkeit des Overwatch-Service, solange ReceiverState.availabilityStatus = unavailable ist. Bei erfolgreicher Prüfung wird die reguläre Zustellung wieder freigegeben.

processNextDelivery

processNextDelivery ist die zentrale koordinierende Funktion des DeliveryService. Sie wird ausgeführt, wenn eine reguläre Zustellung durchgeführt werden kann und ein DeliveryRecord zur Verarbeitung ansteht.

Der Ablauf ist:





Der `outputBuffer` bestimmt die globale Reihenfolge der Zustellungen. Sein Kopf entspricht aufgrund der Buffer-Invariante stets dem noch nicht bestätigten `DeliveryRecord` mit der kleinsten `deliverySequence`. Der persistente `DeliveryRecord` bestimmt dagegen den aktuellen Lebenszykluszustand der Zustellung. Nur ein führender `DeliveryRecord` mit `status = pending` darf einen neuen HTTP-Zustellversuch starten. `retry-wait`, `blocked` und `in-flight` verbleiben im Kopf des `outputBuffer` und verhindern dadurch, dass ein späterer `DeliveryRecord` den führenden Datensatz überholt.

buildHttpMessage

`buildHttpMessage` erzeugt aus dem persistenten `DeliveryRecord` und den über `messageId` referenzierten Telemetriedaten die HTTP-Message für den Overwatch-Service.

Die Funktion unterscheidet anhand von `DeliveryRecord.deliveryType` zwischen:

- `original` – Zustellung der Telemetrierohdaten;
- `canonical` – Zustellung der transformierten Telemetriedaten.

Die folgende Tabelle beschreibt die an den Overwatch-Service gesendeten HTTP-Messages.

HTTP-Message für Telemetrierohdaten

Attribut	Erklärung
<code>deliveryId</code>	Idempotenzschlüssel dieser Zustellung an den Empfänger (siehe Abschnitt "Idempotenz").
<code>deliveryType</code>	Fester Wert <code>original</code> .
<code>deliverySequence</code>	Fortlaufende Zahl, welche eine eindeutige Reihenfolge der DeliveryRecords garantiert.
<code>messageId</code>	Korreliert die Zustellung der Telemetrierohdaten mit der Zustellung der transformierten Telemetrie.
<code>encodedTopic</code>	Überträgt den Quellnamensraum und den verlustfrei codierten ursprünglichen MQTT-Topic.
<code>payload</code>	Enthält die Telemetrierohdaten.
<code>createdAt</code>	Zeitpunkt, zu dem dieser Record erstellt wurde.
<code>sentAt</code>	Zeitpunkt, zu dem dieser konkrete HTTP-Zustellversuch gestartet wurde. Der Wert wird nicht zur Duplikaterkennung herangezogen. Der Wert entspricht <code>DeliveryRecord.lastAttemptAt</code> des aktuellen Zustellversuchs.

HTTP-Message für transformierte Telemetrie

Attribut	Erklärung
<code>deliveryId</code>	Idempotenzschlüssel dieser Zustellung an den Empfänger (siehe Abschnitt "Idempotenz").
<code>deliveryType</code>	Fester Wert <code>canonical</code> .
<code>deliverySequence</code>	Fortlaufende Zahl, welche eine eindeutige Reihenfolge der <code>DeliveryRecords</code> garantiert.
<code>messageId</code>	Korreliert die Zustellung der transformierten Telemetrie mit der Zustellung der Telemetrierohdaten.
<code>deviceId</code>	Identifiziert die registrierte Drohne.
<code>payload</code>	Enthält die transformierten Telemetriedaten.
<code>createdAt</code>	Zeitpunkt, zu dem dieser Record erstellt wurde.
<code>sentAt</code>	Zeitpunkt, zu dem dieser konkrete HTTP-Zustellversuch gestartet wurde. Der Wert wird nicht zur Duplikaterkennung herangezogen. Der Wert entspricht <code>DeliveryRecord.lastAttemptAt</code> des aktuellen Zustellversuchs.

sendHttpRequest

`sendHttpRequest` führt genau einen HTTP-POST-Zustellversuch aus. Die Funktion erhält die von `buildHttpRequest` erzeugte HTTP-Message und überträgt diese an den Overwatch-Service. `sendHttpRequest` entscheidet nicht selbst über einen erneuten Zustellversuch. Das Ergebnis wird an `handleDeliveryResult` übergeben.

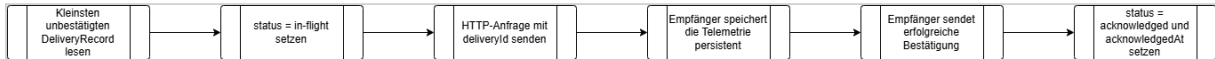
handleDeliveryResult

`handleDeliveryResult` bewertet das Ergebnis von `sendHttpRequest`, klassifiziert die HTTP-Antwort beziehungsweise den aufgetretenen Fehler und führt den in der Tabelle "Normative Zustandsübergänge" festgelegten Zustandsübergang aus. Ein `DeliveryRecord` wird nur nach einem erfolgreichen persistenten Wechsel auf `acknowledged` aus dem `outputBuffer` entfernt. Bei allen anderen Zuständen verbleibt seine `deliveryId` am Kopf des `outputBuffer`. `handleDeliveryResult` verarbeitet genau das Ergebnis des aktuellen Zustellversuchs. Ein

späterer Retry wird ausschließlich über den persistenten Zustand und den Retry-Timer des `DeLiveryService` ausgelöst.

HTTP-Zustellablauf

Für die Zustellung von Telemetriedaten an den Overwatch-Service sind folgende Schritte notwendig:



Eine erfolgreiche HTTP-Antwort (HTTP 2xx) ist nur gültig, wenn der Empfänger garantiert, dass sie erst nach der persistenten Speicherung zurückgegeben wird.

Bestätigung / Acknowledgement

Der Empfänger darf erst dann eine erfolgreiche Antwort zurückgeben, wenn:

1. Die vollständige Anfrage empfangen wurde.
2. `deliveryId` auf ein Duplikat geprüft wurde.
3. Die Telemetrie persistent gespeichert wurde.
4. Die Empfängertransaktion erfolgreich abgeschlossen wurde.

Eine Bestätigung nach ausschließlicher Ablage der Nachricht in einem flüchtigen Buffer ist unzureichend.

Erfolglose Zustellung

Für jeden HTTP-Zustellversuch gilt ein Gesamtzeitlimit von 30 Sekunden. Es beginnt unmittelbar vor dem Verbindungsaufbau und endet, sobald die vollständige HTTP-Response empfangen wurde. Das Zeitlimit umfasst Verbindungsaufbau, gegebenenfalls TLS-Handshake, Übertragung der Anfrage und Warten auf die Antwort. Wird innerhalb dieser 30 Sekunden keine vollständige erfolgreiche Antwort empfangen, gilt der Zustellversuch als fehlgeschlagen.

Ein Zustellversuch gilt als erfolgreich, wenn der Overwatch-Service eine erfolgreiche HTTP-2xx-Antwort sendet.

Folgende Fehler gelten als temporär und werden erneut versucht:

- Netzwerkfehler,
- Überschreitung des HTTP-Timeouts,
- HTTP 408 Request Timeout,
- HTTP 429 Too Many Requests,
- HTTP 5xx

Nicht automatisch wiederholbare Fehler führen gemäß der Tabelle "Normative Zustandsübergänge" zum Zustand `blocked`. Die administrative Freigabe eines blockierten Datensatzes erfolgt ebenfalls gemäß dieser Tabelle.

Der Response-Status und eine begrenzte Fehlerbeschreibung werden für die Diagnose im `DeLiveryRecord` gespeichert.

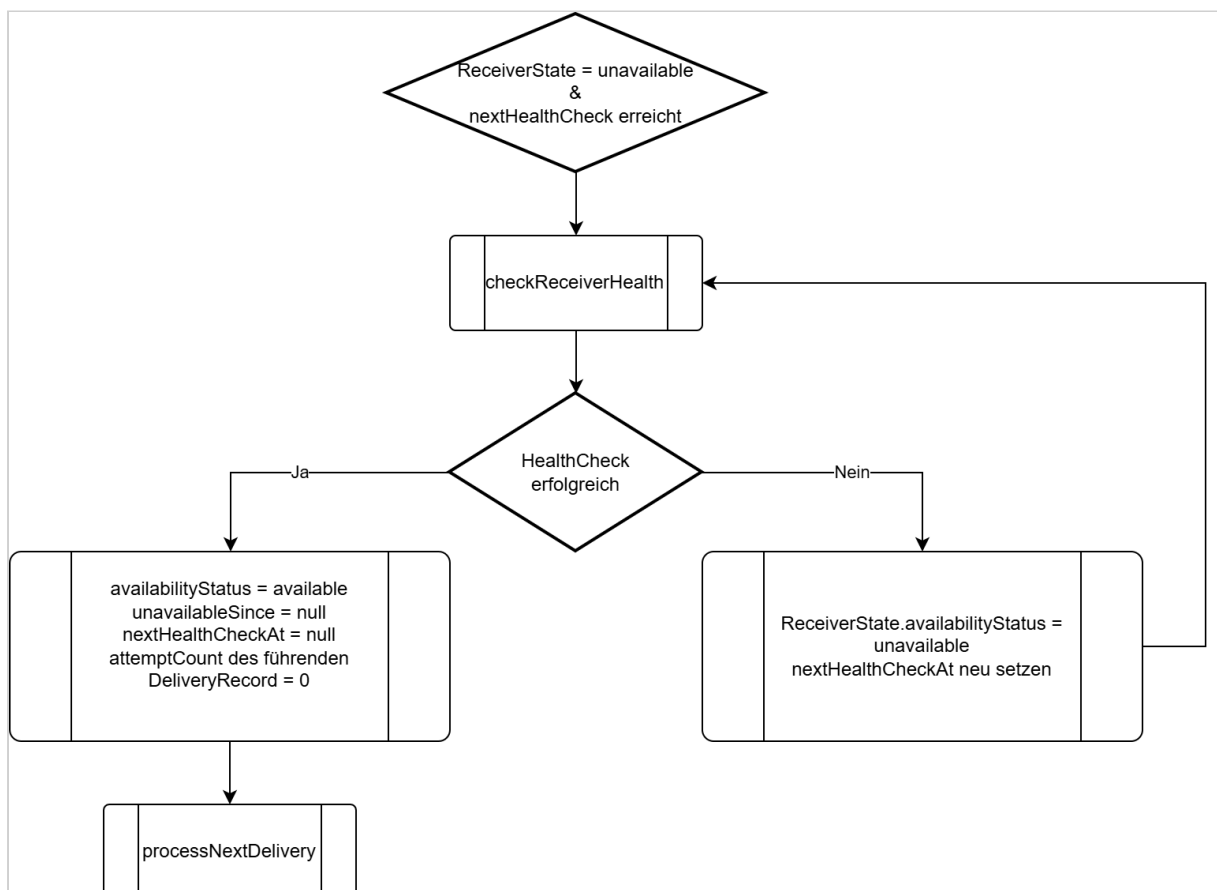
Die erste HTTP-Zustellung wird ohne vorherige Wartezeit ausgeführt. Nach einem temporären Fehler bei den Zustellversuchen 1 - 12 setzt der `DeliveryService DeliveryRecord.nextAttemptAt` wie folgt:

- nach den Versuchen 1–3: aktueller Zeitpunkt + 0,5 Sekunden,
- nach den Versuchen 4–6: aktueller Zeitpunkt + 1 Sekunde,
- nach den Versuchen 7–9: aktueller Zeitpunkt + 2 Sekunden,
- nach den Versuchen 10–12: aktueller Zeitpunkt + 10 Sekunden.

Versuch 13 ist der letzte reguläre Zustellversuch eines Zustellzyklus. Schlägt auch dieser Versuch mit einem temporären Fehler fehl, wird der Receiver gemäß der Tabelle "Normative Zustandsübergänge" auf `ReceiverState.availabilityStatus = unavailable` gesetzt.

checkReceiverHealth

Solange `ReceiverState.availabilityStatus = unavailable` ist, werden keine regulären Zustellversuche gestartet. Es werden ausschließlich die geplanten HealthChecks durchgeführt. Das Ergebnis eines HealthChecks wird gemäß der Tabelle "Normative Zustandsübergänge" verarbeitet.

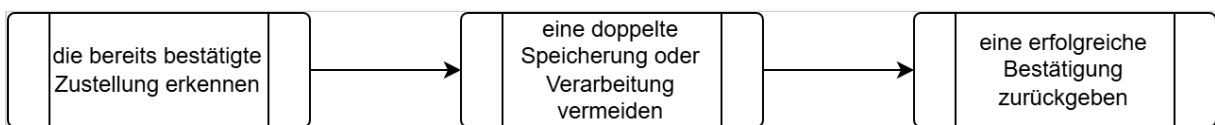


Idempotenz

Die `deliveryId` ist der **Idempotenzschlüssel** des Empfängers, d. h. anhand dieses Schlüssels kann der Empfänger erkennen, ob er exakt diese Zustellung schon einmal empfangen hat oder nicht.

Der Overwatch-Service führt die Duplikaterkennung ausschließlich anhand von `deliveryId` durch. Andere Attribute der erneut zugestellten HTTP-Message werden nicht zur Bestimmung der Identität der Zustellung verwendet. Dies gilt insbesondere für `sentAt`, da dieser Wert bei jedem Zustellversuch unterschiedlich sein kann. Wurde eine `deliveryId` bereits erfolgreich verarbeitet, darf eine erneute Zustellung mit derselben `deliveryId` keinen zweiten Verarbeitungsvorgang auslösen. Der Overwatch-Service muss für das erkannte Duplikat erneut eine erfolgreiche HTTP-Response senden.

Wenn eine doppelte `deliveryId` empfangen wird, muss der Empfänger:



`messageId`

Doppelte HTTP-Zustellungen bleiben möglich, wenn der Empfänger eine Nachricht speichert, die Antwort jedoch verloren geht. Die Idempotenz auf Empfängerseite macht solche Wiederholungen sicher.

Die ist der Schlüssel, der eine Verbindung zwischen Original- und transformierter Telemetrie herstellt. Sie ist nicht der Idempotenzschlüssel eines Zustellversuchs.

Database-Package

Das [Database-Package \(see page 89\)](#) kapselt sämtliche Datenbankzugriffe des Telemetry Data Adapters. Dazu gehören sowohl der persistente Nachrichtenspeicher als auch die für Identifikation, Validierung und Transformation benötigten Repository-Daten, beispielsweise `Message Definitions`, Validierungsschemata und Transformationsdefinitionen. Das [Database-Package \(see page 89\)](#) enthält keine fachliche Verarbeitungslogik. Die aufrufende Komponente entscheidet, welche Daten gelesen oder verändert werden müssen. Für Änderungen des persistenten Nachrichtenzustands führt das [Database-Package \(see page 89\)](#) die angeforderten Operationen atomar aus. Repository-Daten werden den fachlichen Packages über reine Leseoperationen zur Verfügung gestellt.

Repository-Einträge, die über persistente `Message Definition` - beziehungsweise `ErrorCaseProcessingContext` -Referenzen benötigt werden, müssen über ihren Identifikator weiterhin eindeutig lesbar bleiben. Fachliche Änderungen werden deshalb als neue Repository-Version mit neuem Identifikator angelegt und überschreiben keine bereits referenzierte Version.

B. Overwatch-Service - White-Box View

C. Overwatch-UI - White-Box View

D. Overwach Topic Routing Plugin - White-Box View

3.4 Nutzer-/Rollenkonzept

Rollen

Rolle	Beschreibung	Hauptaufgaben
Administrator	Vollzugriff auf alle Systemfunktionen. Der erste Administrator ist geschützt (kann nicht gelöscht/deaktiviert werden).	Nutzerverwaltung, Systemkonfiguration, alle Missionsfunktionen
Commander	Missions- und Device-Verwalter ohne Nutzerverwaltung.	Missionen verwalten, Operatoren zuweisen, Devices verwalten
Operator	Operativer Nutzer mit Steuerungsrechten für Drohnen.	Drohnen steuern, Screenshots erstellen, Kartenobjekte zeichnen
Observer	Reiner Lesenzugriff ohne Schreibrechte.	Video/Telemetrie ansehen, temporär messen, Fokusmodus
Share-Link-Nutzer	Externe Nutzer mit zeitlich begrenztem Zugriff via Share-Link.	Nur freigegebene Funktionen (Video, Karte, Fokusmodus, Messen)

Rollen-/Rechtematrix

Berechtigung	Administrator	Commander	Operator	Observer	Share-Link-Nutzer
Nutzer erstellen	✓	✗	✗	✗	✗
Nutzer bearbeiten	✓	✗	✗	✗	✗
Nutzer löschen	✓	✗	✗	✗	✗
Passwörter zurücksetzen	✓	✗	✗	✗	✗
Share-Links erstellen	✓	✓	✗	✗	✗
Share-Links widerrufen	✓	✓	✗	✗	✗
Devices aktivieren	✓	✓	✗	✗	✗

Berechtigung	Administrator	Commander	Operator	Observer	Share-Link-Nutzer
Devices deaktivieren	✓	✓	✗	✗	✗
Devices in Wartung setzen	✓	✓	✗	✗	✗
Streams bereinigen	✓	✓	✗	✗	✗
Screenshots löschen (alle)	✓	✓	✗	✗	✗
Missionen erstellen	✓	✓	✗	✗	✗
Missionen bearbeiten	✓	✓	✗	✗	✗
Missionen löschen	✓	✓	✗	✗	✗
Operatoren zuweisen	✓	✓	✗	✗	✗
Drohnen steuern	✓	✓	✓	✗	✗
Screenshots erstellen	✓	✓	✓	✗	✗
Screenshots löschen (eigene)	✓	✓	✓	✗	✗
Kartenobjekte zeichnen	✓	✓	✓	✗	✗
Kartenobjekte bearbeiten	✓	✓	✓	✗	✗
Kartenobjekte löschen	✓	✓	✓	✗	✗
Sperrflächen erstellen	✓	✓	✓	✗	✗
Chat senden	✓	✓	✓	✓	✗
Chat empfangen	✓	✓	✓	✓	✗
Temporär messen (Entfernung)	✓	✓	✓	✓	✓

Berechtigung	Administrator	Commander	Operator	Observer	Share-Link-Nutzer
Temporär messen (Fläche)	✓	✓	✓	✓	✓
Video lesen	✓	✓	✓	✓	✓ (freigegeben)
Telemetrie lesen	✓	✓	✓	✓	✓ (freigegeben)
Fokusmodus nutzen	✓	✓	✓	✓	✓
Report ansehen	✓	✓	✓	✓	✓ (freigegeben)